

TDD V1 — Section 1

Document Overview & Technical Scope

Document: Technical Design Document (TDD V1)

Section: 1

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

1.1 Purpose

This document defines the technical implementation blueprint for TODP (The Opportunity Discovery Platform).

TDD V1 converts approved architectural and implementation decisions into precise engineering specifications.

This document is intended for:

- Backend Engineers
- Frontend Engineers
- Database Engineers
- Infrastructure Engineers
- Security Engineers
- QA Engineers
- DevOps Engineers

The objective is to eliminate implementation ambiguity and provide a single authoritative technical reference for development.

1.2 Relationship To Other Documents

PRD V1

Defines business requirements.

ADD V1

Defines architectural philosophy and guiding principles.

DDD V1

Defines domain model and domain behavior.

SDD V1

Defines system architecture.

IAD V1

Defines implementation architecture.

TDD V1

Defines exact technical implementation specifications.

1.3 Technical Design Objectives

The technical design must:

- Preserve Evidence First
- Preserve Authority Before Recommendation
- Preserve Ranking Before Recommendation
- Preserve Auditability
- Preserve Traceability
- Preserve Reproducibility
- Preserve Transparency

The implementation must not introduce behavior that violates approved architecture.

1.4 Platform Scope

TODP is an evidence-driven opportunity discovery platform.

The platform supports:

1. Discover Online Opportunities
2. Discover Physical Opportunities
3. Create Something Similar
4. Create Something New

The platform operates across:

Institution



Local



District



State



National



International

1.5 Technology Baseline

Frontend

React

TypeScript

Backend

FastAPI

Python

Database

PostgreSQL

pgvector

Cache

Redis

Observability

OpenTelemetry

Prometheus

Grafana

Deployment

Containerized Infrastructure

1.6 Technical Architecture Layers

Presentation Layer

↓

API Layer



Application Layer



Domain Layer



Persistence Layer



Infrastructure Layer



Observability Layer

1.7 Engineering Principles

ENG-01

Explicit Design

ENG-02

Strong Typing

ENG-03

Separation of Concerns

ENG-04

Single Responsibility

ENG-05

Dependency Inversion

ENG-06

Testability

ENG-07

Observability

ENG-08

Governance Compliance

1.8 Repository Strategy

TODP uses a monorepo architecture.

Repository Structure:

frontend/

backend/

infrastructure/

docs/

tests/

scripts/

1.9 Development Standards

All code must satisfy:

Type Safety

Code Review

Testing

Security Validation

Observability Requirements

Governance Requirements

Documentation Requirements

1.10 Core Technical Modules

Module 1

Authentication

Module 2

Authorization

Module 3

Opportunity Management

Module 4

Authority Management

Module 5

Evidence Management

Module 6

Retrieval Engine

Module 7

Ranking Engine

Module 8

Recommendation Engine

Module 9

Governance Engine

Module 10

Audit Engine

Module 11

Observability Platform

1.11 Retrieval Architecture Scope

Retrieval uses:

BM25 Search

+

Vector Search

+

Hybrid Fusion

Purpose:

Generate candidate opportunity sets.

Retrieval does not perform ranking.

1.12 Ranking Architecture Scope

Ranking uses:

Money Cost

+

Time Cost

+

Distance Cost

Total Cost

=

Money Cost + Time Cost + Distance Cost

Lowest Total Cost ranks highest.

Ranking occurs before recommendation.

1.13 Recommendation Architecture Scope

Recommendations are generated only after:

Evidence Validation



Authority Verification



Ranking Completion

Recommendations cannot bypass ranking.

1.14 Authority Architecture Scope

Authority subsystem manages:

Authority Records

Authority Verification

Authority Confidence

Authority History

Authority Traceability

AI cannot act as authority.

1.15 Evidence Architecture Scope

Evidence subsystem manages:

Evidence Records

Evidence Validation

Evidence Versioning

Evidence Lineage

Evidence Traceability

Evidence Auditability

1.16 Auditability Requirements

Every critical action generates:

Audit Record

Audit records are immutable.

Audit records are searchable.

Audit records are reproducible.

1.17 Traceability Requirements

Every decision must support:

Source Tracking

Decision Tracking

Relationship Tracking

Historical Reconstruction

1.18 Reproducibility Requirements

Every critical output must support:

Replay

Version Tracking

Snapshot Preservation

Historical Reconstruction

1.19 Security Requirements

The technical implementation must support:

Authentication

Authorization

Encryption

Secret Management

Security Monitoring

Security Auditing

Incident Response

1.20 Performance Targets

API Requests

P95 < 300 ms

Search Operations

P95 < 2 seconds

Ranking Operations

P95 < 1 second

Recommendation Operations

P95 < 500 ms

Cache Reads

P95 < 5 ms

1.21 Scalability Targets

The platform must support:

Horizontal Scaling

Stateless Services

Independent Deployments

Service Isolation

Cache Scaling

Database Scaling

1.22 Observability Requirements

All critical workflows generate:

Metrics

Logs

Traces

Audit Records

Correlation IDs

1.23 Governance Requirements

All critical actions require:

Approval Controls

Audit Controls

Traceability Controls

Version Controls

Policy Controls

1.24 Technical Deliverables

TDD V1 shall define:

Repository Structures

Package Structures

Class Designs

API Contracts

DTO Designs

Database Schemas

Interface Definitions

Event Contracts

Testing Specifications

Technical Constraints

1.25 Technical Assumptions

Assumption 1

PostgreSQL remains primary source of truth.

Assumption 2

Redis remains cache only.

Assumption 3

Hybrid Retrieval remains mandatory.

Assumption 4

Ranking remains deterministic.

Assumption 5

Auditability remains mandatory.

1.26 Technical Constraints

TC-01

No sponsored content.

TC-02

No paid placement.

TC-03

No hidden ranking logic.

TC-04

AI is not an authority.

TC-05

Authority before recommendation.

TC-06

Evidence before recommendation.

TC-07

Ranking before recommendation.

TC-08

Traceability mandatory.

TC-09

Auditability mandatory.

TC-10

Reproducibility mandatory.

1.27 Success Criteria

TDD V1 is successful when:

Developers can implement TODP without architectural ambiguity.

Every critical component has a defined technical specification.

All implementation decisions remain aligned with approved architecture.

1.28 Section Summary

This section establishes the technical scope, implementation boundaries, engineering principles, architectural constraints, and technical objectives governing TODP implementation.

TDD V1 serves as the authoritative engineering blueprint translating approved architecture into executable technical design specifications.

TDD Progress

📄 Section 1 — Document Overview & Technical Scope

📄 Section 2 — Repository Technical Structure

TDD V1 — Section 2

Repository Technical Structure

Document: Technical Design Document (TDD V1)

Section: 2

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

2.1 Purpose

This section defines the physical repository structure of TODP.

The repository structure establishes:

- Source code organization
- Ownership boundaries
- Module separation

- Dependency direction
- Deployment packaging
- Testing organization

The repository structure is the foundation for all implementation activities.

2.2 Repository Philosophy

Repository organization must optimize for:

- Maintainability
- Scalability
- Discoverability
- Testability
- Governance
- Operational simplicity

The structure must support long-term platform evolution.

2.3 Repository Strategy

Repository Type:

Monorepo

Reasoning:

- Single source of truth
- Unified versioning
- Shared governance
- Shared CI/CD
- Shared documentation
- Simplified dependency management

2.4 Top-Level Repository Structure

todp/

```
├── frontend/  
├── backend/  
├── infrastructure/  
├── docs/  
├── tests/  
├── scripts/  
├── .github/  
├── docker/  
├── .env.example  
└── README.md
```

└─ LICENSE

2.5 Root Directory Responsibilities

frontend/

User Interface

backend/

Business Logic

infrastructure/

Infrastructure as Code

docs/

Project Documentation

tests/

Cross-System Testing

scripts/

Operational Utilities

.github/

CI/CD Pipelines

docker/

Container Assets

2.6 Frontend Repository Structure

frontend/

- |— public/
- |— src/
- |— tests/
- |— package.json
- |— tsconfig.json
- └─ vite.config.ts

2.7 Frontend Source Structure

frontend/src/

- |— app/
- |— pages/
- |— components/
- |— features/
- |— services/
- |— hooks/
- |— contexts/
- |— routes/
- |— types/
- |— utils/

└─ assets/

└─ styles/

2.8 Frontend Directory Responsibilities

app/

Application bootstrap

pages/

Route-level pages

components/

Reusable UI components

features/

Business features

services/

API communication

hooks/

React hooks

contexts/

Global state providers

routes/

Routing configuration

types/

TypeScript models

utils/

Utility functions

2.9 Backend Repository Structure

backend/

├─ app/

├─ migrations/

├─ tests/

├─ requirements/

├─ pyproject.toml

└─ alembic.ini

2.10 Backend Application Structure

backend/app/

- |— api/
- |— domains/
- |— services/
- |— repositories/
- |— schemas/
- |— models/
- |— security/
- |— governance/
- |— infrastructure/
- |— observability/
- └─ main.py

2.11 Backend Layer Responsibilities

api/

HTTP layer

domains/

Domain entities

services/

Business logic

repositories/

Persistence abstraction

schemas/

Request/Response DTOs

models/

Database models

security/

Authentication & authorization

governance/

Auditability & traceability

infrastructure/

Technical integrations

observability/

Telemetry

2.12 API Layer Structure

backend/app/api/

- |— auth/
- |— opportunities/
- |— authorities/
- |— evidence/
- |— retrieval/
- |— rankings/
- |— recommendations/
- |— governance/
- |— audits/
- └─ health/

2.13 Domain Structure

backend/app/domains/

- |— opportunity/
- |— authority/
- |— evidence/
- |— retrieval/
- |— ranking/
- |— recommendation/
- |— governance/
- └─ user/

2.14 Opportunity Domain Structure

domains/opportunity/

- |— entities.py
- |— value_objects.py
- |— constants.py
- |— policies.py
- └— exceptions.py

2.15 Authority Domain Structure

domains/authority/

- |— entities.py
- |— verification.py
- |— confidence.py
- |— policies.py
- └— exceptions.py

2.16 Evidence Domain Structure

domains/evidence/

- |— entities.py
- |— lineage.py
- |— validation.py
- |— policies.py
- └─ exceptions.py

2.17 Service Layer Structure

backend/app/services/

- |— authentication/
- |— authorization/
- |— opportunity/
- |— authority/
- |— evidence/
- |— retrieval/
- |— ranking/
- |— recommendation/
- |— governance/
- └─ audit/

2.18 Repository Layer Structure

backend/app/repositories/

- |— user_repository.py
- |— opportunity_repository.py
- |— authority_repository.py
- |— evidence_repository.py
- |— ranking_repository.py
- |— recommendation_repository.py
- └— audit_repository.py

2.19 Schema Layer Structure

backend/app/schemas/

- |— auth/
- |— opportunity/
- |— authority/
- |— evidence/
- |— retrieval/
- |— ranking/
- |— recommendation/

└─ governance/

└─ audit/

2.20 Model Layer Structure

backend/app/models/

└─ user.py

└─ opportunity.py

└─ authority.py

└─ evidence.py

└─ ranking.py

└─ recommendation.py

└─ audit.py

└─ governance.py

2.21 Security Structure

backend/app/security/

└─ authentication/

└─ authorization/

└─ permissions/

- └─ roles/
- └─ middleware/
- └─ tokens/

2.22 Governance Structure

backend/app/governance/

- └─ audit/
- └─ traceability/
- └─ lineage/
- └─ reproducibility/
- └─ compliance/

2.23 Infrastructure Structure

backend/app/infrastructure/

- └─ database/
- └─ cache/
- └─ messaging/
- └─ search/
- └─ storage/

└─ external/

2.24 Observability Structure

backend/app/observability/

└─ metrics/

└─ tracing/

└─ logging/

└─ dashboards/

└─ alerts/

2.25 Database Migration Structure

backend/migrations/

└─ versions/

└─ env.py

└─ script.py.mako

2.26 Infrastructure Repository Structure

infrastructure/

└─ environments/

└─ networking/

└─ compute/

└─ database/

└─ cache/

└─ observability/

└─ security/

└─ modules/

2.27 Environment Structure

infrastructure/environments/

└─ dev/

└─ test/

└─ stage/

└─ prod/

2.28 Documentation Structure

docs/

- |— PRD/
- |— ADD/
- |— DDD/
- |— SDD/
- |— IAD/
- |— TDD/
- |— API/
- |— RUNBOOKS/
- |— GOVERNANCE/
- └─ RELEASES/

2.29 Test Repository Structure

tests/

- |— unit/
- |— integration/
- |— contract/
- |— security/
- |— governance/
- |— performance/
- └─ e2e/

2.30 Unit Test Structure

tests/unit/

└─ backend/

└─ frontend/

└─ services/

└─ repositories/

└─ domains/

2.31 Integration Test Structure

tests/integration/

└─ api/

└─ database/

└─ cache/

└─ retrieval/

└─ ranking/

└─ recommendation/

2.32 E2E Test Structure

tests/e2e/

└─ discovery/

└─ authority/

└─ evidence/

└─ governance/

└─ recommendations/

2.33 Script Repository Structure

scripts/

└─ setup/

└─ maintenance/

└─ migration/

└─ backup/

└─ restore/

└─ diagnostics/

2.34 Docker Structure

docker/

├─ frontend/

├─ backend/

├─ postgres/

├─ redis/

├─ prometheus/

└─ grafana/

2.35 Dependency Rules

Allowed Direction:

API

↓

Services

↓

Repositories

↓

Infrastructure

Not Allowed:

Repositories

↓

Services

Infrastructure

↓

Domain Logic

2.36 Ownership Rules

Each module must have:

Technical Owner

Business Owner

Documentation Owner

Operational Owner

2.37 Repository Governance Rules

REP-GOV-01

Domain boundaries must be preserved.

REP-GOV-02

Module ownership must be defined.

REP-GOV-03

Cross-layer violations are prohibited.

REP-GOV-04

Repository structure changes require review.

REP-GOV-05

Critical modules require documentation.

REP-GOV-06

Repository structure remains version controlled.

2.38 Architectural Constraints

AC-REP-01

Business logic cannot exist in API routes.

AC-REP-02

Database access cannot bypass repositories.

AC-REP-03

Infrastructure cannot contain business logic.

AC-REP-04

Security controls cannot be bypassed.

AC-REP-05

Governance controls cannot be bypassed.

AC-REP-06

Repository structure changes require approval.

2.39 Section Summary

This section defines the physical repository organization, ownership model, dependency rules, module boundaries, and governance controls for TODP.

The repository structure establishes the implementation foundation that all future development must follow.

TDD Progress

▢ Section 1 — Document Overview & Technical Scope

▢ Section 2 — Repository Technical Structure

▢ Section 3 — Backend Package Design

TDD V1 — Section 3

Backend Package Design

Document: Technical Design Document (TDD V1)

Section: 3

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

3.1 Purpose

This section defines the backend package design for TODP.

The backend package architecture establishes:

- Package boundaries
- Dependency rules
- Service ownership

- Domain isolation
- Technical organization
- Code scalability

The package structure must support long-term maintainability and independent evolution of platform capabilities.

3.2 Backend Architecture Style

Architecture Pattern:

Layered Architecture

+

Domain Driven Design

+

Service-Oriented Design

The backend is organized around domains rather than database tables.

3.3 Backend Technology Stack

Language:

Python

Version:

Python 3.11+

Framework:

FastAPI

ORM:

SQLAlchemy

Validation:

Pydantic

Migration Tool:

Alembic

Authentication:

JWT

3.4 Backend Application Root

backend/app/

├─ api/

├─ domains/

├─ services/

├─ repositories/

├─ schemas/

├─ models/

├─ security/

├─ governance/

├─ infrastructure/

└─ observability/

└─ main.py

3.5 Package Responsibilities

api/

Transport Layer

domains/

Business Concepts

services/

Business Logic

repositories/

Persistence Access

schemas/

DTO Layer

models/

Database Layer

security/

Access Control

governance/

Auditability & Traceability

infrastructure/

External Systems

observability/

Telemetry

3.6 Application Entry Point

File:

main.py

Responsibilities:

Application Startup

Dependency Registration

Router Registration

Middleware Registration

Lifecycle Events

Health Registration

3.7 Application Bootstrap Flow

Application Start

↓

Load Configuration

↓

Initialize Database

↓

Initialize Redis

↓

Initialize Telemetry

↓

Register Middleware

↓

Register Routers

↓

Application Ready

3.8 API Package Design

backend/app/api/

|— auth/

|— opportunities/

|— authorities/

|— evidence/

|— retrieval/

|— rankings/

|— recommendations/

|— governance/

└─ audits/

└─ health/

3.9 API Package Responsibilities

Each API package contains:

router.py

dependencies.py

validators.py

exceptions.py

responses.py

3.10 API Router Standard

Example:

backend/app/api/opportunities/router.py

Contains:

HTTP Endpoints Only

No Business Logic

No Database Logic

No Ranking Logic

3.11 Domain Package Design

backend/app/domains/

├─ opportunity/

├─ authority/

├─ evidence/

├─ retrieval/

├─ ranking/

├─ recommendation/

├─ governance/

└─ user/

3.12 Domain Package Structure

Example:

domains/opportunity/

├── entities.py

├── value_objects.py

├── constants.py

├── policies.py

└── exceptions.py

3.13 Domain Responsibilities

Domain layer owns:

Business Concepts

Business Rules

Policies

Constraints

Value Objects

Domain Exceptions

Domain layer does not know:

Databases

HTTP

Redis

FastAPI

Infrastructure

3.14 Service Package Design

backend/app/services/

└─ authentication/

└─ authorization/

└─ opportunity/

└─ authority/

└─ evidence/

└─ retrieval/

└─ ranking/

└─ recommendation/

└─ governance/

└─ audit/

3.15 Service Layer Responsibilities

Services execute:

Use Cases

Business Workflows

Validation Coordination

Repository Coordination

Audit Generation

Event Publication

3.16 Service Package Structure

Example:

services/opportunity/

├─ create_service.py

├─ update_service.py

├─ delete_service.py

├─ search_service.py

└─ query_service.py

3.17 Service Design Rules

Services may access:

Repositories

Domain Objects

DTOs

Events

Audit Services

Services may not access:

HTTP Requests Directly

Database Sessions Directly

External APIs Directly

3.18 Repository Package Design

backend/app/repositories/

Purpose:

Persistence Abstraction

Repository Pattern

3.19 Repository Structure

repositories/

├─ user_repository.py

├─ opportunity_repository.py

├─ authority_repository.py

├─ evidence_repository.py

├─ ranking_repository.py

├─ recommendation_repository.py

└─ audit_repository.py

3.20 Repository Responsibilities

Repositories own:

CRUD Operations

Queries

Transactions

Persistence Mapping

Repositories do not own:

Business Logic

Ranking Logic

Recommendation Logic

3.21 Schema Package Design

backend/app/schemas/

Purpose:

Request DTOs

Response DTOs

Internal DTOs

Validation Models

3.22 Schema Structure Example

schemas/opportunity/

└─ create_request.py

└─ update_request.py

└─ response.py

└─ filters.py

3.23 Schema Rules

Schemas use:

Pydantic

Type Validation

Field Validation

Serialization

Schemas do not contain business logic.

3.24 Model Package Design

backend/app/models/

Purpose:

Database Mapping Layer

3.25 Model Structure Example

models/

|— user.py

|— opportunity.py

|— authority.py

|— evidence.py

|— ranking.py

|— recommendation.py

|— audit.py

3.26 Model Responsibilities

Models contain:

Table Definitions

Column Definitions

Relationships

Indexes

Constraints

Models do not contain business workflows.

3.27 Security Package Design

backend/app/security/

├─ authentication/

├─ authorization/

├─ middleware/

├─ permissions/

├─ roles/

└─ tokens/

3.28 Security Responsibilities

Authentication

Authorization

Permission Resolution

Role Resolution

JWT Validation

Security Middleware

3.29 Governance Package Design

backend/app/governance/

└─ audit/

└─ traceability/

└─ lineage/

└─ reproducibility/

└─ compliance/

3.30 Governance Responsibilities

Audit Generation

Trace Generation

Lineage Generation

Replay Support

Compliance Validation

3.31 Infrastructure Package Design

backend/app/infrastructure/

└─ database/

└─ cache/

└─ messaging/

└─ search/

└─ storage/

└─ external/

3.32 Infrastructure Responsibilities

Database Connections

Redis Connections

Search Engines

Message Transport

External Integrations

Storage Integrations

Infrastructure never owns business rules.

3.33 Observability Package Design

backend/app/observability/

- └─ metrics/
- └─ tracing/
- └─ logging/
- └─ dashboards/
- └─ alerts/

3.34 Observability Responsibilities

Metrics

Logs

Traces

Alerts

Telemetry Registration

Correlation Tracking

3.35 Dependency Flow

Allowed:

API



Services



Repositories



Infrastructure

Domains are shared by services.

3.36 Forbidden Dependencies

Repositories

→ Services

?

Infrastructure

→ Services

?

Infrastructure

→ Domains

?

API

→ Database Directly

?

3.37 Dependency Injection Strategy

FastAPI Dependency Injection

Used For:

Services

Repositories

Security Components

Configuration

Telemetry

3.38 Exception Architecture

Exception Types:

ValidationException

AuthenticationException

AuthorizationException

BusinessRuleException

RepositoryException

InfrastructureException

3.39 Package Naming Standards

All backend packages use:

snake_case

Examples:

authority_verification

evidence_validation

recommendation_engine

3.40 Package Testing Requirements

Every package requires:

Unit Tests

Integration Tests

Observability Validation

Security Validation

Governance Validation

3.41 Package Ownership Requirements

Each package must define:

Technical Owner

Business Owner

Reviewer

Operational Owner

3.42 Governance Rules

PKG-GOV-01

Domain boundaries must remain preserved.

PKG-GOV-02

Business logic belongs only in services and domains.

PKG-GOV-03

Repositories own persistence only.

PKG-GOV-04

Infrastructure owns integrations only.

PKG-GOV-05

Security controls are mandatory.

PKG-GOV-06

Governance controls are mandatory.

3.43 Architectural Constraints

AC-PKG-01

Business logic cannot exist in API routes.

AC-PKG-02

Repositories cannot contain business workflows.

AC-PKG-03

Infrastructure cannot contain business rules.

AC-PKG-04

Direct database access is prohibited outside repositories.

AC-PKG-05

Security controls cannot be bypassed.

AC-PKG-06

Governance controls cannot be bypassed.

3.44 Section Summary

This section defines the backend package architecture, dependency rules, ownership boundaries, service organization, repository design, and governance constraints.

The backend package structure provides a scalable, maintainable, and governance-compliant foundation for TODP implementation.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

TDD V1 — Section 4

Frontend Package Design

Document: Technical Design Document (TDD V1)

Section: 4

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

4.1 Purpose

This section defines the frontend package design for TODP.

The frontend architecture establishes:

- UI package organization
- Component boundaries
- State management strategy
- API integration patterns
- Routing structure
- Frontend governance rules

The frontend must remain scalable, maintainable, testable, and aligned with TODP architectural principles.

4.2 Frontend Architecture Style

Architecture Pattern:

Feature-Based Architecture

+

Component-Based Architecture

+

Service Layer Architecture

The frontend is organized around business capabilities rather than pages alone.

4.3 Frontend Technology Stack

Framework:

React

Language:

TypeScript

Build Tool:

Vite

Routing:

React Router

HTTP Client:

Axios

State Management:

React Context

Custom Hooks

Validation:

Zod

Testing:

Vitest

React Testing Library

4.4 Frontend Root Structure

frontend/

├─ public/

├─ src/

```
├─ tests/
├─ package.json
├─ tsconfig.json
├─ vite.config.ts
└─ .env
```

4.5 Frontend Source Structure

frontend/src/

```
├─ app/
├─ pages/
├─ components/
├─ features/
├─ services/
├─ hooks/
├─ contexts/
├─ routes/
├─ types/
├─ utils/
├─ assets/
└─ styles/
```

4.6 Frontend Layer Responsibilities

app/

Application bootstrap

pages/

Route-level screens

components/

Reusable UI components

features/

Business functionality

services/

API communication

hooks/

Reusable logic

contexts/

Shared application state

routes/

Routing configuration

types/

TypeScript contracts

utils/

Utility functions

4.7 Application Bootstrap Structure

frontend/src/app/

├─ App.tsx

├─ providers.tsx

├─ configuration.ts

└─ initialization.ts

4.8 Application Startup Flow

Application Load

↓

Configuration Load

↓

Context Initialization

↓

Authentication Initialization

↓

Route Registration

↓

Application Ready

4.9 Page Structure

frontend/src/pages/

|— Home/

|— Search/

|— Opportunity/

|— Authority/

|— Evidence/

|— Recommendation/

|— Governance/

|— Login/

|— Error/

4.10 Page Responsibilities

Pages:

Compose UI

Coordinate features

Manage route behavior

Trigger workflows

Pages do not contain business logic.

4.11 Component Architecture

frontend/src/components/

├─ common/

├─ forms/

├─ tables/

├─ cards/

├─ layout/

├─ navigation/

└─ feedback/

4.12 Common Components

Examples:

Button

Input

Select

Modal

Dialog

Badge

Tooltip

Spinner

4.13 Layout Components

Examples:

Header

Sidebar

Footer

PageLayout

DashboardLayout

ContentContainer

4.14 Feedback Components

Examples:

Alert

Notification

Toast

ErrorBoundary

LoadingIndicator

StatusMessage

4.15 Feature Architecture

frontend/src/features/

└─ authentication/

└─ opportunities/

└─ authorities/

└─ evidence/

└─ retrieval/

└─ ranking/

└─ recommendations/

└─ governance/

└─ audit/

4.16 Feature Structure Example

features/opportunities/

└─ components/

└─ hooks/

└─ services/

└─ types/

└─ validators/

└─ pages/

4.17 Feature Responsibilities

Features contain:

Business UI

Feature Hooks

Feature Services

Feature Validation

Feature State

4.18 Service Layer Structure

frontend/src/services/

- |— api_client.ts
- |— auth_service.ts
- |— opportunity_service.ts
- |— authority_service.ts
- |— evidence_service.ts
- |— retrieval_service.ts
- |— ranking_service.ts
- |— recommendation_service.ts
- |— governance_service.ts

4.19 Service Responsibilities

Services own:

HTTP Requests

API Mapping

Response Handling

Error Translation

Authentication Headers

Services do not own UI logic.

4.20 API Client Design

File:

api_client.ts

Responsibilities:

Axios Configuration

Request Interceptors

Response Interceptors

Authentication Headers

Correlation IDs

Error Handling

4.21 Hook Structure

frontend/src/hooks/

|— use_auth.ts

|— use_search.ts

|— use_ranking.ts

|— use_recommendations.ts

|— use_authorities.ts

|— use_evidence.ts

4.22 Hook Responsibilities

Hooks provide:

State Management

Workflow Logic

Data Loading

Caching

Request Coordination

Hooks do not perform direct API calls.

Hooks use services.

4.23 Context Structure

frontend/src/contexts/

|— AuthContext.tsx

|— UserContext.tsx

|— ThemeContext.tsx

|— NotificationContext.tsx

└─ GovernanceContext.tsx

4.24 Context Responsibilities

Contexts manage:

Global State

Authentication State

User Information

Notifications

Application Preferences

4.25 Routing Structure

frontend/src/routes/

└─ index.tsx

└─ protected_routes.tsx

└─ public_routes.tsx

└─ route_guards.tsx

4.26 Route Categories

Public Routes

Login

Documentation

Public Discovery

Protected Routes

Recommendations

Governance

Audit Views

Administration

4.27 Route Guard Design

Responsibilities:

Authentication Validation

Authorization Validation

Permission Validation

Redirection Logic

4.28 Type System Structure

frontend/src/types/

|— auth.ts

|— opportunity.ts

|— authority.ts

|— evidence.ts

|— retrieval.ts

|— ranking.ts

|— recommendation.ts

|— governance.ts

4.29 Type Design Rules

All API contracts require:

Request Types

Response Types

Error Types

Domain Types

Validation Types

4.30 Utility Structure

frontend/src/utils/

├─ date_utils.ts

├─ formatting_utils.ts

├─ validation_utils.ts

└─ security_utils.ts

└─ telemetry_utils.ts

4.31 Form Architecture

Forms use:

React Controlled Components

Zod Validation

Error Boundaries

Typed Inputs

4.32 State Management Strategy

Local State

Component State

Feature State

Hooks

Global State

Contexts

Backend remains source of truth.

4.33 Authentication Flow

Login

↓

Token Storage

↓

Auth Context Update



Protected Route Access



API Authorization

4.34 Error Handling Architecture

Error Sources:

Validation Errors

API Errors

Authorization Errors

Network Errors

System Errors

4.35 Frontend Observability

Every critical workflow generates:

UI Events

Metrics

Logs

Trace IDs

Correlation IDs

4.36 Accessibility Standards

Requirements:

Keyboard Navigation

Screen Reader Support

Semantic HTML

Accessible Forms

Accessible Error Messages

4.37 Performance Standards

Initial Page Load:

< 3 Seconds

API Response Rendering:

< 500 ms

Search Result Rendering:

< 1 Second

4.38 Testing Requirements

Frontend must support:

Unit Tests

Component Tests

Integration Tests

Accessibility Tests

E2E Tests

4.39 Governance Rules

UI-GOV-01

Business logic must not exist in components.

UI-GOV-02

API access occurs through services.

UI-GOV-03

Authentication controls are mandatory.

UI-GOV-04

Authorization controls are mandatory.

UI-GOV-05

Accessibility requirements are mandatory.

UI-GOV-06

Observability requirements are mandatory.

4.40 Architectural Constraints

AC-UI-01

Components cannot call APIs directly.

AC-UI-02

Pages cannot contain business logic.

AC-UI-03

Services cannot contain UI logic.

AC-UI-04

Protected routes require authentication.

AC-UI-05

Protected routes require authorization.

AC-UI-06

Governance controls cannot be bypassed.

4.41 Section Summary

This section defines the frontend package architecture, feature organization, service layer design, state management strategy, routing structure, and UI governance controls.

The frontend architecture provides a scalable and maintainable implementation framework aligned with TODP's principles of transparency, traceability, auditability, and governance.

TDD Progress

📄 Section 1 — Document Overview & Technical Scope

📄 Section 2 — Repository Technical Structure

📄 Section 3 — Backend Package Design

📄 Section 4 — Frontend Package Design

▣ Section 5 — Database Physical Schema Design

TDD V1 — Section 5

Database Physical Schema Design

Document: Technical Design Document (TDD V1)

Section: 5

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

5.1 Purpose

This section defines the physical database schema design for TODP.

The schema design provides:

- Persistent storage structure
- Entity relationships
- Data integrity rules
- Indexing strategy
- Versioning strategy
- Auditability support
- Traceability support

PostgreSQL is the authoritative source of truth.

5.2 Database Technology

Database Engine:

PostgreSQL

Extension:

pgvector

ORM:

SQLAlchemy

Migration Tool:

Alembic

5.3 Database Design Principles

DB-01

Data integrity first.

DB-02

Normalization before optimization.

DB-03

Explicit relationships.

DB-04

Auditability by design.

DB-05

Traceability by design.

DB-06

Reproducibility by design.

5.4 Schema Organization

todp

├─ auth

├─ opportunity

├─ authority

├─ evidence

├─ retrieval

├─ ranking

├─ recommendation

├─ governance

├─ audit

└─ observability

5.5 Naming Standards

Tables:

snake_case

Columns:

snake_case

Primary Keys:

id

Foreign Keys:

entity_name_id

Indexes:

idx_table_column

5.6 UUID Strategy

Primary Key Type:

UUID

Reason:

Global uniqueness

Distributed scalability

Safer external exposure

5.7 Common Audit Columns

Every business table contains:

id

created_at

updated_at

created_by

updated_by

version

5.8 Authentication Tables

Schema:

auth

Tables:

users

roles

permissions

user_roles

role_permissions

sessions

5.9 Users Table

Table:

auth.users

Columns:

id UUID PK

email VARCHAR UNIQUE

password_hash TEXT

status VARCHAR

last_login TIMESTAMP

created_at TIMESTAMP

updated_at TIMESTAMP

version INTEGER

5.10 Roles Table

Table:

auth.roles

Columns:

id UUID PK

name VARCHAR UNIQUE

description TEXT

created_at TIMESTAMP

updated_at TIMESTAMP

5.11 Permissions Table

Table:

auth.permissions

Columns:

id UUID PK

name VARCHAR UNIQUE

description TEXT

created_at TIMESTAMP

updated_at TIMESTAMP

5.12 Opportunities Table

Schema:

opportunity

Table:

opportunities

Purpose:

Master opportunity record.

5.13 Opportunities Table Columns

id UUID PK

title VARCHAR

description TEXT

opportunity_type VARCHAR

institution_level VARCHAR

location_level VARCHAR

status VARCHAR

created_at TIMESTAMP

updated_at TIMESTAMP

version INTEGER

5.14 Opportunity Location Table

Table:

opportunity.locations

Columns:

id UUID PK

opportunity_id FK

country VARCHAR

state VARCHAR

district VARCHAR

city VARCHAR

latitude NUMERIC

longitude NUMERIC

5.15 Opportunity Cost Table

Table:

opportunity.costs

Columns:

id UUID PK

opportunity_id FK

money_cost NUMERIC

time_cost NUMERIC

distance_cost NUMERIC

currency VARCHAR

5.16 Authorities Table

Schema:

authority

Table:

authorities

Purpose:

Verified authority records.

5.17 Authorities Table Columns

id UUID PK

name VARCHAR

authority_type VARCHAR

verification_status VARCHAR

confidence_score NUMERIC

created_at TIMESTAMP

updated_at TIMESTAMP

version INTEGER

5.18 Authority Verification Table

Table:

authority.verifications

Columns:

id UUID PK

authority_id FK

verification_result VARCHAR

verified_at TIMESTAMP

verified_by UUID

evidence_reference UUID

5.19 Evidence Table

Schema:

evidence

Table:

evidences

Purpose:

Evidence records supporting decisions.

5.20 Evidence Columns

id UUID PK

title VARCHAR

description TEXT

source VARCHAR

status VARCHAR

created_at TIMESTAMP

updated_at TIMESTAMP

version INTEGER

5.21 Evidence Lineage Table

Table:

evidence.lineage

Columns:

id UUID PK

parent_evidence_id FK

child_evidence_id FK

relationship_type VARCHAR

created_at TIMESTAMP

5.22 Retrieval Query Table

Schema:

retrieval

Table:

search_queries

Purpose:

Track search executions.

5.23 Retrieval Query Columns

id UUID PK

query_text TEXT

query_hash VARCHAR

executed_at TIMESTAMP

user_id UUID

correlation_id UUID

5.24 Embeddings Table

Schema:

retrieval

Table:

embeddings

Columns:

id UUID PK

entity_type VARCHAR

entity_id UUID

embedding VECTOR

created_at TIMESTAMP

5.25 Ranking Execution Table

Schema:

ranking

Table:

executions

Purpose:

Store ranking runs.

5.26 Ranking Execution Columns

id UUID PK

search_id UUID

formula_version VARCHAR

executed_at TIMESTAMP

correlation_id UUID

5.27 Ranking Result Table

Table:

ranking.results

Columns:

id UUID PK

execution_id FK

opportunity_id FK

money_cost NUMERIC

time_cost NUMERIC

distance_cost NUMERIC

total_cost NUMERIC

rank_position INTEGER

5.28 Recommendation Table

Schema:

recommendation

Table:

recommendations

Purpose:

Generated recommendations.

5.29 Recommendation Columns

id UUID PK

ranking_execution_id FK

recommendation_text TEXT

generated_at TIMESTAMP

correlation_id UUID

5.30 Recommendation Evidence Mapping

Table:

recommendation.evidence_links

Columns:

id UUID PK

recommendation_id FK

evidence_id FK

5.31 Recommendation Authority Mapping

Table:

recommendation.authority_links

Columns:

id UUID PK

recommendation_id FK

authority_id FK

5.32 Governance Decisions Table

Schema:

governance

Table:

decisions

Columns:

id UUID PK

decision_type VARCHAR

decision_text TEXT

status VARCHAR

approved_by UUID

created_at TIMESTAMP

5.33 Audit Records Table

Schema:

audit

Table:

audit_records

Purpose:

Immutable audit log.

5.34 Audit Record Columns

id UUID PK

entity_type VARCHAR

entity_id UUID

action VARCHAR

actor_id UUID

correlation_id UUID

metadata JSONB

created_at TIMESTAMP

5.35 Trace Records Table

Table:

audit.trace_records

Columns:

id UUID PK

source_entity VARCHAR

target_entity VARCHAR

relationship_type VARCHAR

correlation_id UUID

created_at TIMESTAMP

5.36 Snapshot Table

Table:

audit.snapshots

Purpose:

Reproducibility support.

Columns:

id UUID PK

entity_type VARCHAR

entity_id UUID

snapshot_data JSONB

version INTEGER

created_at TIMESTAMP

5.37 Observability Events Table

Schema:

observability

Table:

events

Columns:

id UUID PK

event_type VARCHAR

correlation_id UUID

metadata JSONB

created_at TIMESTAMP

5.38 Indexing Strategy

Primary Indexes:

All Primary Keys

Secondary Indexes:

email

query_hash

status

correlation_id

verification_status

created_at

5.39 Vector Index Strategy

Table:

retrieval.embeddings

Index:

HNSW

Purpose:

Fast semantic retrieval.

5.40 Foreign Key Strategy

All relationships use:

Explicit Foreign Keys

Cascade Deletes:

Restricted

Soft Deletes Preferred

5.41 Versioning Strategy

Versioned Tables:

opportunities

authorities

evidences

recommendations

governance.decisions

Version Column:

INTEGER

5.42 Soft Delete Strategy

Columns:

deleted_at

deleted_by

Purpose:

Historical preservation.

5.43 Partitioning Strategy

Future Capability:

audit.audit_records

audit.trace_records

observability.events

Partition By:

created_at

5.44 Database Governance Rules

DB-GOV-01

All tables require primary keys.

DB-GOV-02

Business entities require versioning.

DB-GOV-03

Critical actions require audit records.

DB-GOV-04

Relationships require foreign keys.

DB-GOV-05

Historical preservation is mandatory.

DB-GOV-06

Schema changes require migrations.

5.45 Architectural Constraints

AC-DB-01

PostgreSQL remains source of truth.

AC-DB-02

Redis cannot store authoritative data.

AC-DB-03

Direct schema modifications are prohibited.

AC-DB-04

Audit records are immutable.

AC-DB-05

Foreign keys cannot be bypassed.

AC-DB-06

Governance controls cannot be bypassed.

5.46 Section Summary

This section defines the physical PostgreSQL schema architecture, table structures, relationships, indexing strategy, versioning strategy, and governance controls for TODP.

The schema establishes the persistent foundation supporting retrieval, ranking, recommendation, governance, auditability, traceability, and reproducibility.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

☐ Section 6 — SQLAlchemy Models Design

TDD V1 — Section 6

SQLAlchemy Models Design

Document: Technical Design Document (TDD V1)

Section: 6

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

6.1 Purpose

This section defines the SQLAlchemy ORM model architecture for TODP.

The ORM layer provides:

- Database abstraction
- Entity persistence
- Relationship management
- Transaction support
- Query support
- Schema consistency

The ORM layer acts as the mapping layer between business entities and PostgreSQL tables.

6.2 ORM Technology

ORM:

SQLAlchemy 2.x

Pattern:

Declarative Mapping

Migration Tool:

Alembic

Database:

PostgreSQL

6.3 Model Design Principles

ORM-01

One model per table.

ORM-02

Models map persistence only.

ORM-03

Models do not contain business logic.

ORM-04

Relationships are explicit.

ORM-05

Models support auditability.

ORM-06

Models support versioning.

6.4 Model Repository Structure

backend/app/models/

├─ base.py

├─ user.py

- └─ role.py
- └─ permission.py
- └─ opportunity.py
- └─ authority.py
- └─ evidence.py
- └─ ranking.py
- └─ recommendation.py
- └─ governance.py
- └─ audit.py
- └─ observability.py

6.5 Base Model Design

File:

base.py

Purpose:

Shared model functionality.

Responsibilities:

UUID Support

Timestamp Support

Version Support

Audit Columns

Common Utilities

6.6 Base Entity Fields

All business models inherit:

id

created_at

updated_at

created_by

updated_by

version

6.7 Base Model Example

Class:

BaseEntity

Fields:

id: UUID

created_at: datetime

updated_at: datetime

created_by: UUID

updated_by: UUID

version: int

6.8 User Model

File:

user.py

Class:

UserModel

Table:

auth.users

6.9 User Model Fields

id

email

password_hash

status

last_login

created_at

updated_at

version

6.10 User Relationships

User



Roles

User



Sessions

User



Audit Records

6.11 Role Model

File:

role.py

Class:

RoleModel

Table:

auth.roles

6.12 Role Model Fields

id

name

description

created_at

updated_at

6.13 Permission Model

File:

permission.py

Class:

PermissionModel

Table:

auth.permissions

6.14 Permission Fields

id

name

description

created_at

updated_at

6.15 UserRole Model

Purpose:

Many-to-Many Mapping

User

↔

Role

Table:

auth.user_roles

6.16 RolePermission Model

Purpose:

Many-to-Many Mapping

Role

↔

Permission

Table:

auth.role_permissions

6.17 Opportunity Model

File:

opportunity.py

Class:

OpportunityModel

Table:

opportunity.opportunities

6.18 Opportunity Fields

id

title

description

opportunity_type

institution_level

location_level

status

created_at

updated_at

version

6.19 Opportunity Relationships

Opportunity

↓

Location

Opportunity

↓

Cost

Opportunity

↓

Ranking Results

Opportunity



Recommendations

6.20 OpportunityLocation Model

Class:

OpportunityLocationModel

Table:

opportunity.locations

Fields:

country

state

district

city

latitude

longitude

6.21 OpportunityCost Model

Class:

OpportunityCostModel

Table:

opportunity.costs

Fields:

money_cost

time_cost

distance_cost

currency

6.22 Authority Model

File:

authority.py

Class:

AuthorityModel

Table:

authority.authorities

6.23 Authority Fields

id

name

authority_type

verification_status

confidence_score

created_at

updated_at

version

6.24 Authority Relationships

Authority



Verifications

Authority



Recommendations

Authority

↓

Evidence References

6.25 AuthorityVerification Model

Class:

AuthorityVerificationModel

Table:

authority.verifications

Fields:

verification_result

verified_at

verified_by

evidence_reference

6.26 Evidence Model

File:

evidence.py

Class:

EvidenceModel

Table:

evidence.evidences

6.27 Evidence Fields

id

title

description

source

status

created_at

updated_at

version

6.28 Evidence Relationships

Evidence

↓

Lineage

Evidence

↓

Authorities

Evidence



Recommendations

6.29 EvidenceLineage Model

Class:

EvidenceLineageModel

Table:

evidence.lineage

Fields:

parent_evidence_id

child_evidence_id

relationship_type

6.30 SearchQuery Model

File:

ranking.py

Class:

SearchQueryModel

Table:

retrieval.search_queries

Fields:

query_text

query_hash

executed_at

user_id

correlation_id

6.31 Embedding Model

Class:

EmbeddingModel

Table:

retrieval.embeddings

Fields:

entity_type

entity_id

embedding

created_at

6.32 RankingExecution Model

Class:

RankingExecutionModel

Table:

ranking.executions

Fields:

search_id

formula_version

executed_at

correlation_id

6.33 RankingResult Model

Class:

RankingResultModel

Table:

ranking.results

Fields:

opportunity_id

money_cost

time_cost

distance_cost

total_cost

rank_position

6.34 Recommendation Model

File:

recommendation.py

Class:

RecommendationModel

Table:

recommendation.recommendations

Fields:

ranking_execution_id

recommendation_text

generated_at

correlation_id

6.35 RecommendationEvidenceLink Model

Class:

RecommendationEvidenceLinkModel

Table:

recommendation.evidence_links

Purpose:

Recommendation

↔

Evidence

6.36 RecommendationAuthorityLink Model

Class:

RecommendationAuthorityLinkModel

Table:

recommendation.authority_links

Purpose:

Recommendation

↔

Authority

6.37 GovernanceDecision Model

File:

governance.py

Class:

GovernanceDecisionModel

Table:

governance.decisions

Fields:

decision_type

decision_text

status

approved_by

created_at

6.38 AuditRecord Model

File:

audit.py

Class:

AuditRecordModel

Table:

audit.audit_records

Fields:

entity_type

entity_id

action

actor_id

correlation_id

metadata

created_at

6.39 TraceRecord Model

Class:

TraceRecordModel

Table:

audit.trace_records

Fields:

source_entity

target_entity

relationship_type

correlation_id

created_at

6.40 Snapshot Model

Class:

SnapshotModel

Table:

audit.snapshots

Fields:

entity_type

entity_id

snapshot_data

version

created_at

6.41 ObservabilityEvent Model

File:

observability.py

Class:

ObservabilityEventModel

Table:

observability.events

Fields:

event_type

correlation_id

metadata

created_at

6.42 SQLAlchemy Relationship Strategy

Use:

relationship()

back_populates

lazy loading

explicit joins

Avoid:

implicit relationship behavior

hidden joins

6.43 Model Validation Rules

Models enforce:

Database Constraints

Nullability

Uniqueness

Foreign Keys

Indexes

Business validation belongs elsewhere.

6.44 Transaction Management

Transactions managed by:

Repository Layer

Service Layer Coordination

Models do not manage transactions.

6.45 Soft Delete Support

Optional Fields:

deleted_at

deleted_by

Purpose:

Historical preservation.

6.46 Versioning Support

Version Column:

INTEGER

Used For:

Optimistic Locking

Historical Tracking

Replay Support

6.47 Model Governance Rules

ORM-GOV-01

Models map persistence only.

ORM-GOV-02

Business logic prohibited in models.

ORM-GOV-03

Relationships must be explicit.

ORM-GOV-04

Critical entities require versioning.

ORM-GOV-05

Auditability support is mandatory.

ORM-GOV-06

Schema consistency is mandatory.

6.48 Architectural Constraints

AC-ORM-01

Models cannot contain business workflows.

AC-ORM-02

Models cannot access services.

AC-ORM-03

Models cannot access APIs.

AC-ORM-04

Models cannot bypass repositories.

AC-ORM-05

Audit entities remain immutable.

AC-ORM-06

Governance controls cannot be bypassed.

6.49 Section Summary

This section defines the SQLAlchemy ORM architecture, model hierarchy, relationship strategy, versioning support, transaction boundaries, and governance controls.

The ORM layer provides a structured mapping between TODP business entities and PostgreSQL persistence structures while preserving auditability, traceability, and reproducibility.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

❏ Section 6 — SQLAlchemy Models Design

❏ Section 7 — Alembic Migration Design

TDD V1 — Section 7

Alembic Migration Design

Document: Technical Design Document (TDD V1)

Section: 7

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

7.1 Purpose

This section defines the database migration architecture for TODP.

The migration framework is responsible for:

- Schema evolution
- Version control
- Environment consistency
- Database reproducibility
- Rollback capability
- Deployment safety

All database changes must occur through managed migrations.

7.2 Migration Philosophy

Database schema is code.

Schema changes must be:

Versioned

Reviewed

Tested

Audited

Reproducible

Manual production schema modifications are prohibited.

7.3 Migration Technology

Migration Tool:

Alembic

ORM:

SQLAlchemy 2.x

Database:

PostgreSQL

7.4 Migration Objectives

MIG-01

Controlled schema evolution.

MIG-02

Reproducible environments.

MIG-03

Safe deployments.

MIG-04

Rollback support.

MIG-05

Auditability.

MIG-06

Traceability.

7.5 Migration Repository Structure

backend/

```
├─ migrations/
|   ├─ env.py
|   ├─ script.py.mako
|   └─ versions/
```

7.6 Migration Directory Structure

migrations/versions/

```
├─ 001_initial_schema.py
├─ 002_auth_schema.py
├─ 003_opportunity_schema.py
├─ 004_authority_schema.py
├─ 005_evidence_schema.py
├─ 006_retrieval_schema.py
├─ 007_ranking_schema.py
├─ 008_recommendation_schema.py
├─ 009_governance_schema.py
└─ 010_observability_schema.py
```

7.7 Migration Naming Standard

Format:

<sequence>_<description>.py

Examples:

001_initial_schema.py

002_auth_schema.py

011_add_opportunity_indexes.py

7.8 Migration Lifecycle

Schema Change

↓

Migration Creation

↓

Review



Testing



Approval



Deployment



Validation

7.9 Initial Migration Strategy

Migration 001

Creates:

Schemas

Core Tables

Primary Keys

Foreign Keys

Indexes

Extensions

7.10 PostgreSQL Extension Migration

Required Extensions:

pgvector

uuid-oss

Migration Example:

Create Extension



Verify Installation



Record Migration

7.11 Schema Creation Migration

Creates:

auth

opportunity

authority

evidence

retrieval

ranking

recommendation

governance

audit

observability

7.12 Authentication Migration

Creates:

users

roles

permissions

user_roles

role_permissions

sessions

7.13 Opportunity Migration

Creates:

opportunities

locations

costs

Indexes

Relationships

Constraints

7.14 Authority Migration

Creates:

authorities

verifications

Indexes

Relationships

Constraints

7.15 Evidence Migration

Creates:

evidences

lineage

Indexes

Relationships

Constraints

7.16 Retrieval Migration

Creates:

search_queries

embeddings

Vector Indexes

Search Indexes

7.17 Ranking Migration

Creates:

executions

results

Indexes

Constraints

7.18 Recommendation Migration

Creates:

recommendations

authority_links

evidence_links

Indexes

Relationships

7.19 Governance Migration

Creates:

decisions

Policies

Approval Structures

7.20 Audit Migration

Creates:

audit_records

trace_records

snapshots

Indexes

Constraints

7.21 Observability Migration

Creates:

events

Telemetry Structures

Indexes

Retention Support

7.22 Migration Template Structure

Every migration contains:

revision

down_revision

upgrade()

downgrade()

7.23 Upgrade Function

Responsibilities:

Create Tables

Create Indexes

Create Constraints

Create Extensions

Populate Seed Data

7.24 Downgrade Function

Responsibilities:

Reverse Migration

Remove Structures

Restore Prior State

Validate Rollback

7.25 Data Migration Strategy

Schema Changes

↓

Data Migration



Validation



Cleanup

Purpose:

Preserve production data.

7.26 Seed Data Migration

Initial Data Includes:

System Roles

System Permissions

Default Configuration

Governance Policies

7.27 Role Seed Data

Create:

User

Reviewer

Administrator

Governance Administrator

Security Administrator

System Administrator

7.28 Permission Seed Data

Create:

opportunity.read

opportunity.create

authority.verify

evidence.validate

ranking.execute

recommendation.generate

audit.read

policy.manage

7.29 Index Migration Strategy

Primary Indexes



Foreign Key Indexes



Search Indexes



Vector Indexes



Performance Validation

7.30 Vector Index Migration

Table:

retrieval.embeddings

Index:

HNSW

Purpose:

Semantic Search Performance

7.31 Constraint Migration Strategy

Constraints:

NOT NULL

UNIQUE

FOREIGN KEY

CHECK

All constraints explicitly defined.

7.32 Migration Validation

Validate:

Tables

Columns

Indexes

Constraints

Relationships

Extensions

7.33 Migration Testing

Required Tests:

Migration Execution

Migration Rollback

Data Preservation

Constraint Validation

Index Validation

7.34 Rollback Strategy

Supported For:

Schema Changes

Index Changes

Configuration Changes

Seed Changes

Critical Requirement:

Rollback Tested Before Release

7.35 Production Migration Workflow

Migration Approved

↓

Stage Validation

↓

Backup Verification



Migration Execution



Validation



Deployment Complete

7.36 Backup Requirements

Before Migration:

Database Backup

Configuration Backup

Recovery Validation

7.37 Migration Observability

Metrics:

Migration Duration

Migration Success Rate

Migration Failures

Rollback Count

Schema Drift Count

7.38 Migration Auditability

Every migration generates:

Migration Event

↓

Audit Record



Deployment Record



Version Record

7.39 Schema Version Tracking

Table:

governance.schema_versions

Fields:

version

migration_id

applied_at

applied_by

status

7.40 Environment Consistency

All environments must share:

Migration History

Schema State

Extension State

Version State

7.41 Schema Drift Detection

Validate:

Expected Schema

↓

Actual Schema



Difference Detection



Alert

7.42 Migration Security Requirements

Migration Execution Requires:

Authorized User

Approved Deployment

Audit Record

Rollback Plan

7.43 Governance Rules

MIG-GOV-01

All schema changes require migrations.

MIG-GOV-02

Manual schema changes prohibited.

MIG-GOV-03

Migrations require review.

MIG-GOV-04

Rollbacks require validation.

MIG-GOV-05

Migration history must be preserved.

MIG-GOV-06

Migration actions must be auditable.

7.44 Architectural Constraints

AC-MIG-01

Production schema changes without migrations are prohibited.

AC-MIG-02

Migration approval is mandatory.

AC-MIG-03

Rollback procedures are mandatory.

AC-MIG-04

Schema drift is prohibited.

AC-MIG-05

Migration history cannot be deleted.

AC-MIG-06

Governance controls cannot be bypassed.

7.45 Section Summary

This section defines the Alembic migration architecture, migration lifecycle, versioning strategy, rollback framework, validation process, and governance controls.

The migration framework ensures that TODP database evolution remains controlled, auditable, traceable, reproducible, and operationally safe.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

☐ Section 6 — SQLAlchemy Models Design

☐ Section 7 — Alembic Migration Design

☐ Section 8 — API Contract Standards

TDD V1 — Section 8

API Contract Standards

Document: Technical Design Document (TDD V1)

Section: 8

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

8.1 Purpose

This section defines the API contract standards governing all TODP APIs.

The API contract standard establishes:

- Request formats
- Response formats
- Error formats
- Validation standards
- Versioning standards
- Security requirements
- Traceability requirements

All APIs must follow these standards.

8.2 API Philosophy

APIs are contracts.

Contracts must be:

Consistent

Predictable

Versioned

Auditable

Traceable

Documented

Breaking changes must be controlled.

8.3 API Architecture

Client



API Gateway



Authentication



Authorization



Validation



Business Service



Response Mapping



Client

8.4 Base URL Structure

Production

/api/v1

Future Versions

/api/v2

/api/v3

Breaking changes require new versions.

8.5 Resource Naming Standards

Use plural nouns.

Examples:

/opportunities

/authorities

/evidence

/recommendations

/users

Avoid:

/getOpportunities

/createAuthority

/fetchEvidence

8.6 HTTP Method Standards

GET

Read

POST

Create

Execute

PUT

Full Update

PATCH

Partial Update

DELETE

Soft Delete

8.7 Content Type Standards

Request:

application/json

Response:

application/json

Character Encoding:

UTF-8

8.8 API Versioning Standard

Version Location:

URL Path

Example:

/api/v1/opportunities

Headers are not used for versioning.

8.9 Endpoint Naming Standard

Good:

/opportunities/{id}

/authorities/{id}

/evidence/{id}

Avoid:

/opportunityById

/authorityDetails

/evidenceFetch

8.10 Correlation ID Standard

Every request includes:

X-Correlation-ID

Generated If Missing

Used For:

Tracing

Auditing

Observability

Debugging

8.11 Request Header Standard

Required Headers:

Content-Type

Authorization

X-Correlation-ID

Optional:

Accept-Language

X-Request-ID

8.12 Authentication Standard

Protected APIs require:

Bearer Token

Format:

Authorization: Bearer <jwt>

Authentication occurs before request execution.

8.13 Authorization Standard

After authentication:

Role Resolution

↓

Permission Resolution

↓

Access Validation



Execution

8.14 Success Response Standard

Every success response follows:

```
{  
  "success": true,  
  "data": {},  
  "timestamp": "",  
  "correlation_id": ""  
}
```

8.15 Success Response Fields

success

Boolean

data

Response Payload

timestamp

UTC Timestamp

correlation_id

Request Identifier

8.16 Error Response Standard

```
{  
  "success": false,  
  "errors": [],  
  "timestamp": "",
```

```
"correlation_id": ""  
}
```

8.17 Error Object Structure

```
{  
  "code": "",  
  "message": "",  
  "field": ""  
}
```

8.18 Error Code Standards

VALIDATION_ERROR

AUTHENTICATION_ERROR

AUTHORIZATION_ERROR

RESOURCE_NOT_FOUND

CONFLICT_ERROR

RATE_LIMIT_ERROR

SYSTEM_ERROR

8.19 HTTP Status Codes

200

Success

201

Created

400

Bad Request

401

Unauthorized

403

Forbidden

404

Not Found

409

Conflict

429

Rate Limited

500

Internal Error

8.20 Pagination Standard

Request:

?page=1&page_size=20

Response:

```
{  
  "items": [],  
  "page": 1,  
  "page_size": 20,  
  "total": 100  
}
```

8.21 Sorting Standard

Request:

?sort=created_at

Descending:

?sort=-created_at

Multiple Fields:

?sort=created_at,title

8.22 Filtering Standard

Examples:

?status=active

?authority_type=government

?verification_status=verified

Filters remain explicit.

8.23 Search Standard

Example:

POST /retrieval/search

Request:

```
{  
  "query": "engineering scholarship"  
}
```

Search requests use POST.

8.24 Date Format Standard

Format:

ISO 8601 UTC

Example:

2026-06-14T10:30:00Z

8.25 UUID Standard

All identifiers:

UUID

Example:

550e8400-e29b-41d4-a716-446655440000

8.26 Idempotency Standard

Required For:

Critical POST Operations

Example:

Recommendation Generation

Ranking Execution

Header:

Idempotency-Key

8.27 Validation Standard

Validation Levels:

Transport Validation

Schema Validation

Business Validation

Permission Validation

8.28 DTO Standard

Every API defines:

Request DTO

Response DTO

Error DTO

Validation DTO

8.29 Request DTO Naming

Examples:

CreateOpportunityRequest

UpdateOpportunityRequest

SearchRequest

GenerateRecommendationRequest

8.30 Response DTO Naming

Examples:

OpportunityResponse

AuthorityResponse

EvidenceResponse

RecommendationResponse

8.31 Bulk Operation Standard

Request:

```
{  
  "ids": []  
}
```

Response:

```
{  
  "processed": 10,  
  "failed": 0  
}
```

8.32 Async Operation Standard

For long-running tasks:

Request

↓

Accepted

↓

Job ID Returned

↓

Status Endpoint

8.33 Async Response Example

```
{  
  "job_id": "",  
  "status": "pending"  
}
```

8.34 Health Endpoint Standard

Public Endpoint:

GET /health

Response:

```
{  
  "status": "healthy"  
}
```

8.35 Metrics Endpoint Standard

Protected Endpoint:

GET /metrics

Purpose:

Prometheus Scraping

8.36 OpenAPI Standard

Every endpoint must expose:

Request Schema

Response Schema

Error Schema

Security Requirements

Generated Automatically:

FastAPI

8.37 API Documentation Requirements

Documentation Includes:

Purpose

Request Format

Response Format

Examples

Error Codes

Permissions

8.38 Rate Limiting Standard

Anonymous:

100 Requests/Hour

Authenticated:

1000 Requests/Hour

Administrative APIs:

Governed Separately

8.39 Security Headers

Responses include:

HSTS

X-Frame-Options

X-Content-Type-Options

Content-Security-Policy

8.40 Auditability Requirements

Every API request generates:

API Event



Audit Record



Trace Record



Correlation ID

8.41 Observability Requirements

Every API request generates:

Metrics

Logs

Traces

Performance Data

8.42 Performance Requirements

GET Requests

P95 < 200 ms

POST Requests

P95 < 500 ms

Search Requests

P95 < 2 s

Recommendation Requests

P95 < 500 ms

8.43 Governance Rules

API-GOV-01

All APIs are versioned.

API-GOV-02

All APIs are documented.

API-GOV-03

Protected APIs require authentication.

API-GOV-04

Protected APIs require authorization.

API-GOV-05

All APIs generate audit records.

API-GOV-06

All APIs generate observability data.

8.44 Architectural Constraints

AC-API-01

Breaking changes require new API versions.

AC-API-02

Response format consistency is mandatory.

AC-API-03

Authentication cannot be bypassed.

AC-API-04

Authorization cannot be bypassed.

AC-API-05

Correlation IDs are mandatory.

AC-API-06

Governance controls cannot be bypassed.

8.45 Section Summary

This section defines the API contract standards governing request structures, response structures, validation rules, versioning rules, security requirements, observability requirements, and governance controls.

These standards provide a consistent and predictable API surface across the entire TODP platform.

TDD Progress

📄 Section 1 — Document Overview & Technical Scope

📄 Section 2 — Repository Technical Structure

📄 Section 3 — Backend Package Design

📄 Section 4 — Frontend Package Design

📄 Section 5 — Database Physical Schema Design

☞ Section 6 — SQLAlchemy Models Design

☞ Section 7 — Alembic Migration Design

☞ Section 8 — API Contract Standards

☞ Section 9 — Authentication APIs

TDD V1 — Section 9

Authentication APIs

Document: Technical Design Document (TDD V1)

Section: 9

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

9.1 Purpose

This section defines the Authentication API specifications for TODP.

Authentication APIs are responsible for:

- User authentication
- Token issuance
- Token refresh
- Session management
- Logout operations
- Authentication verification

Authentication identifies users.

Authorization determines what users can do.

9.2 Authentication Architecture

Client



Authentication API



Authentication Service

↓

User Repository

↓

PostgreSQL

↓

JWT Generation

↓

Client

9.3 Authentication Base Route

/api/v1/auth

9.4 Authentication Endpoints

POST /login

POST /refresh

POST /logout

GET /me

GET /validate

POST /change-password

POST /forgot-password

POST /reset-password

9.5 Login Endpoint

Endpoint:

POST /api/v1/auth/login

Purpose:

Authenticate user.

Issue tokens.

9.6 Login Request DTO

```
{  
  "email": "user@example.com",  
  "password": "password"  
}
```

9.7 Login Validation Rules

email

Required

Valid email format

password

Required

Minimum length validation

9.8 Login Processing Flow

Validate Request



Locate User



Verify Password



Verify User Status



Generate Tokens



Create Session



Return Response

9.9 Login Response DTO

```
{
  "success": true,
  "data": {
    "access_token": "",
    "refresh_token": "",
    "token_type": "Bearer",
    "expires_in": 3600
  },
  "correlation_id": ""
}
```

9.10 Login Failure Cases

Invalid Email

↓

401

Invalid Password

↓

401

Inactive User

↓

403

Locked Account

↓

403

9.11 Refresh Token Endpoint

Endpoint:

POST /api/v1/auth/refresh

Purpose:

Issue new access token.

9.12 Refresh Request DTO

```
{  
  "refresh_token": ""  
}
```

9.13 Refresh Processing Flow

Validate Token

↓

Verify Session

↓

Generate New Access Token

↓

Return Response

9.14 Refresh Response DTO

```
{  
  "success": true,  
  "data": {  
    "access_token": "",  
    "expires_in": 3600  
  }  
}
```

9.15 Logout Endpoint

Endpoint:

POST /api/v1/auth/logout

Purpose:

Terminate user session.

Invalidate refresh token.

9.16 Logout Request

Authorization Header Required.

No request body required.

9.17 Logout Processing Flow

Authenticate User

↓

Locate Session

↓

Invalidate Session

↓

Revoke Refresh Token

↓

Return Success

9.18 Logout Response DTO

```
{  
  "success": true,  
  "data": {  
    "message": "Logout successful"  
  }  
}
```

9.19 Current User Endpoint

Endpoint:

GET /api/v1/auth/me

Purpose:

Return authenticated user profile.

9.20 Current User Response DTO

```
{  
  "success": true,  
  "data": {  
    "id": "",  
    "email": "",  
    "roles": [],  
    "permissions": []  
  }  
}
```

9.21 Token Validation Endpoint

Endpoint:

GET /api/v1/auth/validate

Purpose:

Verify token validity.

9.22 Validation Response DTO

```
{  
  "success": true,  
  "data": {  
    "authenticated": true  
  }  
}
```

9.23 Change Password Endpoint

Endpoint:

POST /api/v1/auth/change-password

Purpose:

Allow authenticated users to change password.

9.24 Change Password Request DTO

```
{  
  "current_password": "",  
  "new_password": ""  
}
```

9.25 Change Password Validation

Current Password Required

New Password Required

Password Complexity Validation

Password Reuse Prevention

9.26 Change Password Flow

Authenticate User



Verify Current Password



Validate New Password



Hash Password



Update User

↓

Invalidate Sessions

↓

Return Success

9.27 Forgot Password Endpoint

Endpoint:

POST /api/v1/auth/forgot-password

Purpose:

Initiate password reset workflow.

9.28 Forgot Password Request DTO

```
{  
  "email": "user@example.com"  
}
```

9.29 Forgot Password Processing

Validate Email



Locate User



Generate Reset Token



Store Reset Token



Send Notification



Return Success

9.30 Reset Password Endpoint

Endpoint:

POST /api/v1/auth/reset-password

Purpose:

Complete password reset.

9.31 Reset Password Request DTO

```
{  
  "token": "",  
  "new_password": ""  
}
```

9.32 Reset Password Processing

Validate Token



Validate Password



Hash Password



Update User



Invalidate Sessions



Return Success



9.33 JWT Structure

Access Token Claims:

sub

email

roles

permissions

iat

exp

jti

9.34 Access Token Configuration

Algorithm:

HS256

(or approved equivalent)

Expiration:

60 Minutes

9.35 Refresh Token Configuration

Expiration:

30 Days

Stored Securely

Revocable

9.36 Session Management

Session Table:

auth.sessions

Tracks:

User

Token

Login Time

Expiration

Status

9.37 Password Security Requirements

Hashing Algorithm:

bcrypt

Password Never Stored Plaintext

Password Verification Constant Time

9.38 Account Status Types

ACTIVE

INACTIVE

LOCKED

DISABLED

PENDING

9.39 Login Rate Limiting

Per User:

10 Attempts / Hour

Per IP:

100 Attempts / Hour

Excessive Failures Trigger Alerts

9.40 Authentication Exceptions

AuthenticationException

InvalidCredentialsException

InactiveUserException

LockedUserException

ExpiredTokenException

SessionExpiredException

9.41 Authentication Service Interface

AuthenticationService

Methods:

login()

refresh()

logout()

validate()

change_password()

forgot_password()

reset_password()

9.42 Repository Dependencies

Authentication Service Uses:

UserRepository

SessionRepository

AuditRepository

9.43 Security Controls

Authentication APIs Require:

Input Validation

Rate Limiting

Audit Logging

Trace Logging

Session Validation

9.44 Auditability Requirements

Every authentication event generates:

Authentication Event

↓

Audit Record



Trace Record



Correlation ID

9.45 Observability Requirements

Metrics:

Login Count

Login Failures

Token Refreshes

Password Changes

Session Count

9.46 Governance Rules

AUTH-GOV-01

Passwords must be hashed.

AUTH-GOV-02

Refresh tokens must be revocable.

AUTH-GOV-03

Authentication events must be audited.

AUTH-GOV-04

Session management is mandatory.

AUTH-GOV-05

Rate limiting is mandatory.

AUTH-GOV-06

Token expiration is mandatory.

9.47 Architectural Constraints

AC-AUTH-01

Passwords cannot be stored in plaintext.

AC-AUTH-02

Authentication cannot bypass audit logging.

AC-AUTH-03

Refresh tokens require secure storage.

AC-AUTH-04

Expired tokens are invalid.

AC-AUTH-05

Session validation is mandatory.

AC-AUTH-06

Governance controls cannot be bypassed.

9.48 Section Summary

This section defines the Authentication API architecture, endpoint specifications, DTOs, token lifecycle, session management framework, security controls, and governance requirements.

The Authentication APIs establish the identity foundation for all protected operations within TODP.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

☐ Section 6 — SQLAlchemy Models Design

☐ Section 7 — Alembic Migration Design

☐ Section 8 — API Contract Standards

☐ Section 9 — Authentication APIs

☐ Section 10 — Opportunity APIs

TDD V1 — Section 10

Opportunity APIs

Document: Technical Design Document (TDD V1)

Section: 10

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

10.1 Purpose

This section defines the Opportunity API specifications for TODP.

Opportunity APIs are responsible for:

- Opportunity creation
- Opportunity updates
- Opportunity retrieval
- Opportunity search
- Opportunity lifecycle management
- Opportunity archival

Opportunities are the primary discoverable entities within TODP.

10.2 Opportunity Architecture

Client



Opportunity API



Opportunity Service



Opportunity Repository



PostgreSQL



Response DTO

↓

Client

10.3 Base Route

/api/v1/opportunities

10.4 Opportunity Endpoints

POST /

GET /

GET /{opportunity_id}

PUT /{opportunity_id}

PATCH /{opportunity_id}

DELETE /{opportunity_id}

GET /{opportunity_id}/costs

GET /{opportunity_id}/authorities

GET /{opportunity_id}/evidence

10.5 Create Opportunity Endpoint

Endpoint:

POST /api/v1/opportunities

Purpose:

Create a new opportunity.

10.6 Create Opportunity Request DTO

```
{  
  "title": "",  
  "description": "",  
  "opportunity_type": "",
```

```
"institution_level": "",  
"location_level": "",  
"country": "",  
"state": "",  
"district": "",  
"city": "",  
"money_cost": 0,  
"time_cost": 0,  
"distance_cost": 0  
}
```

10.7 Create Opportunity Validation

title

Required

Maximum 500 Characters

description

Required

opportunity_type

Required

institution_level

Required

location_level

Required

10.8 Opportunity Type Values

ONLINE

PHYSICAL

SIMILAR_CREATION

NEW_CREATION

10.9 Discovery Hierarchy Values

INSTITUTION

LOCAL

DISTRICT

STATE

NATIONAL

INTERNATIONAL

10.10 Create Opportunity Flow

Validate Request

↓

Validate Business Rules



Create Opportunity



Create Location Record



Create Cost Record



Generate Audit Record



Return Response

10.11 Create Opportunity Response DTO

{

```
"success": true,  
"data": {  
  "id": "",  
  "title": "",  
  "status": "ACTIVE"  
}  
}
```

10.12 List Opportunities Endpoint

Endpoint:

GET /api/v1/opportunities

Purpose:

Retrieve opportunities.

Supports filtering and pagination.

10.13 List Opportunities Query Parameters

?page=1

&page_size=20

&status=ACTIVE

&institution_level=STATE

&opportunity_type=ONLINE

10.14 List Opportunities Response DTO

```
{
  "success": true,
  "data": {
    "items": [],
    "page": 1,
    "page_size": 20,
    "total": 100
  }
}
```

10.15 Get Opportunity Endpoint

Endpoint:

GET /api/v1/opportunities/{opportunity_id}

Purpose:

Retrieve a specific opportunity.

10.16 Path Parameters

opportunity_id

UUID

Required

10.17 Get Opportunity Response DTO

```
{  
  "success": true,  
  "data": {  
    "id": "",  
    "title": "",
```

```
"description": "",  
"opportunity_type": "",  
"institution_level": "",  
"status": ""  
}  
}
```

10.18 Update Opportunity Endpoint

Endpoint:

PUT /api/v1/opportunities/{opportunity_id}

Purpose:

Replace opportunity data.

10.19 Update Opportunity Request DTO

```
{  
  "title": "",  
  "description": "",  
  "opportunity_type": "",
```

```
"institution_level": "",  
"status": ""  
}
```

10.20 Update Opportunity Flow

Authenticate



Authorize



Load Opportunity



Validate Request



Update Opportunity



Increment Version



Audit Record



Return Response

10.21 Patch Opportunity Endpoint

Endpoint:

PATCH /api/v1/opportunities/{opportunity_id}

Purpose:

Partial update.

10.22 Patch Rules

Only supplied fields updated.

Version increment required.

Audit record required.

10.23 Delete Opportunity Endpoint

Endpoint:

DELETE /api/v1/opportunities/{opportunity_id}

Purpose:

Soft delete opportunity.

10.24 Delete Flow

Authenticate

↓

Authorize



Locate Opportunity



Mark Deleted



Generate Audit Record



Return Success

10.25 Opportunity Status Values

DRAFT

ACTIVE

INACTIVE

ARCHIVED

DELETED

10.26 Opportunity Costs Endpoint

Endpoint:

GET /api/v1/opportunities/{opportunity_id}/costs

Purpose:

Retrieve ranking cost inputs.

10.27 Cost Response DTO

```
{  
  "money_cost": 1000,  
  "time_cost": 5,  
  "distance_cost": 25  
}
```

10.28 Opportunity Authorities Endpoint

Endpoint:

GET /api/v1/opportunities/{opportunity_id}/authorities

Purpose:

Retrieve authority references.

10.29 Authority Response DTO

```
{  
  "authorities": [  
    {  
      "id": "",  
      "name": "",  
      "verification_status": ""  
    }  
  ]  
}
```

10.30 Opportunity Evidence Endpoint

Endpoint:

GET /api/v1/opportunities/{opportunity_id}/evidence

Purpose:

Retrieve evidence references.

10.31 Evidence Response DTO

```
{  
  "evidence": [  
    {  
      "id": "",  
      "title": "",  
      "status": ""  
    }  
  ]  
}
```

10.32 Search Opportunities Endpoint

Endpoint:

POST /api/v1/opportunities/search

Purpose:

Structured opportunity search.

10.33 Search Request DTO

```
{  
  "query": "",  
  "institution_level": "",  
  "location_level": "",  
  "opportunity_type": ""  
}
```

10.34 Search Processing Flow

Validate Query

↓

Execute Search



Apply Filters



Generate Response



Audit Search



Return Results

10.35 Opportunity Service Interface

OpportunityService

Methods:

create()

get_by_id()

list()

update()

patch()

delete()

search()

10.36 Repository Dependencies

Opportunity Service Uses:

OpportunityRepository

AuthorityRepository

EvidenceRepository

AuditRepository

10.37 Opportunity Repository Interface

Methods:

create()

get()

list()

update()

delete()

search()

exists()

10.38 Validation Rules

Title Required

Description Required

Valid Discovery Hierarchy

Valid Opportunity Type

Valid Status

Valid UUID

10.39 Error Scenarios

Opportunity Not Found

↓

404

Invalid Request

↓

400

Unauthorized

↓

401

Forbidden

↓

403

Conflict

↓

409

10.40 Performance Requirements

Create Opportunity

P95 < 500 ms

Get Opportunity

P95 < 200 ms

List Opportunities

P95 < 500 ms

Search Opportunities

P95 < 2 Seconds

10.41 Auditability Requirements

Every opportunity action generates:

Opportunity Event



Audit Record



Trace Record



Correlation ID

10.42 Observability Requirements

Metrics:

Opportunity Count

Create Count

Update Count

Delete Count

Search Count

Search Latency

10.43 Security Requirements

Create

Update

Delete

Require Authentication

Require Authorization

Require Audit Logging

Read Operations

Require Authorization

10.44 Governance Rules

OPP-GOV-01

Opportunity creation requires validation.

OPP-GOV-02

Opportunity updates require version tracking.

OPP-GOV-03

Deletes use soft delete strategy.

OPP-GOV-04

Search activity is auditable.

OPP-GOV-05

Critical changes require audit records.

OPP-GOV-06

Discovery hierarchy values are controlled.

10.45 Architectural Constraints

AC-OPP-01

Opportunities cannot bypass validation.

AC-OPP-02

Opportunities cannot bypass audit logging.

AC-OPP-03

Deletes must preserve history.

AC-OPP-04

Versioning is mandatory.

AC-OPP-05

Authority references remain traceable.

AC-OPP-06

Governance controls cannot be bypassed.

10.46 Section Summary

This section defines the Opportunity API architecture, endpoint contracts, DTOs, lifecycle operations, validation rules, service interfaces, and governance requirements.

The Opportunity APIs provide the primary platform interface for creating, managing, retrieving, and discovering opportunities across all TODP discovery levels.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

☐ Section 6 — SQLAlchemy Models Design

☐ Section 7 — Alembic Migration Design

☐ Section 8 — API Contract Standards

☐ Section 9 — Authentication APIs

❏ Section 10 — Opportunity APIs

❏ Section 11 — Authority APIs

TDD V1 — Section 11

Authority APIs

Document: Technical Design Document (TDD V1)

Section: 11

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

11.1 Purpose

This section defines the Authority API specifications for TODP.

Authority APIs are responsible for:

- Authority registration
- Authority retrieval
- Authority verification
- Authority confidence management
- Authority lifecycle management
- Authority traceability

Authority records establish trusted sources within the TODP ecosystem.

AI cannot be treated as an authority.

11.2 Authority Architecture

Client



Authority API



Authority Service



Authority Verification Service



Authority Repository



PostgreSQL



Response DTO



Client

11.3 Base Route

/api/v1/authorities

11.4 Authority Endpoints

POST /

GET /

GET /{authority_id}

PUT /{authority_id}

PATCH /{authority_id}

DELETE /{authority_id}

POST /{authority_id}/verify

GET /{authority_id}/verification-history

GET /{authority_id}/evidence

GET /{authority_id}/confidence

11.5 Create Authority Endpoint

Endpoint:

POST /api/v1/authorities

Purpose:

Create authority record.

11.6 Create Authority Request DTO

```
{  
  "name": "",  
  "authority_type": "",  
  "description": "",  
  "website": "",  
  "country": ""  
}
```

11.7 Authority Type Values

GOVERNMENT

INSTITUTION

UNIVERSITY

ORGANIZATION

COMPANY

COMMUNITY

INDIVIDUAL_EXPERT

11.8 Create Authority Validation

name

Required

Maximum 500 Characters

authority_type

Required

Must be valid type

website

Optional

Must be valid URL

11.9 Create Authority Flow

Validate Request



Validate Authority Type



Create Authority



Generate Audit Record

↓

Return Response

11.10 Create Authority Response DTO

```
{  
  "success": true,  
  "data": {  
    "id": "",  
    "name": "",  
    "verification_status": "PENDING"  
  }  
}
```

11.11 List Authorities Endpoint

Endpoint:

GET /api/v1/authorities

Purpose:

Retrieve authority records.

11.12 Query Parameters

?page=1

&page_size=20

&authority_type=UNIVERSITY

&verification_status=VERIFIED

11.13 Get Authority Endpoint

Endpoint:

GET /api/v1/authorities/{authority_id}

Purpose:

Retrieve authority details.

11.14 Authority Response DTO

```
{  
  "success": true,  
  "data": {  
    "id": "",  
    "name": "",  
    "authority_type": "",  
    "verification_status": "",  
    "confidence_score": 0  
  }  
}
```

11.15 Update Authority Endpoint

Endpoint:

PUT /api/v1/authorities/{authority_id}

Purpose:

Replace authority information.

11.16 Patch Authority Endpoint

Endpoint:

PATCH /api/v1/authorities/{authority_id}

Purpose:

Partial update.

11.17 Delete Authority Endpoint

Endpoint:

DELETE /api/v1/authorities/{authority_id}

Purpose:

Soft delete authority.

11.18 Authority Status Values

PENDING

VERIFIED

REJECTED

SUSPENDED

ARCHIVED

DELETED

11.19 Verify Authority Endpoint

Endpoint:

POST /api/v1/authorities/{authority_id}/verify

Purpose:

Execute authority verification workflow.

11.20 Verify Authority Request DTO

```
{  
  "verification_source": "",  
  "verification_notes": "",  
  "evidence_ids": []  
}
```

11.21 Verification Processing Flow

Load Authority



Load Evidence



Execute Verification Rules



Calculate Confidence



Store Verification Result

↓

Generate Audit Record

↓

Return Result

11.22 Verification Response DTO

```
{  
  "success": true,  
  "data": {  
    "verification_status": "VERIFIED",  
    "confidence_score": 92.5  
  }  
}
```

11.23 Verification Status Values

PENDING

UNDER_REVIEW

VERIFIED

REJECTED

EXPIRED

11.24 Verification History Endpoint

Endpoint:

GET /api/v1/authorities/{authority_id}/verification-history

Purpose:

Retrieve historical verification events.

11.25 Verification History Response DTO

{

"history": [


```
{  
  "verification_result": "",  
  "verified_at": "",  
  "confidence_score": 0  
}  
]  
}
```

11.26 Authority Evidence Endpoint

Endpoint:

GET /api/v1/authorities/{authority_id}/evidence

Purpose:

Retrieve supporting evidence.

11.27 Evidence Response DTO

```
{  
  "evidence": [  
    {
```

```
"id": "",  
"title": "",  
"status": ""  
}  
]  
}
```

11.28 Authority Confidence Endpoint

Endpoint:

GET /api/v1/authorities/{authority_id}/confidence

Purpose:

Retrieve confidence information.

11.29 Confidence Response DTO

```
{  
"authority_id": "",  
"confidence_score": 92.5,  
"verification_status": "VERIFIED"
```

}

11.30 Authority Confidence Rules

0 - 25

Low Confidence

26 - 50

Moderate Confidence

51 - 75

High Confidence

76 - 100

Verified Confidence

11.31 Authority Service Interface

AuthorityService

Methods:

create()

get_by_id()

list()

update()

patch()

delete()

verify()

get_history()

get_confidence()

11.32 Authority Verification Service

AuthorityVerificationService

Methods:

verify_authority()

calculate_confidence()

validate_evidence()

generate_result()

11.33 Repository Dependencies

Authority Service Uses:

AuthorityRepository

EvidenceRepository

AuditRepository

TraceRepository

11.34 Authority Repository Interface

Methods:

create()

get()

list()

update()

delete()

exists()

search()

11.35 Validation Rules

Authority Name Required

Authority Type Required

Valid Status Required

Valid UUID Required

Evidence References Valid

11.36 Error Scenarios

Authority Not Found

↓

404

Invalid Authority Type

↓

400

Verification Failure

↓

422

Unauthorized

↓

401

Forbidden

↓

403

11.37 Authority-Evidence Relationship

Authority



Evidence



Verification



Confidence Score



Recommendation Eligibility

11.38 Recommendation Dependency

Recommendations may use:

Verified Authorities

Recommendations must not rely on:

Rejected Authorities

Suspended Authorities

Deleted Authorities

11.39 Performance Requirements

Create Authority

P95 < 500 ms

Get Authority

P95 < 200 ms

List Authorities

P95 < 500 ms

Verification

P95 < 2 Seconds

11.40 Auditability Requirements

Every authority action generates:

Authority Event

↓

Audit Record

↓

Trace Record

↓

Correlation ID

11.41 Traceability Requirements

Every authority verification stores:

Verification Source

Evidence References

Decision Path

Confidence Calculation

Historical Record

11.42 Observability Requirements

Metrics:

Authority Count

Verification Count

Verification Failures

Confidence Distribution

Verification Latency

11.43 Security Requirements

Authority Creation

Authority Update

Authority Verification

Authority Delete

Require Authentication

Require Authorization

Require Audit Logging

11.44 Governance Rules

AUTHORITY-GOV-01

AI cannot be registered as authority.

AUTHORITY-GOV-02

Authority verification requires evidence.

AUTHORITY-GOV-03

Verification actions are auditable.

AUTHORITY-GOV-04

Confidence calculations are traceable.

AUTHORITY-GOV-05

Authority history must be preserved.

AUTHORITY-GOV-06

Recommendations require verified authorities.

11.45 Architectural Constraints

AC-AUTHORITY-01

Unverified authorities cannot be treated as verified.

AC-AUTHORITY-02

Authority verification cannot bypass evidence.

AC-AUTHORITY-03

Verification history cannot be deleted.

AC-AUTHORITY-04

Confidence calculations must be reproducible.

AC-AUTHORITY-05

Authority records require versioning.

AC-AUTHORITY-06

Governance controls cannot be bypassed.

11.46 Section Summary

This section defines the Authority API architecture, verification workflows, confidence management framework, endpoint contracts, DTOs, service interfaces, and governance requirements.

The Authority APIs establish the trusted source framework that supports evidence validation, ranking integrity, and recommendation trustworthiness throughout TODP.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

TDD V1 — Section 12

Evidence APIs

Document: Technical Design Document (TDD V1)

Section: 12

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

12.1 Purpose

This section defines the Evidence API specifications for TODP.

Evidence APIs are responsible for:

- Evidence creation
- Evidence retrieval
- Evidence validation

- Evidence versioning
- Evidence lineage tracking
- Evidence lifecycle management

Evidence is the foundational input for authority verification, ranking integrity, and recommendation generation.

Evidence First remains a core TODP principle.

12.2 Evidence Architecture

Client



Evidence API



Evidence Service



Evidence Validation Service



Evidence Repository



PostgreSQL



Response DTO



Client

12.3 Base Route

/api/v1/evidence

12.4 Evidence Endpoints

POST /

GET /

GET /{evidence_id}

PUT /{evidence_id}

PATCH /{evidence_id}

DELETE /{evidence_id}

POST /{evidence_id}/validate

GET /{evidence_id}/versions

GET /{evidence_id}/lineage

GET /{evidence_id}/authorities

12.5 Create Evidence Endpoint

Endpoint:

POST /api/v1/evidence

Purpose:

Create evidence record.

12.6 Create Evidence Request DTO

```
{  
  "title": "",  
  "description": "",  
  "source": "",  
  "source_type": "",  
  "source_url": "",  
  "publication_date": ""  
}
```

12.7 Evidence Source Type Values

GOVERNMENT

UNIVERSITY

INSTITUTION

ORGANIZATION

COMPANY

RESEARCH

PUBLIC_RECORD

COMMUNITY

12.8 Create Evidence Validation

title

Required

Maximum 500 Characters

source

Required

source_type

Required

publication_date

Optional

ISO-8601 Format

12.9 Create Evidence Flow

Validate Request

↓

Validate Source Type

↓

Create Evidence

↓

Create Version Record

↓

Generate Audit Record

↓

Return Response

12.10 Create Evidence Response DTO

```
{  
  "success": true,  
  "data": {  
    "id": "",  
    "title": "",  
    "status": "PENDING"  
  }  
}
```

12.11 List Evidence Endpoint

Endpoint:

GET /api/v1/evidence

Purpose:

Retrieve evidence records.

12.12 Query Parameters

?page=1

&page_size=20

&status=VALIDATED

&source_type=GOVERNMENT

12.13 Get Evidence Endpoint

Endpoint:

GET /api/v1/evidence/{evidence_id}

Purpose:

Retrieve evidence details.

12.14 Evidence Response DTO

```
{  
  "success": true,  
  "data": {  
    "id": "",  
    "title": "",  
    "description": "",  
    "source": "",  
    "status": ""  
  }  
}
```

12.15 Update Evidence Endpoint

Endpoint:

PUT /api/v1/evidence/{evidence_id}

Purpose:

Replace evidence information.

12.16 Patch Evidence Endpoint

Endpoint:

PATCH /api/v1/evidence/{evidence_id}

Purpose:

Partial update.

12.17 Delete Evidence Endpoint

Endpoint:

DELETE /api/v1/evidence/{evidence_id}

Purpose:

Soft delete evidence.

12.18 Evidence Status Values

PENDING

VALIDATED

REJECTED

SUPERSEDED

ARCHIVED

DELETED

12.19 Validate Evidence Endpoint

Endpoint:

POST /api/v1/evidence/{evidence_id}/validate

Purpose:

Execute evidence validation workflow.

12.20 Validate Evidence Request DTO

```
{  
  "validation_notes": "",  
  "validation_source": "",  
  "validation_type": ""  
}
```

12.21 Validation Type Values

SOURCE_VALIDATION

CONTENT_VALIDATION

AUTHORITY_VALIDATION

LINEAGE_VALIDATION

FULL_VALIDATION

12.22 Validation Processing Flow

Load Evidence



Validate Source



Validate Content



Validate Lineage



Generate Validation Result



Store Validation Record

↓

Generate Audit Record

↓

Return Response

12.23 Validation Response DTO

```
{  
  "success": true,  
  "data": {  
    "status": "VALIDATED",  
    "validated_at": ""  
  }  
}
```

12.24 Evidence Versions Endpoint

Endpoint:

GET /api/v1/evidence/{evidence_id}/versions

Purpose:

Retrieve historical versions.

12.25 Versions Response DTO

```
{  
  "versions": [  
    {  
      "version": 1,  
      "created_at": ""  
    }  
  ]  
}
```

12.26 Evidence Lineage Endpoint

Endpoint:

GET /api/v1/evidence/{evidence_id}/lineage

Purpose:

Retrieve evidence lineage graph.

12.27 Lineage Response DTO

```
{  
  "parent_evidence": [],  
  "child_evidence": []  
}
```

12.28 Evidence Authorities Endpoint

Endpoint:

GET /api/v1/evidence/{evidence_id}/authorities

Purpose:

Retrieve linked authorities.

12.29 Authorities Response DTO

```
{  
  "authorities": [  
    {  
      "id": "",  
      "name": "",  
      "verification_status": ""  
    }  
  ]  
}
```

12.30 Evidence Versioning Rules

Every modification:

Creates Version

Increments Version Number

Preserves Previous State

Creates Snapshot

12.31 Evidence Lineage Rules

Lineage Supports:

Parent Evidence

Child Evidence

Derived Evidence

Supporting Evidence

Conflicting Evidence

12.32 Evidence Validation Rules

Validation Requires:

Source Verification

Integrity Verification

Traceability Verification

Lineage Verification

12.33 Evidence Service Interface

EvidenceService

Methods:

create()

get_by_id()

list()

update()

patch()

delete()

validate()

get_versions()

get_lineage()

12.34 Evidence Validation Service

EvidenceValidationService

Methods:

validate_source()

validate_content()

validate_lineage()

validate_authority()

generate_result()

12.35 Repository Dependencies

Evidence Service Uses:

EvidenceRepository

AuthorityRepository

AuditRepository

TraceRepository

SnapshotRepository

12.36 Evidence Repository Interface

Methods:

create()

get()

list()

update()

delete()

exists()

search()

get_versions()

12.37 Validation Rules

Evidence Title Required

Source Required

Source Type Required

Valid UUID Required

Version Tracking Required

12.38 Error Scenarios

Evidence Not Found

↓

404

Validation Failure



422

Invalid Source Type



400

Unauthorized



401

Forbidden



403

12.39 Authority Dependency

Authority Verification Uses:

Validated Evidence

Rejected Evidence

Cannot Support Authority Verification

12.40 Recommendation Dependency

Recommendations Require:

Evidence



Authority



Ranking



Recommendation

Evidence remains the first dependency.

12.41 Performance Requirements

Create Evidence

P95 < 500 ms

Get Evidence

P95 < 200 ms

List Evidence

P95 < 500 ms

Validation

P95 < 2 Seconds

12.42 Auditability Requirements

Every evidence action generates:

Evidence Event

↓

Audit Record

↓

Trace Record



Correlation ID

12.43 Traceability Requirements

Evidence Stores:

Source

Validation History

Version History

Lineage

Authority Links

Decision References

12.44 Reproducibility Requirements

Evidence Supports:

Version Replay

Snapshot Replay

Historical Reconstruction

Lineage Reconstruction

12.45 Observability Requirements

Metrics:

Evidence Count

Validation Count

Validation Failures

Version Count

Lineage Depth

Validation Latency

12.46 Security Requirements

Create

Update

Delete

Validate

Require Authentication

Require Authorization

Require Audit Logging

12.47 Governance Rules

EVIDENCE-GOV-01

Evidence must support versioning.

EVIDENCE-GOV-02

Evidence must support lineage.

EVIDENCE-GOV-03

Validation actions are auditable.

EVIDENCE-GOV-04

Historical versions must be preserved.

EVIDENCE-GOV-05

Evidence traceability is mandatory.

EVIDENCE-GOV-06

Evidence precedes authority verification.

12.48 Architectural Constraints

AC-EVIDENCE-01

Evidence cannot bypass validation.

AC-EVIDENCE-02

Version history cannot be deleted.

AC-EVIDENCE-03

Lineage must remain reconstructable.

AC-EVIDENCE-04

Validation history must be preserved.

AC-EVIDENCE-05

Evidence must remain traceable.

AC-EVIDENCE-06

Governance controls cannot be bypassed.

12.49 Section Summary

This section defines the Evidence API architecture, validation workflows, lineage framework, versioning strategy, endpoint contracts, DTOs, service interfaces, and governance requirements.

The Evidence APIs establish the factual foundation supporting authority verification, ranking integrity, recommendation trustworthiness, auditability, traceability, and reproducibility across TODP.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

TDD V1 — Section 13

Retrieval APIs

Document: Technical Design Document (TDD V1)

Section: 13

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

13.1 Purpose

This section defines the Retrieval API specifications for TODP.

Retrieval APIs are responsible for:

- Opportunity discovery
- Candidate generation
- Keyword search
- Semantic search
- Hybrid retrieval
- Search observability

Retrieval generates candidate opportunities.

Retrieval does not perform ranking.

Retrieval does not generate recommendations.

13.2 Retrieval Architecture

Client

↓

Retrieval API

↓

Retrieval Service

↓

BM25 Engine

+

Vector Engine



Fusion Engine



Candidate Set



Response DTO



Client

13.3 Retrieval Principle

Retrieval Stage:

Find potentially relevant opportunities.

Ranking Stage:

Order opportunities.

Recommendation Stage:

Generate recommendations.

These stages remain separate.

13.4 Base Route

/api/v1/retrieval

13.5 Retrieval Endpoints

POST /search

POST /keyword-search

POST /semantic-search

POST /hybrid-search

GET /searches/{search_id}

GET /searches/{search_id}/results

GET /health

13.6 Search Endpoint

Endpoint:

POST /api/v1/retrieval/search

Purpose:

Default retrieval entry point.

Uses hybrid retrieval.

13.7 Search Request DTO

```
{  
  "query": "",  
  "institution_level": "",  
  "location_level": "",  
  "opportunity_type": "",  
  "page": 1,  
  "page_size": 20  
}
```

13.8 Query Validation

query

Required

Minimum Length: 2

Maximum Length: 1000

page

Minimum: 1

page_size

Maximum: 100

13.9 Search Processing Flow

Validate Request



Normalize Query



Execute BM25 Search



Execute Vector Search



Fuse Results



Apply Filters



Generate Search Record



Return Candidate Set

13.10 Search Response DTO

```
{  
  "success": true,  
  "data": {  
    "search_id": "",  
    "results": [],  
    "total": 100  
  }  
}
```

13.11 Keyword Search Endpoint

Endpoint:

POST /api/v1/retrieval/keyword-search

Purpose:

BM25-only retrieval.

13.12 Keyword Search Request DTO

```
{  
  "query": "",  
  "page": 1,  
  "page_size": 20  
}
```

13.13 Keyword Search Flow

Validate Query



Tokenize Query



Execute BM25



Generate Candidate Set



Return Results

13.14 Semantic Search Endpoint

Endpoint:

POST /api/v1/retrieval/semantic-search

Purpose:

Vector-only retrieval.

13.15 Semantic Search Request DTO

```
{  
  "query": "",  
  "page": 1,  
  "page_size": 20  
}
```

13.16 Semantic Search Flow

Validate Query

↓

Generate Embedding

↓

Execute Vector Search

↓

Generate Candidate Set

↓

Return Results

13.17 Hybrid Search Endpoint

Endpoint:

POST /api/v1/retrieval/hybrid-search

Purpose:

Explicit hybrid retrieval execution.

13.18 Hybrid Search Flow

Validate Query

↓

BM25 Retrieval

+

Vector Retrieval

↓

Fusion Layer

↓

Candidate Set

↓

Return Results

13.19 Search Result Structure

```
{  
  "opportunity_id": "",  
  "title": "",  
  "retrieval_score": 0.0,
```

```
"retrieval_method": ""  
}
```

13.20 Retrieval Method Values

BM25

SEMANTIC

HYBRID

13.21 Candidate Set Rules

Retrieval Produces:

Candidate Opportunities

Retrieval Does Not Produce:

Ranks

Recommendations

Authority Decisions

13.22 Search History Endpoint

Endpoint:

GET /api/v1/retrieval/searches/{search_id}

Purpose:

Retrieve search metadata.

13.23 Search Metadata DTO

```
{  
  "search_id": "",  
  "query": "",  
  "executed_at": "",  
  "retrieval_method": ""  
}
```

13.24 Search Results Endpoint

Endpoint:

GET /api/v1/retrieval/searches/{search_id}/results

Purpose:

Retrieve stored candidate results.

13.25 Search Persistence

Stored Data:

Search Query

Search Hash

Execution Time

Candidate Results

Correlation ID

13.26 Retrieval Service Interface

RetrievalService

Methods:

search()

keyword_search()

semantic_search()

hybrid_search()

get_search()

get_results()

13.27 BM25 Service Interface

BM25Service

Methods:

tokenize()

index()

search()

score()

13.28 Vector Service Interface

VectorService

Methods:

embed()

search()

similarity()

index()

13.29 Fusion Service Interface

FusionService

Methods:

merge()

normalize()

rerank()

generate_candidates()

13.30 Repository Dependencies

Retrieval Service Uses:

OpportunityRepository

EmbeddingRepository

SearchRepository

AuditRepository

13.31 Search Filters

Supported Filters:

Institution Level

Location Level

Opportunity Type

Status

Country

State

District

13.32 Institution Level Values

INSTITUTION

LOCAL

DISTRICT

STATE

NATIONAL

INTERNATIONAL

13.33 Opportunity Type Filters

ONLINE

PHYSICAL

SIMILAR_CREATION

NEW_CREATION

13.34 Query Normalization

Normalize:

Whitespace

Case

Punctuation

Unicode Variants

Store Original Query Separately.

13.35 Search Record Generation

Every search creates:

search_id

query_hash

timestamp

user_id

correlation_id

13.36 Embedding Requirements

Embeddings Stored In:

retrieval.embeddings

Linked To:

Opportunity Records

Authority Records

Evidence Records

13.37 Error Scenarios

Invalid Query

↓

400

Search Not Found

↓

404

Embedding Failure

↓

500

Unauthorized

↓

401

Forbidden

↓

403

13.38 Performance Requirements

Keyword Search

P95 < 500 ms

Semantic Search

P95 < 1.5 Seconds

Hybrid Search

P95 < 2 Seconds

Search History Retrieval

P95 < 200 ms

13.39 Auditability Requirements

Every retrieval action generates:

Search Event

↓

Audit Record

↓

Trace Record

↓

Correlation ID

13.40 Traceability Requirements

Store:

Original Query

Normalized Query

Retrieval Method

Fusion Version

Search Results

Execution Metadata

13.41 Reproducibility Requirements

Search Replay Supports:

Query Replay

Filter Replay

Candidate Reconstruction

Fusion Reconstruction

13.42 Observability Requirements

Metrics:

Search Count

Keyword Searches

Semantic Searches

Hybrid Searches

Latency

Failure Rate

Candidate Volume

13.43 Security Requirements

Protected Search APIs Require:

Authentication

Authorization

Rate Limiting

Audit Logging

13.44 Governance Rules

RET-GOV-01

Retrieval cannot perform ranking.

RET-GOV-02

Retrieval cannot generate recommendations.

RET-GOV-03

Search activity is auditable.

RET-GOV-04

Search activity is traceable.

RET-GOV-05

Search replay must be supported.

RET-GOV-06

Fusion logic must be versioned.

13.45 Architectural Constraints

AC-RET-01

Candidate generation must remain separate from ranking.

AC-RET-02

Retrieval scores cannot be treated as ranks.

AC-RET-03

Recommendations cannot be generated by retrieval.

AC-RET-04

Search history must be preserved.

AC-RET-05

Search metadata must remain reproducible.

AC-RET-06

Governance controls cannot be bypassed.

13.46 Section Summary

This section defines the Retrieval API architecture, hybrid retrieval workflow, search contracts, candidate generation framework, service interfaces, and governance controls.

The Retrieval APIs establish the discovery layer of TODP by generating candidate opportunity sets through BM25, vector search, and hybrid fusion while maintaining strict separation from ranking and recommendation systems.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

🔗 Section 14 — Ranking APIs

TDD V1 — Section 14

Ranking APIs

Document: Technical Design Document (TDD V1)

Section: 14

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

14.1 Purpose

This section defines the Ranking API specifications for TODP.

Ranking APIs are responsible for:

- Ranking candidate opportunities
- Calculating total cost
- Applying ranking rules
- Producing ordered opportunity sets
- Preserving ranking reproducibility
- Supporting ranking traceability

Ranking occurs after retrieval and before recommendation.

14.2 Ranking Architecture

Client



Ranking API



Ranking Service



Ranking Engine



Cost Calculator



Ranking Results



Response DTO



Client

14.3 Ranking Principle

Retrieval

↓

Ranking

↓

Recommendation

Retrieval finds candidates.

Ranking orders candidates.

Recommendation explains ranked outcomes.

14.4 Universal Ranking Formula

Total Cost

=

Money Cost

+

Time Cost

+

Distance Cost

Lowest Total Cost ranks highest.

14.5 Base Route

/api/v1/rankings

14.6 Ranking Endpoints

POST /execute

GET /executions/{execution_id}

GET /executions/{execution_id}/results

POST /compare

GET /formula

GET /health

14.7 Execute Ranking Endpoint

Endpoint:

POST /api/v1/rankings/execute

Purpose:

Execute ranking on retrieved opportunities.

14.8 Execute Ranking Request DTO

```
{  
  "search_id": "",  
  "formula_version": "v1"  
}
```

14.9 Ranking Request Validation

search_id

Required

Valid UUID

formula_version

Required

Supported Version

14.10 Ranking Processing Flow

Load Search Results



Load Cost Data



Calculate Total Cost



Sort Ascending



Resolve Ties



Store Results



Generate Audit Record

↓

Return Results

14.11 Ranking Response DTO

```
{  
  "success": true,  
  "data": {  
    "execution_id": "",  
    "total_results": 100,  
    "formula_version": "v1"  
  }  
}
```

14.12 Ranking Result Structure

```
{  
  "opportunity_id": "",  
  "money_cost": 0,
```

```
"time_cost": 0,  
"distance_cost": 0,  
"total_cost": 0,  
"rank_position": 1  
}
```

14.13 Cost Inputs

Money Cost

Financial cost required.

Time Cost

Time investment required.

Distance Cost

Physical distance burden.

14.14 Cost Rules

Costs must be:

Numeric

Non-Negative

Traceable

Versioned

Auditable

14.15 Total Cost Calculation

Formula:

total_cost =

money_cost +

time_cost +

distance_cost

No hidden weighting.

No hidden modifiers.

14.16 Ranking Sort Rule

Sort By:

total_cost ASC

Lowest value:

Rank 1

Highest value:

Last Rank

14.17 Tie Resolution Policy

If total_cost is equal:

Sort By:

money_cost ASC



time_cost ASC



distance_cost ASC



opportunity_id ASC

14.18 Formula Endpoint

Endpoint:

GET /api/v1/rankings/formula

Purpose:

Expose active ranking formula.

14.19 Formula Response DTO

```
{  
  "formula_version": "v1",  
  "formula": "money_cost + time_cost + distance_cost"  
}
```

14.20 Ranking Execution Endpoint

Endpoint:

GET /api/v1/rankings/executions/{execution_id}

Purpose:

Retrieve ranking execution metadata.

14.21 Execution Metadata DTO

```
{  
  "execution_id": "",  
  "search_id": "",  
  "formula_version": "",  
  "executed_at": ""  
}
```

14.22 Ranking Results Endpoint

Endpoint:

GET /api/v1/rankings/executions/{execution_id}/results

Purpose:

Retrieve ranking results.

14.23 Results Response DTO

```
{
```

```
"results": [  
  {  
    "rank_position": 1,  
    "opportunity_id": "",  
    "total_cost": 10  
  }  
]  
}
```

14.24 Opportunity Comparison Endpoint

Endpoint:

POST /api/v1/rankings/compare

Purpose:

Compare opportunities using ranking formula.

14.25 Compare Request DTO

```
{  
  "opportunity_ids": [  
    
```



```
""  
"  
"  
]  
}
```

14.26 Compare Response DTO

```
{  
  "comparison": [  
    {  
      "opportunity_id": "",  
      "total_cost": 0  
    }  
  ]  
}
```

14.27 Ranking Execution Record

Stored Data:

execution_id

search_id

formula_version

execution_time

correlation_id

14.28 Ranking Result Record

Stored Data:

opportunity_id

money_cost

time_cost

distance_cost

total_cost

rank_position

14.29 Ranking Versioning

Every ranking run stores:

Formula Version

Execution Timestamp

Input Dataset

Output Dataset

14.30 Ranking Service Interface

RankingService

Methods:

execute()

compare()

get_execution()

get_results()

get_formula()

14.31 Ranking Engine Interface

RankingEngine

Methods:

calculate_total_cost()

rank()

resolve_ties()

validate_inputs()

14.32 Cost Calculator Interface

CostCalculator

Methods:

calculate_money_cost()

calculate_time_cost()

calculate_distance_cost()

calculate_total_cost()

14.33 Repository Dependencies

Ranking Service Uses:

SearchRepository

OpportunityRepository

RankingRepository

AuditRepository

14.34 Ranking Validation Rules

Valid Search Required

Valid Formula Version Required

Valid Cost Inputs Required

Candidate Set Required

14.35 Error Scenarios

Search Not Found

↓

404

Invalid Formula Version

↓

400

Ranking Failure

↓

500

Unauthorized

↓

401

Forbidden

↓

403

14.36 Recommendation Dependency

Recommendations require:

Completed Ranking

Recommendations cannot bypass ranking.

14.37 Ranking Reproducibility

Every execution stores:

Input Candidates

Formula Version

Cost Inputs

Ranking Outputs

Execution Metadata

14.38 Traceability Requirements

Store:

Search Reference

Execution Reference

Formula Version

Cost Components

Rank Position

14.39 Auditability Requirements

Every ranking execution generates:

Ranking Event

↓

Audit Record

↓

Trace Record



Correlation ID

14.40 Observability Requirements

Metrics:

Ranking Count

Ranking Duration

Ranking Failures

Average Candidate Count

Formula Usage

14.41 Performance Requirements

Ranking Execution

P95 < 1 Second

Result Retrieval

P95 < 200 ms

Comparison Request

P95 < 500 ms

14.42 Security Requirements

Execute Ranking

Requires Authentication

Requires Authorization

Requires Audit Logging

Formula Endpoint

Read Access Controlled

14.43 Governance Rules

RANK-GOV-01

Ranking occurs after retrieval.

RANK-GOV-02

Ranking occurs before recommendation.

RANK-GOV-03

Formula must be transparent.

RANK-GOV-04

Formula must be versioned.

RANK-GOV-05

Ranking executions must be reproducible.

RANK-GOV-06

Ranking executions must be auditable.

14.44 Architectural Constraints

AC-RANK-01

No hidden ranking logic permitted.

AC-RANK-02

No sponsored ranking permitted.

AC-RANK-03

No paid placement permitted.

AC-RANK-04

Formula modifications require governance approval.

AC-RANK-05

Ranking history must be preserved.

AC-RANK-06

Governance controls cannot be bypassed.

14.45 Section Summary

This section defines the Ranking API architecture, ranking workflow, total cost calculation framework, endpoint contracts, service interfaces, reproducibility controls, and governance requirements.

The Ranking APIs provide the deterministic ordering layer of TODP using the transparent Total Cost formula while preserving auditability, traceability, reproducibility, and ranking integrity.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

❏ Section 8 — API Contract Standards

❏ Section 9 — Authentication APIs

❏ Section 10 — Opportunity APIs

❏ Section 11 — Authority APIs

❏ Section 12 — Evidence APIs

❏ Section 13 — Retrieval APIs

❏ Section 14 — Ranking APIs

❏ Section 15 — Recommendation APIs

TDD V1 — Section 15

Recommendation APIs

Document: Technical Design Document (TDD V1)

Section: 15

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

15.1 Purpose

This section defines the Recommendation API specifications for TODP.

Recommendation APIs are responsible for:

- Recommendation generation
- Recommendation retrieval
- Recommendation explanation
- Evidence linkage
- Authority linkage
- Recommendation traceability

Recommendations are derived outputs.

Recommendations do not create authority.

Recommendations do not create evidence.

Recommendations do not perform ranking.

15.2 Recommendation Architecture

Client



Recommendation API



Recommendation Service



Evidence Validation Check



Authority Verification Check



Ranking Validation Check



Recommendation Engine



Response DTO



Client

15.3 Recommendation Principle

Evidence



Authority



Ranking



Recommendation

Recommendation is the final stage.

15.4 Base Route

/api/v1/recommendations

15.5 Recommendation Endpoints

POST /generate

GET /{recommendation_id}

GET /

GET /{recommendation_id}/evidence

GET /{recommendation_id}/authorities

GET /{recommendation_id}/explanation

GET /executions/{execution_id}

15.6 Generate Recommendation Endpoint

Endpoint:

POST /api/v1/recommendations/generate

Purpose:

Generate recommendations from completed ranking execution.

15.7 Generate Recommendation Request DTO

```
{  
  "ranking_execution_id": "",  
  "recommendation_version": "v1"  
}
```

15.8 Request Validation Rules

ranking_execution_id

Required

Valid UUID

recommendation_version

Required

Supported Version

15.9 Recommendation Processing Flow

Load Ranking Execution

↓

Validate Evidence

↓

Validate Authorities

↓

Validate Ranking Results

↓

Generate Recommendation

↓

Persist Recommendation

↓

Generate Audit Record

↓

Return Result

15.10 Recommendation Response DTO

```
{  
  "success": true,  
  "data": {  
    "recommendation_id": "",  
    "generated_at": "",  
    "recommendation_version": "v1"
```

```
}  
}
```

15.11 Recommendation Structure

```
{  
  "recommendation_id": "",  
  "opportunity_id": "",  
  "recommendation_text": "",  
  "rank_position": 1  
}
```

15.12 Recommendation Generation Rules

Requirement 1

Evidence Must Exist

Requirement 2

Authority Must Be Verified

Requirement 3

Ranking Must Exist

Requirement 4

Recommendation Must Be Traceable

15.13 Recommendation Versioning

Every recommendation stores:

Recommendation Version

Ranking Version

Evidence References

Authority References

Generation Timestamp

15.14 Get Recommendation Endpoint

Endpoint:

GET /api/v1/recommendations/{recommendation_id}

Purpose:

Retrieve recommendation.

15.15 Recommendation Response DTO

```
{  
  "success": true,  
  "data": {  
    "recommendation_id": "",  
    "recommendation_text": "",  
    "rank_position": 1  
  }  
}
```

15.16 List Recommendations Endpoint

Endpoint:

GET /api/v1/recommendations

Purpose:

Retrieve recommendations.

15.17 Query Parameters

?page=1

&page_size=20

&recommendation_version=v1

&opportunity_id=<uuid>

15.18 Recommendation Evidence Endpoint

Endpoint:

GET /api/v1/recommendations/{recommendation_id}/evidence

Purpose:

Retrieve supporting evidence.

15.19 Evidence Response DTO

```
{
  "evidence": [
    {
      "id": "",
      "title": "",
      "status": "VALIDATED"
    }
  ]
}
```

15.20 Recommendation Authorities Endpoint

Endpoint:

GET /api/v1/recommendations/{recommendation_id}/authorities

Purpose:

Retrieve supporting authorities.

15.21 Authority Response DTO

```
{
  "authorities": [
    {
      "id": "",
      "name": "",
      "verification_status": "VERIFIED"
    }
  ]
}
```

15.22 Recommendation Explanation Endpoint

Endpoint:

GET /api/v1/recommendations/{recommendation_id}/explanation

Purpose:

Retrieve recommendation reasoning.

15.23 Explanation Response DTO

```
{  
  "recommendation_id": "",  
  "rank_position": 1,  
  "total_cost": 15,  
  "evidence_count": 5,  
  "authority_count": 2,  
  "explanation": ""  
}
```

15.24 Explanation Requirements

Explanation Must Include:

Ranking Position

Total Cost

Evidence References

Authority References

Recommendation Logic

15.25 Ranking Execution Recommendations Endpoint

Endpoint:

GET /api/v1/recommendations/executions/{execution_id}

Purpose:

Retrieve recommendations generated from ranking execution.

15.26 Execution Response DTO

{

"execution_id": "",

```
"recommendations": []  
}
```

15.27 Recommendation Service Interface

RecommendationService

Methods:

generate()

get()

list()

get_evidence()

get_authorities()

get_explanation()

get_by_execution()

15.28 Recommendation Engine Interface

RecommendationEngine

Methods:

generate_recommendation()

build_explanation()

validate_dependencies()

generate_links()

15.29 Repository Dependencies

Recommendation Service Uses:

RecommendationRepository

RankingRepository

AuthorityRepository

EvidenceRepository

AuditRepository

TraceRepository

15.30 Recommendation Repository Interface

Methods:

create()

get()

list()

get_by_execution()

get_evidence()

get_authorities()

15.31 Recommendation Validation Rules

Valid Ranking Execution Required

Validated Evidence Required

Verified Authorities Required

Valid Recommendation Version Required

15.32 Recommendation Dependency Chain

Evidence

↓

Authority

↓

Ranking

↓

Recommendation

Dependency order cannot be altered.

15.33 Recommendation Eligibility Rules

Eligible:

Validated Evidence

Verified Authority

Completed Ranking

Not Eligible:

Rejected Evidence

Rejected Authority

Missing Ranking

15.34 Recommendation Storage

Stored Data:

Recommendation ID

Recommendation Version

Ranking Execution

Evidence References

Authority References

Explanation

Correlation ID

15.35 Error Scenarios

Recommendation Not Found

↓

404

Ranking Execution Missing

↓

422

Evidence Validation Failure

↓

422

Authority Verification Failure

↓

422

Unauthorized



401

Forbidden



403

15.36 Reproducibility Requirements

Every recommendation stores:

Input Ranking

Input Evidence

Input Authorities

Version Information

Generated Output

15.37 Traceability Requirements

Store:

Evidence Links

Authority Links

Ranking Links

Execution Metadata

Decision Metadata

15.38 Auditability Requirements

Every recommendation action generates:

Recommendation Event



Audit Record



Trace Record



Correlation ID

15.39 Observability Requirements

Metrics:

Recommendation Count

Generation Count

Generation Failures

Average Evidence Count

Average Authority Count

Generation Latency

15.40 Performance Requirements

Recommendation Generation

P95 < 500 ms

Recommendation Retrieval

P95 < 200 ms

Explanation Retrieval

P95 < 200 ms

15.41 Security Requirements

Generate Recommendation

Requires Authentication

Requires Authorization

Requires Audit Logging

Read Operations

Authorization Required

15.42 Governance Rules

REC-GOV-01

Recommendations require validated evidence.

REC-GOV-02

Recommendations require verified authorities.

REC-GOV-03

Recommendations require completed rankings.

REC-GOV-04

Recommendation explanations are mandatory.

REC-GOV-05

Recommendations must be traceable.

REC-GOV-06

Recommendations must be reproducible.

15.43 Architectural Constraints

AC-REC-01

Recommendations cannot bypass ranking.

AC-REC-02

Recommendations cannot bypass authority verification.

AC-REC-03

Recommendations cannot bypass evidence validation.

AC-REC-04

Recommendation explanations must remain available.

AC-REC-05

Recommendation history must be preserved.

AC-REC-06

Governance controls cannot be bypassed.

15.44 Section Summary

This section defines the Recommendation API architecture, recommendation generation workflow, dependency validation framework, endpoint contracts, explanation framework, service interfaces, and governance controls.

The Recommendation APIs provide the final decision-support layer of TODP by transforming validated evidence, verified authorities, and completed rankings into transparent, traceable, auditable, and reproducible recommendations.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

🔗 Section 14 — Ranking APIs

🔗 Section 15 — Recommendation APIs

🔗 Section 16 — Governance APIs

TDD V1 — Section 16

Governance APIs

Document: Technical Design Document (TDD V1)

Section: 16

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

16.1 Purpose

This section defines the Governance API specifications for TODP.

Governance APIs are responsible for:

- Policy management
- Decision management
- Approval workflows
- Governance reviews
- Compliance validation
- Governance traceability

Governance ensures that TODP operates according to approved principles, policies, and architectural constraints.

16.2 Governance Architecture

Client



Governance API



Governance Service



Policy Engine



Approval Engine



Governance Repository



PostgreSQL



Response DTO



Client

16.3 Governance Principles

Governance enforces:

Evidence First

Authority Before Recommendation

Ranking Before Recommendation

Traceability

Auditability

Reproducibility

No Sponsored Content

No Paid Placement

No Hidden Ranking Logic

16.4 Base Route

/api/v1/governance

16.5 Governance Endpoints

POST /decisions

GET /decisions

GET /decisions/{decision_id}

PUT /decisions/{decision_id}

POST /approvals

GET /approvals

POST /policy-validations

GET /policies

GET /compliance

GET /violations

16.6 Create Governance Decision Endpoint

Endpoint:

POST /api/v1/governance/decisions

Purpose:

Create governance decision record.

16.7 Create Decision Request DTO

```
{  
  "decision_type": "",  
  "decision_text": "",  
  "justification": ""  
}
```

16.8 Decision Type Values

POLICY

ARCHITECTURE

SECURITY

DATA

RANKING

RECOMMENDATION

COMPLIANCE

16.9 Create Decision Flow

Validate Request



Create Decision



Generate Audit Record



Generate Trace Record



Return Response

16.10 Create Decision Response DTO

```
{  
  "success": true,  
  "data": {  
    "decision_id": "",
```

```
"status": "PENDING"
}
}
```

16.11 List Decisions Endpoint

Endpoint:

GET /api/v1/governance/decisions

Purpose:

Retrieve governance decisions.

16.12 Query Parameters

?page=1

&page_size=20

&decision_type=POLICY

&status=APPROVED

16.13 Get Decision Endpoint

Endpoint:

GET /api/v1/governance/decisions/{decision_id}

Purpose:

Retrieve governance decision.

16.14 Decision Response DTO

```
{  
  "decision_id": "",  
  "decision_type": "",  
  "status": "",  
  "approved_by": ""  
}
```

16.15 Update Decision Endpoint

Endpoint:

PUT /api/v1/governance/decisions/{decision_id}

Purpose:

Update governance decision.

16.16 Governance Status Values

PENDING

UNDER_REVIEW

APPROVED

REJECTED

ARCHIVED

16.17 Create Approval Endpoint

Endpoint:

POST /api/v1/governance/approvals

Purpose:

Submit governance approval.

16.18 Approval Request DTO

```
{  
  "decision_id": "",  
  "approval_result": "",  
  "comments": ""  
}
```

16.19 Approval Result Values

APPROVED

REJECTED

REQUEST_CHANGES

16.20 Approval Processing Flow

Load Decision



Validate Reviewer



Record Approval



Update Status



Generate Audit Record



Return Response

16.21 Approval Response DTO

```
{  
  "success": true,  
  "data": {  
    "approval_id": "",  
    "decision_status": "APPROVED"  
  }  
}
```

16.22 Policy Validation Endpoint

Endpoint:

POST /api/v1/governance/policy-validations

Purpose:

Validate entity against governance policies.

16.23 Policy Validation Request DTO

```
{  
  "entity_type": "",  
  "entity_id": ""  
}
```

16.24 Policy Validation Flow

Load Entity



Load Policies



Execute Validation Rules



Generate Result



Store Validation Record

↓

Return Response

16.25 Policy Validation Response DTO

```
{  
  "entity_id": "",  
  "compliant": true,  
  "violations": []  
}
```

16.26 Policies Endpoint

Endpoint:

GET /api/v1/governance/policies

Purpose:

Retrieve active governance policies.

16.27 Policy Response DTO

```
{  
  "policies": [  
    {  
      "policy_id": "",  
      "policy_name": ""  
    }  
  ]  
}
```

16.28 Compliance Endpoint

Endpoint:

GET /api/v1/governance/compliance

Purpose:

Retrieve compliance summary.

16.29 Compliance Response DTO

```
{  
  "compliant_entities": 100,  
  "violations": 2  
}
```

16.30 Violations Endpoint

Endpoint:

GET /api/v1/governance/violations

Purpose:

Retrieve governance violations.

16.31 Violation Response DTO

```
{  
  "violations": [  
    {  
      "violation_id": "",
```

```
    "violation_type": ""  
  }  
]  
}
```

16.32 Governance Service Interface

GovernanceService

Methods:

create_decision()

update_decision()

list_decisions()

approve()

validate_policy()

get_compliance()

get_violations()

16.33 Policy Engine Interface

PolicyEngine

Methods:

load_policies()

validate()

detect_violation()

generate_result()

16.34 Approval Engine Interface

ApprovalEngine

Methods:

submit_approval()

validate_reviewer()

update_status()

generate_history()

16.35 Repository Dependencies

Governance Service Uses:

GovernanceRepository

AuditRepository

TraceRepository

PolicyRepository

16.36 Governance Repository Interface

Methods:

create()

get()

list()

update()

get_policies()

get_violations()

16.37 Governance Validation Rules

Valid Decision Type Required

Valid Reviewer Required

Valid Entity Required

Valid Policy Required

16.38 Governance Enforcement Areas

Ranking Policies

Recommendation Policies

Authority Policies

Evidence Policies

Security Policies

Data Policies

16.39 Compliance Categories

FULLY_COMPLIANT

PARTIALLY_COMPLIANT

NON_COMPLIANT

UNDER_REVIEW

16.40 Violation Categories

POLICY_VIOLATION

SECURITY_VIOLATION

RANKING_VIOLATION

AUDIT_VIOLATION

TRACEABILITY_VIOLATION

COMPLIANCE_VIOLATION

16.41 Auditability Requirements

Every governance action generates:

Governance Event

↓

Audit Record

↓

Trace Record



Correlation ID

16.42 Traceability Requirements

Store:

Decision History

Approval History

Policy Version

Compliance Result

Violation Records

16.43 Reproducibility Requirements

Support:

Decision Replay

Approval Replay

Policy Replay

Compliance Replay

16.44 Observability Requirements

Metrics:

Decision Count

Approval Count

Violation Count

Compliance Rate

Validation Latency

16.45 Performance Requirements

Decision Creation

P95 < 500 ms

Approval Processing

P95 < 500 ms

Policy Validation

P95 < 1 Second

16.46 Security Requirements

Governance APIs Require:

Authentication

Authorization

Audit Logging

Trace Logging

Role Validation

16.47 Governance Rules

GOV-GOV-01

Governance decisions are auditable.

GOV-GOV-02

Governance decisions are traceable.

GOV-GOV-03

Policy validations are reproducible.

GOV-GOV-04

Approval history must be preserved.

GOV-GOV-05

Violations must be recorded.

GOV-GOV-06

Compliance reporting is mandatory.

16.48 Architectural Constraints

AC-GOV-01

Governance policies cannot be bypassed.

AC-GOV-02

Approval workflows cannot be bypassed.

AC-GOV-03

Decision history cannot be deleted.

AC-GOV-04

Violation history cannot be deleted.

AC-GOV-05

Policy validation is mandatory where applicable.

AC-GOV-06

Governance controls cannot be bypassed.

16.49 Section Summary

This section defines the Governance API architecture, policy validation framework, approval workflows, compliance reporting mechanisms, endpoint contracts, service interfaces, and governance controls.

The Governance APIs provide the enforcement layer ensuring that TODP remains aligned with its approved principles, architectural constraints, compliance requirements, and operational governance standards.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

🔗 Section 14 — Ranking APIs

🔗 Section 15 — Recommendation APIs

🔗 Section 16 — Governance APIs

🔗 Section 17 — Audit APIs

TDD V1 — Section 17

Audit APIs

Document: Technical Design Document (TDD V1)

Section: 17

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

17.1 Purpose

This section defines the Audit API specifications for TODP.

Audit APIs are responsible for:

- Audit record retrieval
- Audit event inspection
- Historical reconstruction
- Change tracking
- Traceability support
- Reproducibility support

Audit APIs provide visibility into all critical actions occurring within TODP.

Auditability is a mandatory platform principle.

17.2 Audit Architecture

Client

↓

Audit API

↓

Audit Service

↓

Audit Repository

↓

PostgreSQL

↓

Response DTO

↓

Client

17.3 Audit Principles

Every critical action must produce:

Audit Event



Audit Record



Trace Record



Correlation ID

Audit records are immutable.

17.4 Base Route

/api/v1/audits

17.5 Audit Endpoints

GET /records

GET /records/{record_id}

GET /entities/{entity_type}/{entity_id}

GET /correlations/{correlation_id}

GET /events

GET /snapshots

GET /snapshots/{snapshot_id}

GET /replay/{entity_type}/{entity_id}

17.6 Audit Records Endpoint

Endpoint:

GET /api/v1/audits/records

Purpose:

Retrieve audit records.

17.7 Query Parameters

?page=1

&page_size=20

&entity_type=opportunity

&action=create

&actor_id=<uuid>

17.8 Audit Record Structure

```
{  
  "record_id": "",  
  "entity_type": "",
```



```
"entity_id": "",  
"action": "",  
"actor_id": "",  
"created_at": ""  
}
```

17.9 Audit Records Response DTO

```
{  
  "records": []  
}
```

17.10 Get Audit Record Endpoint

Endpoint:

GET /api/v1/audits/records/{record_id}

Purpose:

Retrieve specific audit record.

17.11 Audit Record Detail DTO

```
{  
  "record_id": "",  
  "entity_type": "",  
  "entity_id": "",  
  "action": "",  
  "metadata": {},  
  "correlation_id": ""  
}
```

17.12 Entity Audit History Endpoint

Endpoint:

GET /api/v1/audits/entities/{entity_type}/{entity_id}

Purpose:

Retrieve entity history.

17.13 Entity History Response DTO

```
{  
  "entity_type": "",  
  "entity_id": "",  
  "history": []  
}
```

17.14 Correlation Search Endpoint

Endpoint:

GET /api/v1/audits/correlations/{correlation_id}

Purpose:

Retrieve all records associated with a correlation.

17.15 Correlation Response DTO

```
{  
  "correlation_id": "",  
  "records": []  
}
```

17.16 Audit Events Endpoint

Endpoint:

GET /api/v1/audits/events

Purpose:

Retrieve audit events.

17.17 Audit Event DTO

```
{  
  "event_id": "",  
  "event_type": "",  
  "timestamp": ""  
}
```

17.18 Snapshot Endpoint

Endpoint:

GET /api/v1/audits/snapshots

Purpose:

Retrieve historical snapshots.

17.19 Snapshot Response DTO

```
{  
  "snapshots": []  
}
```

17.20 Get Snapshot Endpoint

Endpoint:

GET /api/v1/audits/snapshots/{snapshot_id}

Purpose:

Retrieve specific snapshot.

17.21 Snapshot Detail DTO

```
{  
  "snapshot_id": "",  
  "entity_type": "",  
  "entity_id": "",  
  "version": 1,  
  "snapshot_data": {}  
}
```

17.22 Replay Endpoint

Endpoint:

GET /api/v1/audits/replay/{entity_type}/{entity_id}

Purpose:

Reconstruct historical state.

17.23 Replay Response DTO

```
{  
  "entity_type": "",  
  "entity_id": "",  
  "reconstructed_state": {}  
}
```

17.24 Audit Record Fields

record_id

entity_type

entity_id

action

actor_id

correlation_id

metadata

timestamp

17.25 Supported Entity Types

USER

OPPORTUNITY

AUTHORITY

EVIDENCE

RANKING

RECOMMENDATION

GOVERNANCE

SYSTEM

17.26 Supported Actions

CREATE

UPDATE

DELETE

VALIDATE

VERIFY

GENERATE

APPROVE

REJECT

LOGIN

LOGOUT

17.27 Audit Service Interface

AuditService

Methods:

get_record()

`list_records()`

`get_entity_history()`

`get_correlation_records()`

`get_snapshot()`

`replay()`

17.28 Snapshot Service Interface

`SnapshotService`

Methods:

`create_snapshot()`

`get_snapshot()`

`list_snapshots()`

`reconstruct()`

17.29 Replay Service Interface

ReplayService

Methods:

load_events()

load_snapshots()

reconstruct_state()

generate_output()

17.30 Repository Dependencies

Audit Service Uses:

AuditRepository

SnapshotRepository

TraceRepository

GovernanceRepository

17.31 Audit Repository Interface

Methods:

create()

get()

list()

get_by_entity()

get_by_correlation()

exists()

17.32 Snapshot Repository Interface

Methods:

create()

get()

list()

get_versions()

17.33 Replay Rules

Replay Must Support:

Entity Replay

Version Replay

Audit Replay

Decision Replay

Ranking Replay

Recommendation Replay

17.34 Historical Reconstruction Rules

Reconstruction Uses:

Audit Records

Snapshots

Version History

Trace Records

17.35 Correlation Tracking Rules

Every critical workflow stores:

Correlation ID

Across:

APIs

Services

Repositories

Audit Records

Trace Records

17.36 Audit Search Filters

Entity Type

Action

Actor

Date Range

Correlation ID

Version

Status

17.37 Error Scenarios

Record Not Found



404

Snapshot Not Found



404

Replay Failure



500

Unauthorized

↓

401

Forbidden

↓

403

17.38 Audit Storage Rules

Audit Records:

Immutable

No Physical Deletion

Append-Only Strategy

17.39 Reproducibility Requirements

Support:

Historical Replay

Version Replay

Decision Replay

Ranking Replay

Recommendation Replay

17.40 Traceability Requirements

Store:

Source Event

Target Entity

Actor

Timestamp

Correlation ID

Version

17.41 Auditability Requirements

Every critical action must generate:

Audit Record

Before Workflow Completion

Missing Audit Record

=

Governance Violation

17.42 Observability Requirements

Metrics:

Audit Record Count

Audit Query Count

Replay Count

Replay Failures

Snapshot Count

17.43 Performance Requirements

Audit Query

P95 < 500 ms

Entity History Query

P95 < 1 Second

Replay Request

P95 < 2 Seconds

Snapshot Retrieval

P95 < 500 ms

17.44 Security Requirements

Audit APIs Require:

Authentication

Authorization

Audit Logging

Role Validation

Data Access Validation

17.45 Governance Rules

AUDIT-GOV-01

Audit records are immutable.

AUDIT-GOV-02

Audit history must be preserved.

AUDIT-GOV-03

Critical actions require audit records.

AUDIT-GOV-04

Replay support is mandatory.

AUDIT-GOV-05

Snapshot support is mandatory.

AUDIT-GOV-06

Correlation tracking is mandatory.

17.46 Architectural Constraints

AC-AUDIT-01

Audit records cannot be modified.

AC-AUDIT-02

Audit records cannot be deleted.

AC-AUDIT-03

Replay must remain available.

AC-AUDIT-04

Historical reconstruction must remain available.

AC-AUDIT-05

Correlation tracking cannot be disabled.

AC-AUDIT-06

Governance controls cannot be bypassed.

17.47 Section Summary

This section defines the Audit API architecture, audit retrieval framework, replay capabilities, snapshot management, historical reconstruction mechanisms, endpoint contracts, and governance controls.

The Audit APIs provide the accountability foundation of TODP by ensuring that all critical actions remain traceable, auditable, reproducible, and historically reconstructable.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

🔗 Section 14 — Ranking APIs

🔗 Section 15 — Recommendation APIs

🔗 Section 16 — Governance APIs

🔗 Section 17 — Audit APIs

🔗 Section 18 — DTO & Schema Design

TDD V1 — Section 18

DTO & Schema Design

Document: Technical Design Document (TDD V1)

Section: 18

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

18.1 Purpose

This section defines the Data Transfer Object (DTO) and Schema architecture for TODP.

DTOs provide:

- API contracts
- Data validation
- Serialization
- Deserialization
- Service communication
- Type safety

DTOs separate external contracts from internal implementation.

18.2 DTO Design Principles

DTO-01

Strong typing.

DTO-02

Immutable where possible.

DTO-03

No business logic.

DTO-04

Explicit validation.

DTO-05

Versionable contracts.

DTO-06

Consistent structure.

18.3 Technology Standard

Framework:

Pydantic v2

Purpose:

Validation

Serialization

Documentation

Type Safety

18.4 Schema Repository Structure

backend/app/schemas/

├─ common/

├─ auth/

├─ opportunity/

├─ authority/

├─ evidence/

├─ retrieval/

├─ ranking/

├─ recommendation/

├─ governance/

└─ audit/

18.5 Schema Categories

Request Schemas

Response Schemas

Filter Schemas

Pagination Schemas

Error Schemas

Internal DTOs

Event DTOs

18.6 Common Schema Structure

schemas/common/

└─ pagination.py

└─ responses.py

└─ errors.py
└─ metadata.py
└─ enums.py

18.7 Base Request DTO

Purpose:

Parent DTO for requests.

Common Fields:

correlation_id

request_timestamp

18.8 Base Response DTO

Purpose:

Parent DTO for responses.

Fields:

success

data

timestamp

correlation_id

18.9 Standard Success Response

```
{  
  "success": true,  
  "data": {},  
  "timestamp": "",  
  "correlation_id": ""  
}
```

18.10 Standard Error Response

```
{  
  "success": false,  
  "errors": [],
```

```
"timestamp": "",  
"correlation_id": ""  
}
```

18.11 Error DTO Structure

Fields:

code

message

field

details

18.12 Pagination DTO

Fields:

page

page_size

total

total_pages

items

18.13 Authentication Schemas

schemas/auth/

├─ login_request.py

├─ login_response.py

├─ refresh_request.py

├─ user_response.py

└─ password_change.py

18.14 Login Request DTO

Class:

LoginRequest

Fields:

email

password

18.15 Login Response DTO

Class:

LoginResponse

Fields:

access_token

refresh_token

expires_in

token_type

18.16 Opportunity Schemas

schemas/opportunity/

|— create_request.py

|— update_request.py

|— response.py

|— search_request.py

└— filters.py

18.17 Create Opportunity DTO

Class:

CreateOpportunityRequest

Fields:

title

description

opportunity_type

institution_level

location_level

money_cost

time_cost

distance_cost

18.18 Opportunity Response DTO

Class:

OpportunityResponse

Fields:

id

title

description

status

created_at

18.19 Authority Schemas

schemas/authority/

└─ create_request.py

└─ verification_request.py

└─ response.py

└─ confidence_response.py

18.20 Authority DTOs

CreateAuthorityRequest

AuthorityResponse

AuthorityVerificationRequest

AuthorityConfidenceResponse

18.21 Evidence Schemas

schemas/evidence/

- |— create_request.py
- |— validate_request.py
- |— response.py
- |— lineage_response.py
- └— version_response.py

18.22 Evidence DTOs

CreateEvidenceRequest

EvidenceResponse

EvidenceValidationRequest

EvidenceVersionResponse

EvidenceLineageResponse

18.23 Retrieval Schemas

schemas/retrieval/

└─ search_request.py

└─ search_response.py

└─ result.py

└─ filters.py

18.24 Search Request DTO

Class:

SearchRequest

Fields:

query

institution_level

location_level

page

page_size

18.25 Search Result DTO

Fields:

opportunity_id

title

retrieval_score

retrieval_method

18.26 Ranking Schemas

schemas/ranking/

├─ execute_request.py

├─ result.py

├─ comparison.py

└─ formula_response.py

18.27 Ranking DTOs

RankingExecutionRequest

RankingResultResponse

RankingComparisonRequest

FormulaResponse

18.28 Ranking Result DTO

Fields:

opportunity_id

money_cost

time_cost

distance_cost

total_cost

rank_position

18.29 Recommendation Schemas

schemas/recommendation/

├─ generate_request.py

├─ response.py

├─ explanation.py

└─ execution_response.py

18.30 Recommendation DTOs

GenerateRecommendationRequest

RecommendationResponse

RecommendationExplanationResponse

ExecutionRecommendationsResponse

18.31 Governance Schemas

schemas/governance/

└─ decision_request.py

└─ approval_request.py

└─ compliance_response.py

└─ violation_response.py

18.32 Governance DTOs

DecisionRequest

ApprovalRequest

ComplianceResponse

ViolationResponse

18.33 Audit Schemas

schemas/audit/

└─ audit_record.py

└─ replay_response.py
└─ snapshot_response.py
└─ correlation_response.py

18.34 Audit DTOs

AuditRecordResponse

ReplayResponse

SnapshotResponse

CorrelationResponse

18.35 Internal Service DTOs

Purpose:

Service-to-Service Communication

Not Exposed Externally

Examples:

RankingInputDTO

RecommendationInputDTO

AuthorityValidationDTO

EvidenceValidationDTO

18.36 Event DTOs

Purpose:

Internal Event Communication

Examples:

OpportunityCreatedEvent

AuthorityVerifiedEvent

EvidenceValidatedEvent

RecommendationGeneratedEvent

18.37 Enum Schema Design

Centralized Enums

Location:

schemas/common/enums.py

18.38 Core Enums

OpportunityType

InstitutionLevel

AuthorityType

EvidenceStatus

VerificationStatus

RecommendationStatus

DecisionStatus

18.39 Validation Rules

Required Fields

Length Constraints

Range Constraints

Regex Constraints

Enum Validation

UUID Validation

18.40 UUID Standard

All IDs:

UUID

Validated at schema level.

18.41 DateTime Standard

Format:

ISO 8601 UTC

Example:

2026-06-14T10:30:00Z

18.42 Serialization Rules

Response DTOs:

JSON Serializable

Datetime Converted

UUID Converted

Enum Converted

18.43 Deserialization Rules

Incoming Requests:

JSON

↓

Validation

↓

DTO Conversion

↓

Service Layer

18.44 DTO Versioning Strategy

Breaking Change



New DTO Version



Backward Compatibility Period

Examples:

OpportunityResponseV1

OpportunityResponseV2

18.45 DTO Naming Standards

Requests:

<CreateEntity>Request

Responses:

<Entity>Response

Events:

<Entity><Action>Event

18.46 Schema Governance Rules

DTO-GOV-01

All APIs require DTOs.

DTO-GOV-02

DTOs cannot contain business logic.

DTO-GOV-03

Validation occurs at schema boundary.

DTO-GOV-04

Contracts must be versionable.

DTO-GOV-05

Enums must be centralized.

DTO-GOV-06

Response formats must remain consistent.

18.47 Architectural Constraints

AC-DTO-01

Controllers cannot bypass DTO validation.

AC-DTO-02

Services cannot accept raw request payloads.

AC-DTO-03

DTOs cannot contain persistence logic.

AC-DTO-04

DTOs cannot contain business workflows.

AC-DTO-05

Contract consistency is mandatory.

AC-DTO-06

Governance controls cannot be bypassed.

18.48 Section Summary

This section defines the DTO and schema architecture, validation framework, serialization standards, contract versioning strategy, schema organization, and governance controls.

The DTO layer provides the type-safe contract foundation connecting APIs, services, repositories, and internal workflows throughout TODP.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

🔗 Section 14 — Ranking APIs

🔗 Section 15 — Recommendation APIs

🔗 Section 16 — Governance APIs

🔗 Section 17 — Audit APIs

🔗 Section 18 — DTO & Schema Design

🔗 Section 19 — Domain Service Interfaces

TDD V1 — Section 19

Domain Service Interfaces

Document: Technical Design Document (TDD V1)

Section: 19

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

19.1 Purpose

This section defines the Domain Service Interface architecture for TODP.

Domain Services encapsulate:

- Business workflows
- Domain orchestration
- Domain validations
- Domain policies
- Cross-entity coordination

Domain Services represent the business behavior layer between APIs and repositories.

19.2 Domain Service Principles

DS-01

Business logic belongs in services.

DS-02

Services are domain-oriented.

DS-03

Services are interface-driven.

DS-04

Services are testable.

DS-05

Services are auditable.

DS-06

Services are traceable.

19.3 Service Layer Position

API Layer



Domain Services



Repositories



Infrastructure

Services coordinate workflows.

Repositories provide persistence.

19.4 Service Repository Structure

backend/app/services/

- |— authentication/
- |— authorization/
- |— opportunity/
- |— authority/
- |— evidence/
- |— retrieval/
- |— ranking/
- |— recommendation/
- |— governance/
- └— audit/

19.5 Service Interface Standard

Every service exposes:

Public Interface

Input DTOs

Output DTOs

Domain Exceptions

Audit Events

19.6 Authentication Service Interface

Interface:

AuthenticationService

Methods:

login()

refresh()

logout()

validate()

change_password()

forgot_password()

reset_password()

19.7 Authentication Service Inputs

LoginRequest

RefreshRequest

PasswordChangeRequest

PasswordResetRequest

19.8 Authentication Service Outputs

LoginResponse

UserResponse

ValidationResponse

PasswordChangeResponse

19.9 Authorization Service Interface

Interface:

AuthorizationService

Methods:

has_permission()

has_role()

authorize()

resolve_permissions()

resolve_roles()

19.10 Authorization Service Responsibilities

Permission Validation

Role Resolution

Policy Enforcement

Access Decisions

19.11 Opportunity Service Interface

Interface:

OpportunityService

Methods:

create()

get_by_id()

list()

update()

patch()

delete()

search()

19.12 Opportunity Service Responsibilities

Opportunity Lifecycle

Business Validation

Version Management

Audit Generation

Trace Generation

19.13 Authority Service Interface

Interface:

AuthorityService

Methods:

create()

get_by_id()

list()

update()

delete()

verify()

get_confidence()

19.14 Authority Service Responsibilities

Authority Lifecycle

Authority Verification

Confidence Management

Verification History

Audit Generation

19.15 Authority Verification Service Interface

Interface:

AuthorityVerificationService

Methods:

verify_authority()

calculate_confidence()

validate_evidence()

generate_result()

19.16 Authority Verification Outputs

Verification Result

Confidence Score

Evidence Links

Audit Events

19.17 Evidence Service Interface

Interface:

EvidenceService

Methods:

create()

get_by_id()

list()

update()

delete()

validate()

get_versions()

get_lineage()

19.18 Evidence Service Responsibilities

Evidence Lifecycle

Versioning

Lineage Management

Validation

Historical Preservation

19.19 Evidence Validation Service Interface

Interface:

EvidenceValidationService

Methods:

validate_source()

validate_content()

validate_lineage()

validate_authority()

generate_result()

19.20 Retrieval Service Interface

Interface:

RetrievalService

Methods:

search()

keyword_search()

semantic_search()

hybrid_search()

get_search()

get_results()

19.21 Retrieval Service Responsibilities

Candidate Generation

Search Execution

Search History

Hybrid Retrieval

Fusion Coordination

19.22 BM25 Service Interface

Interface:

BM25Service

Methods:

index()

tokenize()

search()

score()

19.23 Vector Service Interface

Interface:

VectorService

Methods:

embed()

index()

search()

similarity()

19.24 Fusion Service Interface

Interface:

FusionService

Methods:

merge()

normalize()

rerank()

generate_candidates()

19.25 Ranking Service Interface

Interface:

RankingService

Methods:

execute()

compare()

get_execution()

get_results()

get_formula()

19.26 Ranking Service Responsibilities

Ranking Execution

Formula Application

Result Generation

History Preservation

Audit Generation

19.27 Ranking Engine Interface

Interface:

RankingEngine

Methods:

calculate_total_cost()

rank()

resolve_ties()

validate_inputs()

19.28 Cost Calculator Interface

Interface:

CostCalculator

Methods:

calculate_money_cost()

calculate_time_cost()

calculate_distance_cost()

calculate_total_cost()

19.29 Recommendation Service Interface

Interface:

RecommendationService

Methods:

generate()

get()

list()

get_evidence()

get_authorities()

get_explanation()

19.30 Recommendation Service Responsibilities

Dependency Validation

Recommendation Generation

Explanation Generation

Traceability Support

Audit Support

19.31 Recommendation Engine Interface

Interface:

RecommendationEngine

Methods:

generate_recommendation()

build_explanation()

validate_dependencies()

generate_links()

19.32 Governance Service Interface

Interface:

GovernanceService

Methods:

create_decision()

update_decision()

approve()

validate_policy()

get_compliance()

get_violations()

19.33 Governance Service Responsibilities

Policy Enforcement

Approval Workflows

Compliance Validation

Violation Detection

Decision Tracking

19.34 Policy Engine Interface

Interface:

PolicyEngine

Methods:

load_policies()

validate()

detect_violation()

generate_result()

19.35 Approval Engine Interface

Interface:

ApprovalEngine

Methods:

submit_approval()

validate_reviewer()

update_status()

generate_history()

19.36 Audit Service Interface

Interface:

AuditService

Methods:

get_record()

list_records()

get_entity_history()

get_snapshot()

replay()

19.37 Audit Service Responsibilities

Audit Retrieval

Historical Reconstruction

Replay Support

Correlation Tracking

Snapshot Retrieval

19.38 Snapshot Service Interface

Interface:

SnapshotService

Methods:

create_snapshot()

get_snapshot()

list_snapshots()

reconstruct()

19.39 Replay Service Interface

Interface:

ReplayService

Methods:

load_events()

load_snapshots()

reconstruct_state()

generate_output()

19.40 Domain Service Communication Rules

Service

↓

Repository

Allowed

Service



Service

Allowed Through Interfaces

Repository



Service

Prohibited

19.41 Dependency Injection Standard

All Services Registered Through:

FastAPI Dependency Injection

Benefits:

Loose Coupling

Testing

Mocking

Replacement

19.42 Service Exception Hierarchy

Base Exception

↓

ValidationException

↓

BusinessRuleException

↓

AuthorizationException



GovernanceException



InfrastructureException

19.43 Service Audit Requirements

Every critical service operation generates:

Audit Event



Audit Record



Trace Record



Correlation ID

19.44 Service Traceability Requirements

Services Must Store:

Inputs

Outputs

Execution Metadata

Correlation IDs

Version Information

19.45 Service Reproducibility Requirements

Services Must Support:

Execution Replay

Historical Reconstruction

Version Replay

Decision Replay

19.46 Service Performance Requirements

CRUD Services

P95 < 500 ms

Retrieval Services

P95 < 2 Seconds

Ranking Services

P95 < 1 Second

Recommendation Services

P95 < 500 ms

19.47 Service Governance Rules

SERVICE-GOV-01

Business logic belongs in services.

SERVICE-GOV-02

Services must be interface-driven.

SERVICE-GOV-03

Critical services require auditing.

SERVICE-GOV-04

Critical services require traceability.

SERVICE-GOV-05

Service contracts must remain versioned.

SERVICE-GOV-06

Service execution must be reproducible.

19.48 Architectural Constraints

AC-SERVICE-01

Business logic cannot exist in controllers.

AC-SERVICE-02

Business logic cannot exist in repositories.

AC-SERVICE-03

Services cannot bypass governance.

AC-SERVICE-04

Services cannot bypass audit controls.

AC-SERVICE-05

Services cannot bypass traceability controls.

AC-SERVICE-06

Governance controls cannot be bypassed.

19.49 Section Summary

This section defines the Domain Service Interface architecture, service responsibilities, engine interfaces, dependency rules, orchestration patterns, and governance controls.

The Domain Service layer provides the core business execution framework that transforms TODP policies, rules, and workflows into controlled, auditable, traceable, and reproducible platform behavior.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

- 🔗 Section 9 — Authentication APIs
- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces

🔗 Section 20 — Repository Interfaces

TDD V1 — Section 20

Repository Interfaces

Document: Technical Design Document (TDD V1)

Section: 20

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

20.1 Purpose

This section defines the Repository Interface architecture for TODP.

Repositories are responsible for:

- Persistence abstraction
- Database access
- Query execution
- Transaction participation
- Data retrieval
- Data persistence

Repositories isolate business services from database implementation details.

20.2 Repository Principles

REP-01

Repositories own persistence.

REP-02

Repositories do not contain business logic.

REP-03

Repositories expose interfaces.

REP-04

Repositories support testing.

REP-05

Repositories support auditability.

REP-06

Repositories support traceability.

20.3 Repository Layer Position

API Layer

↓

Domain Services

↓

Repositories

↓

Database

Repositories form the persistence boundary.

20.4 Repository Structure

backend/app/repositories/

|— user_repository.py

|— role_repository.py

|— permission_repository.py

|— session_repository.py

|— opportunity_repository.py

- |— authority_repository.py
- |— evidence_repository.py
- |— search_repository.py
- |— embedding_repository.py
- |— ranking_repository.py
- |— recommendation_repository.py
- |— governance_repository.py
- |— audit_repository.py
- |— snapshot_repository.py
- └— trace_repository.py

20.5 Base Repository Interface

Interface:

BaseRepository

Purpose:

Shared persistence contract.

20.6 Base Repository Methods

create()

get()

update()

delete()

exists()

list()

count()

20.7 Repository Return Rules

Repositories return:

Models

Internal DTOs

Collections

Repositories do not return:

HTTP Responses

API DTOs

UI Objects

20.8 User Repository Interface

Interface:

UserRepository

Methods:

create()

get_by_id()

get_by_email()

update()

delete()

exists()

20.9 User Repository Responsibilities

User Persistence

User Lookup

Authentication Support

Session Support

20.10 Role Repository Interface

Interface:

RoleRepository

Methods:

create()

get()

list()

update()

delete()

20.11 Permission Repository Interface

Interface:

PermissionRepository

Methods:

create()

get()

list()

exists()

20.12 Session Repository Interface

Interface:

SessionRepository

Methods:

create_session()

get_session()

revoke_session()

list_sessions()

cleanup_expired()

20.13 Opportunity Repository Interface

Interface:

OpportunityRepository

Methods:

create()

get()

list()

update()

delete()

search()

exists()

20.14 Opportunity Repository Queries

Query By:

ID

Status

Institution Level

Location Level

Opportunity Type

20.15 Authority Repository Interface

Interface:

AuthorityRepository

Methods:

create()

get()

list()

update()

delete()

exists()

search()

20.16 Authority Repository Queries

Query By:

Authority Type

Verification Status

Confidence Score

Country

20.17 Evidence Repository Interface

Interface:

EvidenceRepository

Methods:

create()

get()

list()

update()

delete()

exists()

get_versions()

get_lineage()

20.18 Evidence Repository Queries

Query By:

Status

Source Type

Version

Validation State

20.19 Search Repository Interface

Interface:

SearchRepository

Methods:

create_search()

get_search()

store_results()

get_results()

delete_expired()

20.20 Search Repository Responsibilities

Search History

Search Metadata

Search Replay

Search Results

20.21 Embedding Repository Interface

Interface:

EmbeddingRepository

Methods:

create_embedding()

get_embedding()

search_similar()

delete_embedding()

20.22 Embedding Repository Queries

Query By:

Entity Type

Entity ID

Vector Similarity

20.23 Ranking Repository Interface

Interface:

RankingRepository

Methods:

create_execution()

get_execution()

store_results()

get_results()

get_formula()

20.24 Ranking Repository Responsibilities

Execution Storage

Result Storage

Replay Support

Version Support

20.25 Recommendation Repository Interface

Interface:

RecommendationRepository

Methods:

create()

get()

list()

get_by_execution()

get_evidence_links()

get_authority_links()

20.26 Recommendation Repository Responsibilities

Recommendation Storage

Recommendation Retrieval

Evidence Linking

Authority Linking

20.27 Governance Repository Interface

Interface:

GovernanceRepository

Methods:

create_decision()

get_decision()

update_decision()

store_approval()

get_violations()

get_compliance()

20.28 Governance Repository Responsibilities

Decision Storage

Approval Storage

Violation Storage

Compliance Storage

20.29 Audit Repository Interface

Interface:

AuditRepository

Methods:

create_record()

get_record()

list_records()

get_by_entity()

get_by_correlation()

20.30 Audit Repository Responsibilities

Audit Persistence

Audit Search

Audit Retrieval

Audit Replay Support

20.31 Snapshot Repository Interface

Interface:

SnapshotRepository

Methods:

create_snapshot()

get_snapshot()

list_snapshots()

get_versions()

20.32 Snapshot Repository Responsibilities

Historical State Storage

Replay Support

Version Preservation

Reconstruction Support

20.33 Trace Repository Interface

Interface:

TraceRepository

Methods:

create_trace()

get_trace()

list_traces()

get_relationships()

20.34 Trace Repository Responsibilities

Relationship Storage

Trace Retrieval

Lineage Support

Dependency Tracking

20.35 Query Design Rules

Queries Must Be:

Explicit

Parameterized

Auditable

Traceable

Optimized

20.36 Transaction Rules

Repositories Participate In:

Transactions

Repositories Do Not Control:

Business Decisions

Workflow Decisions

20.37 Unit of Work Pattern

Service

↓

Repository Operations



Transaction Commit



Audit Record

Rollback On Failure

20.38 Pagination Standards

Supported By:

List Queries

Search Queries

History Queries

Audit Queries

20.39 Filtering Standards

Supported Filters:

Status

Date Range

Version

Type

Correlation ID

20.40 Sorting Standards

Supported Sorting:

created_at

updated_at

status

rank_position

confidence_score

20.41 Repository Exception Types

RepositoryException

EntityNotFoundException

DuplicateEntityException

ConstraintViolationException

TransactionException

20.42 Repository Performance Requirements

Simple Lookup

P95 < 100 ms

Filtered Query

P95 < 300 ms

Search Query

P95 < 500 ms

Batch Query

P95 < 1 Second

20.43 Auditability Requirements

Repository Operations Generate:

Repository Event



Audit Record



Trace Record



Correlation ID

20.44 Traceability Requirements

Repositories Store:

Entity Relationships

Version History

Correlation Metadata

Execution Metadata

20.45 Reproducibility Requirements

Repositories Support:

Historical Retrieval

Replay Retrieval

Version Retrieval

Snapshot Retrieval

20.46 Testing Requirements

Repository Tests Include:

CRUD Tests

Constraint Tests

Transaction Tests

Rollback Tests

Performance Tests

20.47 Repository Governance Rules

REPO-GOV-01

Repositories own persistence only.

REPO-GOV-02

Repositories cannot contain business logic.

REPO-GOV-03

Repositories must expose interfaces.

REPO-GOV-04

Repository operations must be auditable.

REPO-GOV-05

Repository operations must be traceable.

REPO-GOV-06

Repositories must support reproducibility.

20.48 Architectural Constraints

AC-REPO-01

Repositories cannot access APIs.

AC-REPO-02

Repositories cannot access controllers.

AC-REPO-03

Repositories cannot contain workflow logic.

AC-REPO-04

Repositories cannot bypass transactions.

AC-REPO-05

Repositories cannot bypass audit controls.

AC-REPO-06

Governance controls cannot be bypassed.

20.49 Section Summary

This section defines the Repository Interface architecture, persistence contracts, query standards, transaction boundaries, repository responsibilities, and governance controls.

The Repository layer provides the persistence abstraction foundation that enables TODP services to remain database-agnostic, testable, auditable, traceable, and reproducible.

TDD Progress

☐ Section 1 — Document Overview & Technical Scope

☐ Section 2 — Repository Technical Structure

☐ Section 3 — Backend Package Design

☐ Section 4 — Frontend Package Design

☐ Section 5 — Database Physical Schema Design

☐ Section 6 — SQLAlchemy Models Design

☐ Section 7 — Alembic Migration Design

☐ Section 8 — API Contract Standards

☐ Section 9 — Authentication APIs

☐ Section 10 — Opportunity APIs

- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces

🔗 Section 21 — Event Contracts

TDD V1 — Section 21

Event Contracts

Document: Technical Design Document (TDD V1)

Section: 21

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

- IAD V1 (Frozen)

21.1 Purpose

This section defines the Event Contract architecture for TODP.

Event Contracts are responsible for:

- Inter-service communication
- Workflow notifications
- Audit generation
- Trace generation
- Observability support
- Historical reconstruction

Events represent significant domain occurrences.

Events are immutable records of facts.

21.2 Event Principles

EVT-01

Events represent completed facts.

EVT-02

Events are immutable.

EVT-03

Events are versioned.

EVT-04

Events are traceable.

EVT-05

Events are auditable.

EVT-06

Events are reproducible.

21.3 Event Architecture

Domain Service



Event Creation



Event Publisher



Event Store



Consumers



Audit / Trace / Observability

21.4 Event Categories

Domain Events

System Events

Audit Events

Governance Events

Security Events

Observability Events

21.5 Event Repository Structure

backend/app/events/

├─ common/

├─ auth/

├─ opportunity/

├─ authority/

├─ evidence/

├─ retrieval/

├─ ranking/

├─ recommendation/

├─ governance/

├─ audit/

└─ observability/

21.6 Base Event Contract

Every event contains:

event_id

event_type

event_version

correlation_id

timestamp

producer

payload

21.7 Base Event Structure

{

 "event_id": "",

```
"event_type": "",  
"event_version": "v1",  
"correlation_id": "",  
"timestamp": "",  
"producer": "",  
"payload": {}  
}
```

21.8 Event Identifier Standard

Type:

UUID

Requirement:

Globally Unique

Immutable

21.9 Event Versioning Standard

Format:

v1

v2

v3

Breaking changes require new versions.

21.10 Correlation Standard

Every event contains:

correlation_id

Purpose:

Workflow reconstruction

Tracing

Auditing

Replay

21.11 Authentication Events

Event Types:

UserLoggedIn

UserLoggedOut

PasswordChanged

PasswordResetRequested

PasswordResetCompleted

21.12 Authentication Event Example

UserLoggedIn

Payload:

user_id

email

login_timestamp

21.13 Opportunity Events

Event Types:

OpportunityCreated

OpportunityUpdated

OpportunityDeleted

OpportunityArchived

OpportunityViewed

21.14 OpportunityCreated Contract

Payload:

opportunity_id

title

opportunity_type

created_by

version

21.15 Authority Events

Event Types:

AuthorityCreated

AuthorityUpdated

AuthorityVerified

AuthorityRejected

AuthorityArchived

21.16 AuthorityVerified Contract

Payload:

authority_id

verification_status

confidence_score

verified_at

21.17 Evidence Events

Event Types:

EvidenceCreated

EvidenceUpdated

EvidenceValidated

EvidenceRejected

EvidenceVersionCreated

21.18 EvidenceValidated Contract

Payload:

evidence_id

validation_result

validated_at

version

21.19 Retrieval Events

Event Types:

SearchExecuted

KeywordSearchExecuted

SemanticSearchExecuted

HybridSearchExecuted

SearchReplayExecuted

21.20 SearchExecuted Contract

Payload:

search_id

query

retrieval_method

result_count

execution_time

21.21 Ranking Events

Event Types:

RankingExecuted

RankingCompared

RankingReplayExecuted

FormulaViewed

21.22 RankingExecuted Contract

Payload:

execution_id

search_id

formula_version

result_count

execution_time

21.23 Recommendation Events

Event Types:

RecommendationGenerated

RecommendationViewed

RecommendationExplained

RecommendationReplayed

21.24 RecommendationGenerated Contract

Payload:

recommendation_id

ranking_execution_id

recommendation_version

generated_at

21.25 Governance Events

Event Types:

DecisionCreated

DecisionApproved

DecisionRejected

PolicyValidated

ViolationDetected

21.26 DecisionApproved Contract

Payload:

decision_id

approval_result

approved_by

approved_at

21.27 Audit Events

Event Types:

AuditRecordCreated

SnapshotCreated

ReplayExecuted

CorrelationTracked

21.28 AuditRecordCreated Contract

Payload:

record_id

entity_type

entity_id

action

created_at

21.29 Security Events

Event Types:

AuthenticationFailure

AuthorizationFailure

RateLimitExceeded

PermissionViolation

SecurityAlertGenerated

21.30 Observability Events

Event Types:

MetricGenerated

TraceGenerated

AlertGenerated

PerformanceThresholdExceeded

21.31 Event Publisher Interface

Interface:

EventPublisher

Methods:

publish()

publish_batch()

validate()

serialize()

21.32 Event Consumer Interface

Interface:

EventConsumer

Methods:

consume()

validate()

process()

acknowledge()

21.33 Event Validation Rules

Required Fields

Event Type

Event Version

Correlation ID

Timestamp

Payload

21.34 Event Serialization Standard

Format:

JSON

Encoding:

UTF-8

21.35 Event Storage Requirements

Store:

Event Metadata

Payload

Timestamp

Version

Correlation ID

Producer

21.36 Event Ordering Rules

Ordering Guaranteed:

Within Single Workflow

Using Correlation ID

Cross-Workflow Ordering Not Guaranteed

21.37 Event Replay Rules

Supported:

Audit Replay

Ranking Replay

Recommendation Replay

Governance Replay

Evidence Replay

21.38 Event Retention Requirements

Critical Events:

Permanent Retention

Operational Events:

Configurable Retention

21.39 Event Search Filters

Event Type

Date Range

Entity Type

Correlation ID

Producer

Version

21.40 Event Traceability Requirements

Store:

Source Service

Target Workflow

Entity Reference

Correlation ID

Version

21.41 Event Auditability Requirements

Every event creates:

Audit Link

↓

Trace Link

↓

Observability Link

21.42 Event Reproducibility Requirements

Events must support:

Replay

Reconstruction

Version Replay

Workflow Replay

Decision Replay

21.43 Event Performance Requirements

Event Publish

P95 < 50 ms

Event Validation

P95 < 10 ms

Event Retrieval

P95 < 200 ms

21.44 Event Security Requirements

Events Require:

Validation

Integrity Checks

Audit Logging

Access Control

21.45 Event Governance Rules

EVENT-GOV-01

Events are immutable.

EVENT-GOV-02

Events are versioned.

EVENT-GOV-03

Events are auditable.

EVENT-GOV-04

Events are traceable.

EVENT-GOV-05

Events support replay.

EVENT-GOV-06

Critical events require permanent retention.

21.46 Architectural Constraints

AC-EVENT-01

Events cannot be modified after creation.

AC-EVENT-02

Events cannot bypass audit controls.

AC-EVENT-03

Events cannot bypass traceability controls.

AC-EVENT-04

Breaking changes require version increments.

AC-EVENT-05

Correlation IDs are mandatory.

AC-EVENT-06

Governance controls cannot be bypassed.

21.47 Section Summary

This section defines the Event Contract architecture, event structures, event categories, publisher and consumer interfaces, replay capabilities, and governance controls.

The Event layer provides the immutable communication and historical record foundation that enables TODP to maintain auditability, traceability, observability, reproducibility, and workflow transparency.

TDD Progress

🔗 [Section 1 — Document Overview & Technical Scope](#)

🔗 [Section 2 — Repository Technical Structure](#)

🔗 [Section 3 — Backend Package Design](#)

🔗 [Section 4 — Frontend Package Design](#)

🔗 [Section 5 — Database Physical Schema Design](#)

🔗 [Section 6 — SQLAlchemy Models Design](#)

🔗 [Section 7 — Alembic Migration Design](#)

🔗 [Section 8 — API Contract Standards](#)

🔗 [Section 9 — Authentication APIs](#)

🔗 [Section 10 — Opportunity APIs](#)

🔗 [Section 11 — Authority APIs](#)

🔗 [Section 12 — Evidence APIs](#)

🔗 [Section 13 — Retrieval APIs](#)

🔗 [Section 14 — Ranking APIs](#)

🔗 [Section 15 — Recommendation APIs](#)

🔗 [Section 16 — Governance APIs](#)

🔗 [Section 17 — Audit APIs](#)

🔗 [Section 18 — DTO & Schema Design](#)

🔗 [Section 19 — Domain Service Interfaces](#)

🔗 [Section 20 — Repository Interfaces](#)

🔗 Section 21 — Event Contracts

🔗 Section 22 — Redis Key Design

TDD V1 — Section 22

Redis Key Design

Document: Technical Design Document (TDD V1)

Section: 22

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

22.1 Purpose

This section defines the Redis caching architecture and key design standards for TODP.

Redis is responsible for:

- Response caching
- Session caching
- Authorization caching
- Search caching
- Rate limiting
- Temporary workflow storage

Redis is a performance optimization layer.

Redis is not the source of truth.

PostgreSQL remains the authoritative datastore.

22.2 Redis Architecture

Client



API Layer



Redis Cache



Domain Services



PostgreSQL



Cache Hit



Return Cached Result



Cache Miss



Load From Database



Populate Cache



Return Result

22.3 Redis Principles

REDIS-01

Cache only derived data.

REDIS-02

Do not store business truth.

REDIS-03

All keys must be versioned.

REDIS-04

Keys must be namespaced.

REDIS-05

Expiration must be defined.

REDIS-06

Cache invalidation must be explicit.

22.4 Redis Naming Convention

Format:

todp:<domain>:<resource>:<identifier>

Examples:

todp:user:123

todp:opportunity:456

todp:ranking:789

22.5 Environment Prefixes

Development:

dev:todp

Testing:

test:todp

Production:

prod:todp

22.6 Authentication Cache Keys

Purpose:

Session and authentication caching.

Pattern:

todp:auth:session:{session_id}

TTL:

30 Days

22.7 User Permission Cache

Pattern:

todp:auth:permissions:{user_id}

TTL:

1 Hour

Stored Data:

Roles

Permissions

Policy References

22.8 User Profile Cache

Pattern:

todp:user:profile:{user_id}

TTL:

15 Minutes

22.9 Opportunity Cache Keys

Pattern:

todp:opportunity:{opportunity_id}

TTL:

30 Minutes

Stored Data:

Opportunity DTO

Version

Metadata

22.10 Opportunity Search Cache

Pattern:

todp:opportunity:search:{query_hash}

TTL:

15 Minutes

Stored Data:

Search Results

Filters

Pagination

22.11 Authority Cache Keys

Pattern:

todp:authority:{authority_id}

TTL:

30 Minutes

22.12 Authority Verification Cache

Pattern:

todp:authority:verification:{authority_id}

TTL:

1 Hour

Stored Data:

Verification Result

Confidence Score

Status

22.13 Evidence Cache Keys

Pattern:

todp:evidence:{evidence_id}

TTL:

30 Minutes

22.14 Evidence Lineage Cache

Pattern:

todp:evidence:lineage:{evidence_id}

TTL:

1 Hour

22.15 Retrieval Search Cache

Pattern:

todp:retrieval:search:{search_hash}

TTL:

15 Minutes

Stored Data:

Candidate Set

Query Metadata

Execution Data

22.16 Ranking Execution Cache

Pattern:

todp:ranking:execution:{execution_id}

TTL:

30 Minutes

Stored Data:

Ranking Results

Formula Version

Execution Metadata

22.17 Recommendation Cache

Pattern:

todp:recommendation:{recommendation_id}

TTL:

30 Minutes

Stored Data:

Recommendation

Explanation

Authority Links

Evidence Links

22.18 Governance Cache

Pattern:

todp:governance:policy:{policy_id}

TTL:

1 Hour

Stored Data:

Policy Rules

Version

Status

22.19 Audit Cache

Pattern:

todp:audit:entity:{entity_id}

TTL:

10 Minutes

Purpose:

Accelerated audit lookup.

22.20 Trace Cache

Pattern:

todp:trace:{correlation_id}

TTL:

30 Minutes

Purpose:

Workflow tracing.

22.21 Rate Limiting Keys

Pattern:

todp:ratelimit:{user_id}

TTL:

1 Hour

22.22 IP Rate Limiting Keys

Pattern:

todp:ratelimit:ip:{ip_address}

TTL:

1 Hour

22.23 Distributed Lock Keys

Pattern:

todp:lock:{resource_id}

TTL:

60 Seconds

Purpose:

Concurrency protection.

22.24 Idempotency Keys

Pattern:

todp:idempotency:{request_id}

TTL:

24 Hours

Purpose:

Duplicate request prevention.

22.25 Feature Flag Cache

Pattern:

todp:feature:{flag_name}

TTL:

15 Minutes

Stored Data:

Feature State

Version

Metadata

22.26 Health Cache

Pattern:

todp:health:{component}

TTL:

1 Minute

Stored Data:

Status

Timestamp

Metrics

22.27 Metrics Cache

Pattern:

todp:metrics:{metric_name}

TTL:

5 Minutes

Purpose:

Dashboard acceleration.

22.28 Cache Invalidation Rules

Invalidate On:

Create

Update

Delete

Verification

Validation

Approval

22.29 Opportunity Cache Invalidation

Opportunity Updated



Delete Cache



Rebuild On Next Request

22.30 Authority Cache Invalidation

Authority Verified



Delete Cache



Rebuild On Next Request

22.31 Evidence Cache Invalidation

Evidence Validated



Delete Cache



Rebuild On Next Request

22.32 Ranking Cache Invalidation

New Ranking Execution



Create New Cache Entry

Historical Entries Preserved

22.33 Recommendation Cache Invalidation

Recommendation Regenerated



Invalidate Previous Cache



Store New Result

22.34 Redis Data Types

Strings

Hashes

Sets

Sorted Sets

Lists

Streams

22.35 Preferred Redis Structures

Simple Object



Hash

Search Result Set



Sorted Set

Rate Limit



Counter

22.36 Cache Serialization Standard

Format:

JSON

Encoding:

UTF-8

Compression:

Optional

22.37 Cache Security Requirements

Sensitive Data:

Encrypted If Cached

Access Controlled

Audit Logged

22.38 Cache Monitoring Metrics

Hit Rate

Miss Rate

Evictions

Memory Usage

Latency

Key Count

22.39 Cache Performance Targets

Cache Lookup

P95 < 5 ms

Cache Write

P95 < 10 ms

Cache Invalidation

P95 < 20 ms

22.40 Cache Auditability Requirements

Cache Operations Generate:

Cache Event

↓

Audit Event

↓

Trace Event

22.41 Cache Traceability Requirements

Store:

Cache Key

Entity Reference

Version

Correlation ID

Timestamp

22.42 Cache Reproducibility Rules

Cache Is Disposable

Source Of Truth:

PostgreSQL

Cache Can Be Rebuilt

At Any Time

22.43 Redis Governance Rules

REDIS-GOV-01

Redis is not the source of truth.

REDIS-GOV-02

All keys must be namespaced.

REDIS-GOV-03

All keys require TTL.

REDIS-GOV-04

Cache invalidation must be explicit.

REDIS-GOV-05

Sensitive cache entries require protection.

REDIS-GOV-06

Cache rebuild must always be possible.

22.44 Architectural Constraints

AC-REDIS-01

Business truth cannot exist only in Redis.

AC-REDIS-02

Permanent records cannot exist only in Redis.

AC-REDIS-03

Redis outages cannot corrupt data.

AC-REDIS-04

Cache keys must remain versioned.

AC-REDIS-05

TTL values are mandatory.

AC-REDIS-06

Governance controls cannot be bypassed.

22.45 Section Summary

This section defines the Redis caching architecture, key naming conventions, cache invalidation strategy, TTL standards, performance objectives, and governance controls.

The Redis layer provides high-performance caching, session management, rate limiting, and workflow acceleration while preserving PostgreSQL as the authoritative source of truth.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

- 🔗 Section 3 — Backend Package Design
- 🔗 Section 4 — Frontend Package Design
- 🔗 Section 5 — Database Physical Schema Design
- 🔗 Section 6 — SQLAlchemy Models Design
- 🔗 Section 7 — Alembic Migration Design
- 🔗 Section 8 — API Contract Standards
- 🔗 Section 9 — Authentication APIs
- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design

- 🔗 Section 23 — Logging Architecture

TDD V1 — Section 23

Logging Architecture

Document: Technical Design Document (TDD V1)

Section: 23

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

23.1 Purpose

This section defines the Logging Architecture for TODP.

Logging is responsible for:

- Operational visibility
- Troubleshooting
- Security monitoring
- Audit support
- Traceability support
- Incident investigation

Logging records system behavior.

Logging does not replace auditing.

Logging does not replace tracing.

23.2 Logging Architecture

Application



Structured Logger



Log Aggregation



Storage



Monitoring



Alerting



Investigation

23.3 Logging Principles

LOG-01

All logs are structured.

LOG-02

Logs are machine readable.

LOG-03

Logs are searchable.

LOG-04

Logs support correlation IDs.

LOG-05

Sensitive data is protected.

LOG-06

Logs support observability.

23.4 Logging Technology

Application Logging:

Python Logging

Structured Logging:

JSON Format

Aggregation:

Centralized Log Collection

23.5 Log Categories

Application Logs

Security Logs

Access Logs

Error Logs

Performance Logs

Integration Logs

Infrastructure Logs

23.6 Log Levels

DEBUG

INFO

WARNING

ERROR

CRITICAL

23.7 Log Format Standard

Every log entry contains:

timestamp

level

service

message

correlation_id

metadata

23.8 Standard Log Structure

```
{  
  "timestamp": "",  
  "level": "INFO",  
  "service": "",  
  "message": "",  
  "correlation_id": "",  
  "metadata": {}  
}
```

23.9 Correlation ID Requirement

Every request generates:

Correlation ID

Every log entry includes:

Correlation ID

Purpose:

Workflow tracing

Cross-service debugging

Incident investigation

23.10 Application Logs

Purpose:

Business workflow visibility.

Examples:

Opportunity Created

Authority Verified

Evidence Validated

Recommendation Generated

23.11 Security Logs

Purpose:

Security monitoring.

Examples:

Login Success

Login Failure

Permission Denied

Rate Limit Triggered

Security Alert

23.12 Access Logs

Purpose:

API request tracking.

Fields:

Method

Path

Status Code

Duration

User

Correlation ID

23.13 Error Logs

Purpose:

Failure analysis.

Fields:

Exception

Stack Trace Reference

Context

Correlation ID

23.14 Performance Logs

Purpose:

Performance monitoring.

Fields:

Execution Time

Resource Usage

Database Duration

External Call Duration

23.15 Integration Logs

Purpose:

External dependency visibility.

Examples:

LLM Calls

Email Service

Notification Service

Vector Service

23.16 Infrastructure Logs

Purpose:

Platform visibility.

Examples:

Container Start

Container Stop

Health Check Failure

Memory Alert

23.17 Logging Repository Structure

backend/app/logging/

└─ logger.py

|— formatters.py

|— handlers.py

|— filters.py

└— config.py

23.18 Logger Configuration

Environment Specific:

Development

Testing

Production

23.19 Development Logging

Level:

DEBUG

Verbose Output

Enabled

23.20 Testing Logging

Level:

INFO

Focused On:

Validation

Failures

Performance

23.21 Production Logging

Level:

INFO

DEBUG Disabled

Sensitive Data Protected

23.22 Structured Metadata Standard

Metadata Examples:

user_id

entity_id

request_id

correlation_id

execution_time

service_name

23.23 Sensitive Data Rules

Never Log:

Passwords

Tokens

Secrets

Encryption Keys

Personal Sensitive Data

23.24 Exception Logging Rules

Capture:

Exception Type

Error Message

Context

Correlation ID

Avoid:

Sensitive Data

Raw Secrets

23.25 Database Logging

Capture:

Query Duration

Transaction Status

Connection Failures

Do Not Log:

Sensitive Query Data

23.26 API Logging Rules

Capture:

Method

Path

Status

Latency

Correlation ID

Do Not Capture:

Credentials

Secrets

23.27 Authentication Logging

Capture:

Login Success

Login Failure

Logout

Password Change

Token Refresh

23.28 Authorization Logging

Capture:

Permission Checks

Role Checks

Access Denials

Policy Violations

23.29 Retrieval Logging

Capture:

Search Query Hash

Search Method

Execution Duration

Candidate Count

Do Not Store:

Sensitive Search Data

23.30 Ranking Logging

Capture:

Formula Version

Execution ID

Result Count

Duration

23.31 Recommendation Logging

Capture:

Recommendation ID

Generation Time

Evidence Count

Authority Count

23.32 Governance Logging

Capture:

Policy Validation

Approvals

Violations

Compliance Checks

23.33 Audit Integration

Critical Log Events Link To:

Audit Records

↓

Trace Records

↓

Correlation IDs

23.34 Traceability Requirements

Every critical log includes:

Correlation ID

Entity Reference

Version Reference

Workflow Reference

23.35 Retention Policy

Application Logs

90 Days

Security Logs

365 Days

Critical Incident Logs

Permanent Retention

23.36 Log Rotation Policy

Rotation Trigger:

Size

Age

Retention Rule

23.37 Search Capabilities

Logs Searchable By:

Correlation ID

Entity ID

User ID

Date Range

Severity

Service

23.38 Monitoring Integration

Logs Feed:

Dashboards

Alerts

Metrics

Incident Systems

23.39 Alert Generation

Trigger Conditions:

Critical Errors

Authentication Failures

Rate Limit Spikes

Service Failures

Performance Degradation

23.40 Logging Performance Targets

Log Creation

P95 < 5 ms

Log Write

P95 < 10 ms

Log Search

P95 < 500 ms

23.41 Logging Auditability Requirements

Critical Log Events Generate:

Audit Links

↓

Trace Links

↓

Observability Links

23.42 Logging Reproducibility Requirements

Logs Support:

Incident Reconstruction

Workflow Reconstruction

Failure Reconstruction

Execution Reconstruction

23.43 Logging Governance Rules

LOG-GOV-01

Logs must be structured.

LOG-GOV-02

Logs must contain correlation IDs.

LOG-GOV-03

Sensitive data cannot be logged.

LOG-GOV-04

Critical events require logging.

LOG-GOV-05

Security events require logging.

LOG-GOV-06

Retention policies must be enforced.

23.44 Architectural Constraints

AC-LOG-01

Plain text logging is prohibited.

AC-LOG-02

Sensitive information logging is prohibited.

AC-LOG-03

Correlation IDs are mandatory.

AC-LOG-04

Critical workflows require logging.

AC-LOG-05

Logs cannot replace audits.

AC-LOG-06

Governance controls cannot be bypassed.

23.45 Section Summary

This section defines the Logging Architecture, structured logging standards, retention policies, security protections, monitoring integrations, and governance controls.

The logging layer provides operational visibility and incident investigation capabilities while supporting auditability, traceability, observability, and platform reliability.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

- 🔗 Section 3 — Backend Package Design
- 🔗 Section 4 — Frontend Package Design
- 🔗 Section 5 — Database Physical Schema Design
- 🔗 Section 6 — SQLAlchemy Models Design
- 🔗 Section 7 — Alembic Migration Design
- 🔗 Section 8 — API Contract Standards
- 🔗 Section 9 — Authentication APIs
- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design
- 🔗 Section 23 — Logging Architecture

- 🔗 Section 24 — Observability & Monitoring Architecture

TDD V1 — Section 24

Observability & Monitoring Architecture

Document: Technical Design Document (TDD V1)

Section: 24

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

24.1 Purpose

This section defines the Observability and Monitoring Architecture for TODP.

Observability is responsible for:

- System visibility
- Performance monitoring
- Health monitoring
- Failure detection
- Capacity monitoring
- Incident response support

Observability enables operators to understand the current and historical state of the platform.

24.2 Observability Pillars

TODP observability is built on:

Metrics

Logs

Traces

Together they provide:

Monitoring

Alerting

Diagnosis

Root Cause Analysis

24.3 Observability Architecture

Application



Metrics

Logs

Traces



Collection Layer



Storage Layer



Dashboards



Alerting



Operations Team

24.4 Observability Principles

OBS-01

Everything important is measurable.

OBS-02

Everything important is traceable.

OBS-03

Everything important is observable.

OBS-04

Failures are detectable.

OBS-05

Alerts are actionable.

OBS-06

Observability supports governance.

24.5 Monitoring Scope

Application Layer

API Layer

Database Layer

Cache Layer

Infrastructure Layer

Security Layer

Governance Layer

24.6 Metrics Categories

Business Metrics

Application Metrics

Infrastructure Metrics

Security Metrics

Governance Metrics

Performance Metrics

24.7 Business Metrics

Opportunity Count

Authority Count

Evidence Count

Ranking Count

Recommendation Count

Governance Decision Count

24.8 API Metrics

Request Count

Error Rate

Latency

Throughput

Availability

24.9 Authentication Metrics

Login Count

Login Failures

Session Count

Token Refresh Count

Authorization Failures

24.10 Retrieval Metrics

Search Count

Keyword Searches

Semantic Searches

Hybrid Searches

Search Latency

Search Failures

24.11 Ranking Metrics

Ranking Executions

Ranking Latency

Average Candidate Count

Formula Usage

Ranking Failures

24.12 Recommendation Metrics

Recommendation Count

Generation Latency

Generation Failures

Evidence Usage

Authority Usage

24.13 Governance Metrics

Policy Validations

Approvals

Violations

Compliance Rate

Governance Latency

24.14 Audit Metrics

Audit Records Created

Replay Count

Replay Failures

Snapshot Count

Audit Query Count

24.15 Database Metrics

Connection Count

Query Duration

Slow Queries

Deadlocks

Transaction Failures

24.16 Redis Metrics

Cache Hits

Cache Misses

Memory Usage

Evictions

Connection Count

24.17 Infrastructure Metrics

CPU Usage

Memory Usage

Disk Usage

Network Throughput

Container Count

24.18 Health Monitoring

Components:

API

Database

Redis

Vector Store

Background Workers

24.19 Health Endpoint Design

Endpoint:

GET /health

Response:

```
{  
  "status": "healthy"  
}
```

24.20 Readiness Endpoint

Endpoint:

GET /ready

Purpose:

Traffic Acceptance Validation

24.21 Liveness Endpoint

Endpoint:

GET /live

Purpose:

Container Survival Validation

24.22 Distributed Tracing

Every workflow includes:

Trace ID

Span ID

Correlation ID

24.23 Trace Flow Example

API Request



Authentication



Service



Repository



Database



Response

24.24 Trace Metadata

Trace ID

Span ID

Service Name

Operation Name

Duration

Status

24.25 Observability Event Model

Events Generated For:

Errors

Failures

Threshold Breaches

Security Events

Governance Events

24.26 Dashboard Categories

Executive Dashboard

Operations Dashboard

Performance Dashboard

Security Dashboard

Governance Dashboard

24.27 Executive Dashboard Metrics

Availability

Usage

Recommendations Generated

Opportunities Created

Compliance Rate

24.28 Operations Dashboard Metrics

Request Volume

Error Rate

Latency

Infrastructure Health

Cache Performance

24.29 Security Dashboard Metrics

Authentication Failures

Permission Denials

Rate Limit Violations

Security Alerts

24.30 Governance Dashboard Metrics

Policy Violations

Approvals

Compliance Status

Governance Events

24.31 Alert Categories

Critical

High

Medium

Low

24.32 Critical Alerts

Database Down

API Unavailable

Authentication Failure Spike

Security Breach Indicators

Data Corruption Indicators

24.33 High Priority Alerts

High Error Rate

Cache Failure

Performance Degradation

Worker Failure

24.34 Medium Priority Alerts

Slow Queries

Latency Increases

Capacity Thresholds

Memory Growth

24.35 Low Priority Alerts

Configuration Warnings

Maintenance Notifications

Capacity Trends

24.36 Alert Routing

Critical



Immediate Notification

High



Operations Team

Medium



Monitoring Queue

Low



Dashboard Only

24.37 SLO Definitions

Availability

99.9%

API Success Rate

99.5%

Search Success Rate

99%

Recommendation Success Rate

99%

24.38 SLA Support Metrics

Uptime

Latency

Error Rate

Recovery Time

Availability

24.39 Incident Support

Observability Must Support:

Detection

Investigation

Containment

Recovery

Postmortem Analysis

24.40 Capacity Monitoring

Monitor:

CPU

Memory

Storage

Database Growth

Cache Growth

Request Growth

24.41 Auditability Requirements

Observability Events Link To:

Audit Records



Trace Records



Correlation IDs

24.42 Traceability Requirements

Store:

Trace ID

Correlation ID

Entity Reference

Workflow Reference

Version Reference

24.43 Reproducibility Requirements

Observability Supports:

Workflow Replay

Incident Reconstruction

Performance Reconstruction

Failure Reconstruction

24.44 Performance Targets

Metrics Collection

P95 < 10 ms

Trace Generation

P95 < 5 ms

Dashboard Queries

P95 < 2 Seconds

24.45 Observability Governance Rules

OBS-GOV-01

Critical workflows must be observable.

OBS-GOV-02

Critical workflows must be traceable.

OBS-GOV-03

Critical failures must generate alerts.

OBS-GOV-04

Metrics retention must be enforced.

OBS-GOV-05

Dashboard visibility is mandatory.

OBS-GOV-06

Observability data supports governance.

24.46 Architectural Constraints

AC-OBS-01

Critical systems cannot operate without observability.

AC-OBS-02

Correlation IDs are mandatory.

AC-OBS-03

Tracing cannot be disabled for critical workflows.

AC-OBS-04

Health monitoring is mandatory.

AC-OBS-05

Alerting is mandatory.

AC-OBS-06

Governance controls cannot be bypassed.

24.47 Section Summary

This section defines the Observability and Monitoring Architecture, monitoring framework, metrics model, tracing strategy, dashboard design, alerting model, and governance controls.

The observability layer provides complete visibility into TODP operations, enabling monitoring, troubleshooting, governance validation, incident response, performance optimization, and long-term operational reliability.

TDD Progress

📄 Section 1 — Document Overview & Technical Scope

📄 Section 2 — Repository Technical Structure

📄 Section 3 — Backend Package Design

- 🔗 Section 4 — Frontend Package Design
- 🔗 Section 5 — Database Physical Schema Design
- 🔗 Section 6 — SQLAlchemy Models Design
- 🔗 Section 7 — Alembic Migration Design
- 🔗 Section 8 — API Contract Standards
- 🔗 Section 9 — Authentication APIs
- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design
- 🔗 Section 23 — Logging Architecture
- 🔗 Section 24 — Observability & Monitoring Architecture
- 🔗 Section 25 — Security Architecture

TDD V1 — Section 25

Security Architecture

Document: Technical Design Document (TDD V1)

Section: 25

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

25.1 Purpose

This section defines the Security Architecture for TODP.

Security is responsible for:

- Identity protection
- Access control
- Data protection
- Infrastructure protection
- API protection
- Governance enforcement

Security must be built into every layer of the platform.

25.2 Security Objectives

SEC-01

Protect platform assets.

SEC-02

Protect user identities.

SEC-03

Protect data integrity.

SEC-04

Protect availability.

SEC-05

Prevent unauthorized access.

SEC-06

Support compliance and governance.

25.3 Security Layers

User Layer



Authentication Layer



Authorization Layer



API Layer



Application Layer



Data Layer



Infrastructure Layer

25.4 Security Principles

Least Privilege

Defense In Depth

Zero Trust

Secure Defaults

Auditability

Traceability

25.5 Identity Management

Identity Components:

Users

Roles

Permissions

Sessions

Tokens

25.6 Authentication Strategy

Method:

JWT Authentication

Components:

Access Token

Refresh Token

Session Tracking

25.7 Access Token Security

Algorithm:

HS256

(or approved replacement)

Expiration:

60 Minutes

Signed

Validated

Audited

25.8 Refresh Token Security

Expiration:

30 Days

Stored Securely

Revocable

Tracked

Audited

25.9 Authorization Model

Role-Based Access Control (RBAC)

User



Role



Permission



Action

25.10 Core Roles

User

Reviewer

Administrator

Governance Administrator

Security Administrator

System Administrator

25.11 Permission Categories

Read

Create

Update

Delete

Verify

Approve

Manage

Audit

25.12 Session Security

Session Tracking Required

Stored:

Session ID

User

Device Metadata

Expiration

Status

25.13 Password Policy

Minimum Length:

12 Characters

Complexity Required

Password Reuse Prevented

25.14 Password Storage

Hashing Algorithm:

bcrypt

Plaintext Storage:

Prohibited

25.15 API Security

Controls:

Authentication

Authorization

Validation

Rate Limiting

Audit Logging

25.16 Input Validation Security

Validate:

Length

Type

Format

Range

Enums

UUIDs

25.17 Output Security

Prevent:

Sensitive Data Leakage

Internal Error Exposure

Debug Information Exposure

25.18 Data Classification

Public

Internal

Confidential

Restricted

25.19 Data Protection Rules

Confidential Data



Protected

Restricted Data



Strongest Controls

25.20 Encryption In Transit

Protocol:

TLS 1.3

Required For:

All External Communications

25.21 Encryption At Rest

Protected Assets:

Database

Backups

Secrets

Stored Tokens

25.22 Secret Management

Managed Separately From Code

Examples:

JWT Secrets

Database Passwords

API Keys

Encryption Keys

25.23 Secrets Storage Rules

Never Store:

Secrets In Source Code

Secrets In Git

Secrets In Logs

Secrets In Error Messages

25.24 Rate Limiting

User Rate Limit

IP Rate Limit

Authentication Rate Limit

Administrative API Rate Limit

25.25 Security Headers

Required Headers:

HSTS

Content-Security-Policy

X-Frame-Options

X-Content-Type-Options

Referrer-Policy

25.26 CSRF Protection

Required For:

State-Changing Operations

When Applicable

25.27 CORS Policy

Explicit Origins Only

No Wildcards In Production

25.28 SQL Injection Protection

Controls:

ORM Usage

Parameterized Queries

Validation

Input Sanitization

25.29 XSS Protection

Controls:

Output Encoding

Validation

Content Security Policy

25.30 SSRF Protection

Controls:

URL Validation

Network Restrictions

Allow Lists

25.31 Dependency Security

Requirements:

Dependency Scanning

Vulnerability Monitoring

Patch Management

25.32 Container Security

Requirements:

Minimal Images

Security Scanning

Least Privilege Runtime

Image Signing

25.33 Infrastructure Security

Protect:

Servers

Containers

Networks

Storage

Secrets

25.34 Database Security

Controls:

Authentication

Authorization

Encryption

Backups

Audit Logging

25.35 Redis Security

Controls:

Authentication

Network Restrictions

Encryption

TTL Enforcement

25.36 Audit Security

Security Events Logged:

Authentication

Authorization

Permission Changes

Governance Actions

Administrative Actions

25.37 Security Monitoring

Monitor:

Authentication Failures

Authorization Failures

Rate Limit Violations

Privilege Escalation Attempts

Suspicious Activity

25.38 Incident Response Categories

Security Incident

Data Incident

Access Incident

Infrastructure Incident

Compliance Incident

25.39 Security Event Categories

INFO

WARNING

ALERT

CRITICAL

25.40 Security Testing Requirements

Unit Security Tests

Integration Security Tests

Permission Tests

Authentication Tests

Authorization Tests

25.41 Penetration Testing

Required Before:

Major Releases

Architecture Changes

Security Changes

25.42 Compliance Support

Support:

Auditability

Traceability

Governance Validation

Policy Enforcement

25.43 Security Auditability Requirements

Security Actions Generate:

Audit Records

↓

Trace Records

↓

Correlation IDs

25.44 Security Traceability Requirements

Store:

Actor

Action

Timestamp

Entity

Correlation ID

Result

25.45 Security Reproducibility Requirements

Support:

Incident Reconstruction

Access Reconstruction

Permission Reconstruction

Authentication Replay

25.46 Security Governance Rules

SEC-GOV-01

Authentication is mandatory.

SEC-GOV-02

Authorization is mandatory.

SEC-GOV-03

Encryption in transit is mandatory.

SEC-GOV-04

Secrets cannot be stored in source code.

SEC-GOV-05

Security events must be audited.

SEC-GOV-06

Security monitoring is mandatory.

25.47 Architectural Constraints

AC-SEC-01

Anonymous access to protected APIs is prohibited.

AC-SEC-02

Plaintext password storage is prohibited.

AC-SEC-03

Plaintext secret storage is prohibited.

AC-SEC-04

Authorization cannot be bypassed.

AC-SEC-05

Audit logging cannot be bypassed.

AC-SEC-06

Governance controls cannot be bypassed.

25.48 Section Summary

This section defines the Security Architecture, identity management model, authentication framework, authorization framework, encryption strategy, infrastructure protections, monitoring requirements, and governance controls.

The security layer protects TODP assets, users, data, workflows, and infrastructure while ensuring auditability, traceability, compliance support, and operational resilience.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

🔗 Section 10 — Opportunity APIs

🔗 Section 11 — Authority APIs

🔗 Section 12 — Evidence APIs

🔗 Section 13 — Retrieval APIs

🔗 Section 14 — Ranking APIs

- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design
- 🔗 Section 23 — Logging Architecture
- 🔗 Section 24 — Observability & Monitoring Architecture
- 🔗 Section 25 — Security Architecture

- 🔗 Section 26 — Testing Architecture

TDD V1 — Section 26

Testing Architecture

Document: Technical Design Document (TDD V1)

Section: 26

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

26.1 Purpose

This section defines the Testing Architecture for TODP.

Testing is responsible for:

- Functional correctness

- Regression prevention

- Quality assurance

- Security validation

- Performance validation

- Release confidence

Testing is a mandatory activity throughout the software lifecycle.

26.2 Testing Principles

TEST-01

Every critical component is testable.

TEST-02

Testing is automated whenever possible.

TEST-03

Testing occurs at multiple layers.

TEST-04

Regression prevention is mandatory.

TEST-05

Test results are auditable.

TEST-06

Quality gates are enforced.

26.3 Testing Pyramid

End-to-End Tests



Integration Tests



Service Tests



Unit Tests

Unit tests form the largest layer.

26.4 Test Categories

Unit Tests

Integration Tests

API Tests

Repository Tests

Security Tests

Performance Tests

End-to-End Tests

Regression Tests

26.5 Test Repository Structure

backend/tests/

├─ unit/

├─ integration/

├─ api/

├─ repositories/

├─ security/

├─ performance/

├─ e2e/

└─ regression/

26.6 Frontend Test Structure

frontend/tests/

└─ unit/

└─ integration/

└─ components/

└─ pages/

└─ e2e/

26.7 Unit Testing Purpose

Validate:

Functions

Utilities

Services

Validators

Business Rules

26.8 Unit Testing Rules

Isolated Execution

No External Dependencies

Deterministic Results

Fast Execution

26.9 Service Layer Testing

Validate:

Business Rules

Domain Workflows

Validation Logic

Policy Enforcement

Exception Handling

26.10 Repository Testing

Validate:

CRUD Operations

Transactions

Constraints

Indexes

Pagination

Filtering

26.11 API Testing

Validate:

Endpoints

Request Validation

Response Validation

Authentication

Authorization

Error Handling

26.12 Integration Testing

Validate:

API ↔ Service

Service ↔ Repository

Repository ↔ Database

Database ↔ Redis

Cross-Module Workflows

26.13 Authentication Testing

Validate:

Login

Logout

Token Refresh

Session Management

Password Changes

Authorization Checks

26.14 Opportunity Testing

Validate:

Creation

Updates

Deletion

Search

Filtering

Pagination

26.15 Authority Testing

Validate:

Authority Creation

Verification

Confidence Calculation

Status Transitions

26.16 Evidence Testing

Validate:

Evidence Creation

Validation

Versioning

Lineage

Historical Retrieval

26.17 Retrieval Testing

Validate:

Keyword Search

Semantic Search

Hybrid Search

Fusion Logic

Candidate Generation

26.18 Ranking Testing

Validate:

Formula Execution

Tie Resolution

Ranking Order

Historical Replay

Version Support

26.19 Recommendation Testing

Validate:

Generation Workflow

Dependency Validation

Explanation Generation

Evidence Linking

Authority Linking

26.20 Governance Testing

Validate:

Policy Validation

Approval Workflows

Compliance Reporting

Violation Detection

26.21 Audit Testing

Validate:

Audit Generation

Replay

Snapshots

Correlation Tracking

Historical Reconstruction

26.22 Security Testing

Validate:

Authentication

Authorization

Input Validation

Rate Limiting

Secret Protection

26.23 Performance Testing

Validate:

API Latency

Search Latency

Ranking Latency

Recommendation Latency

Database Performance

26.24 Load Testing

Simulate:

Concurrent Users

Concurrent Searches

Concurrent Rankings

Concurrent Recommendations

26.25 Stress Testing

Validate:

System Limits

Failure Recovery

Resource Exhaustion

Graceful Degradation

26.26 End-to-End Testing

Validate Complete Workflows:

Authentication

↓

Retrieval

↓

Ranking



Recommendation



Audit

26.27 Regression Testing

Purpose:

Prevent Reintroduction Of Defects

Executed:

Every Release

Every Major Change

26.28 Test Data Strategy

Use:

Synthetic Data

Controlled Fixtures

Versioned Datasets

Avoid:

Production Data

Sensitive Data

26.29 Test Environment Strategy

Development

Testing

Staging

Production Validation

Each Environment Isolated

26.30 Mocking Strategy

Allowed For:

External Services

Email Services

Notification Services

Future LLM Providers

Not Preferred For:

Core Domain Logic

26.31 Test Coverage Targets

Unit Tests

≥ 80%

Service Layer

≥ 85%

Critical Modules

≥ 90%

26.32 Test Execution Strategy

Developer Machine

↓

Pull Request



CI Pipeline



Release Validation

26.33 Automated Testing

Executed Automatically:

Unit Tests

Integration Tests

API Tests

Security Tests

Regression Tests

26.34 Manual Testing

Used For:

Exploratory Testing

UX Validation

Release Sign-Off

26.35 Test Reporting

Reports Include:

Passed Tests

Failed Tests

Coverage

Execution Time

Trend History

26.36 Failure Handling

Test Failure



Pipeline Failure



Alert



Resolution Required

26.37 Test Metadata

Store:

Test Run ID

Version

Timestamp

Environment

Result

26.38 Observability Validation

Validate:

Metrics

Logs

Traces

Alerts

Health Checks

26.39 Auditability Requirements

Test Runs Generate:

Test Event



Audit Record



Execution Metadata

26.40 Traceability Requirements

Store:

Test Run

Version

Environment

Correlation ID

Execution Time

26.41 Reproducibility Requirements

Support:

Test Replay

Environment Replay

Dataset Replay

Regression Replay

26.42 Performance Targets

Unit Test Suite

< 5 Minutes

Integration Suite

< 15 Minutes

Regression Suite

< 30 Minutes

26.43 Testing Governance Rules

TEST-GOV-01

Critical features require automated tests.

TEST-GOV-02

Regression testing is mandatory.

TEST-GOV-03

Security testing is mandatory.

TEST-GOV-04

Coverage thresholds must be enforced.

TEST-GOV-05

Release validation is mandatory.

TEST-GOV-06

Test history must be preserved.

26.44 Architectural Constraints

AC-TEST-01

Production deployment without testing is prohibited.

AC-TEST-02

Failed critical tests block releases.

AC-TEST-03

Coverage gates cannot be bypassed.

AC-TEST-04

Security tests cannot be bypassed.

AC-TEST-05

Regression suites must remain executable.

AC-TEST-06

Governance controls cannot be bypassed.

26.45 Section Summary

This section defines the Testing Architecture, testing strategy, quality gates, coverage targets, automation framework, validation workflows, and governance controls.

The testing layer provides confidence that TODP remains correct, secure, reliable, performant, auditable, traceable, and production-ready throughout its lifecycle.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

- 🔗 Section 4 — Frontend Package Design
- 🔗 Section 5 — Database Physical Schema Design
- 🔗 Section 6 — SQLAlchemy Models Design
- 🔗 Section 7 — Alembic Migration Design
- 🔗 Section 8 — API Contract Standards
- 🔗 Section 9 — Authentication APIs
- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design
- 🔗 Section 23 — Logging Architecture
- 🔗 Section 24 — Observability & Monitoring Architecture
- 🔗 Section 25 — Security Architecture
- 🔗 Section 26 — Testing Architecture

- 🔗 Section 27 — CI/CD Architecture

TDD V1 — Section 27

CI/CD Architecture

Document: Technical Design Document (TDD V1)

Section: 27

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

27.1 Purpose

This section defines the Continuous Integration and Continuous Deployment (CI/CD) Architecture for TODP.

CI/CD is responsible for:

- Build automation
- Quality enforcement
- Security validation
- Deployment automation

- Release governance
- Environment consistency

CI/CD ensures reliable and repeatable software delivery.

27.2 CI/CD Principles

CICD-01

Every change is validated.

CICD-02

Every build is reproducible.

CICD-03

Every deployment is traceable.

CICD-04

Every release is auditable.

CICD-05

Quality gates are mandatory.

CICD-06

Rollback capability is mandatory.

27.3 Pipeline Architecture

Developer Commit



Source Control



CI Pipeline



Quality Gates



Artifact Creation



CD Pipeline



Deployment



Validation



Production

27.4 Source Control Strategy

Platform:

Git

Branch Types:

main

develop

feature/*

hotfix/*

release/*

27.5 Pull Request Workflow

Developer Branch

↓

Pull Request

↓

Code Review

↓

Automated Validation



Approval



Merge

27.6 CI Pipeline Stages

Stage 1

Source Checkout

Stage 2

Dependency Installation

Stage 3

Static Analysis

Stage 4

Testing

Stage 5

Security Validation

Stage 6

Artifact Build

27.7 Static Analysis Stage

Checks:

Linting

Formatting

Typing

Code Quality

Import Validation

27.8 Code Quality Tools

Python:

Ruff

Black

MyPy

Frontend:

ESLint

TypeScript

Prettier

27.9 Unit Testing Stage

Execute:

Backend Tests

Frontend Tests

Utility Tests

Service Tests

Coverage Threshold:

80% Minimum

27.10 Integration Testing Stage

Validate:

API Workflows

Database Integration

Redis Integration

Repository Layer

Authentication Flow

27.11 Security Validation Stage

Checks:

Dependency Scanning

Secret Detection

Container Scanning

Configuration Validation

27.12 Build Stage

Generate:

Backend Artifact

Frontend Artifact

Docker Images

Build Metadata

27.13 Artifact Requirements

Artifacts Must Be:

Versioned

Immutable

Traceable

Reproducible

27.14 Container Build Process

Source

↓

Docker Build

↓

Security Scan

↓

Image Tagging

↓

Registry Push

27.15 Image Tagging Standard

Format:

application:version

application:commit-hash

application:latest

27.16 Artifact Repository

Stores:

Docker Images

Build Metadata

Release Metadata

Version Information

27.17 CD Pipeline Stages

Stage 1

Environment Validation

Stage 2

Database Migration

Stage 3

Application Deployment

Stage 4

Health Validation

Stage 5

Smoke Testing

Stage 6

Release Approval

27.18 Deployment Environments

Development

Testing

Staging

Production

27.19 Environment Promotion Flow

Development

↓

Testing



Staging



Production

27.20 Database Migration Stage

Execute:

Alembic Migrations

Validate:

Migration Success

Rollback Availability

Schema Consistency

27.21 Deployment Strategy

Preferred:

Rolling Deployment

Alternative:

Blue-Green Deployment

27.22 Rolling Deployment Flow

Deploy New Instance

↓

Health Check

↓

Route Traffic



Remove Old Instance

27.23 Health Validation

Checks:

API Health

Database Health

Redis Health

Search Health

Worker Health

27.24 Smoke Testing

Validate:

Authentication

Opportunity APIs

Authority APIs

Evidence APIs

Search APIs

Recommendation APIs

27.25 Release Approval

Required For:

Production Deployment

Major Releases

Security Changes

Governance Changes

27.26 Rollback Strategy

Trigger Conditions:

Deployment Failure

Health Failure

Security Failure

Critical Defect

27.27 Rollback Workflow

Identify Failure

↓

Rollback Deployment

↓

Validate Health



Restore Service



Generate Incident Record

27.28 Pipeline Configuration Storage

Stored In:

Source Control

Version Controlled

Auditable

27.29 Build Metadata

Stored Data:

Build ID

Commit Hash

Timestamp

Version

Environment

27.30 Deployment Metadata

Stored Data:

Deployment ID

Version

Environment

Timestamp

Operator

27.31 Release Versioning

Format:

MAJOR.MINOR.PATCH

Examples:

1.0.0

1.1.0

1.1.1

27.32 Infrastructure Validation

Validate:

Configuration

Secrets

Networking

Storage

Resource Limits

27.33 Observability Integration

Pipeline Reports:

Build Metrics

Deployment Metrics

Failure Metrics

Rollback Metrics

27.34 Auditability Requirements

Every deployment generates:

Deployment Event



Audit Record



Trace Record



Correlation ID

27.35 Traceability Requirements

Store:

Commit Hash

Build ID

Deployment ID

Release Version

Operator

Environment

27.36 Reproducibility Requirements

Support:

Build Reproduction

Release Reproduction

Deployment Replay

Rollback Replay

27.37 Performance Targets

CI Execution

< 15 Minutes

Deployment Validation

< 10 Minutes

Rollback Execution

< 5 Minutes

27.38 Failure Handling

Pipeline Failure

↓

Stop Progression

↓

Generate Alert



Create Audit Event



Require Resolution

27.39 Governance Gates

Required Before Production:

Code Review

Security Validation

Testing Validation

Migration Validation

Approval

27.40 Security Controls

Pipeline Must Validate:

Secrets

Dependencies

Containers

Configuration

Permissions

27.41 Compliance Requirements

Maintain:

Release History

Deployment History

Approval History

Rollback History

27.42 CI/CD Governance Rules

CICD-GOV-01

All deployments must be automated.

CICD-GOV-02

Production releases require approval.

CICD-GOV-03

Security validation is mandatory.

CICD-GOV-04

Testing validation is mandatory.

CICD-GOV-05

Rollback capability is mandatory.

CICD-GOV-06

Deployment history must be preserved.

27.43 Architectural Constraints

AC-CICD-01

Direct production changes are prohibited.

AC-CICD-02

Pipeline validation cannot be bypassed.

AC-CICD-03

Security checks cannot be bypassed.

AC-CICD-04

Deployment approvals cannot be bypassed.

AC-CICD-05

Rollback procedures must remain available.

AC-CICD-06

Governance controls cannot be bypassed.

27.44 Section Summary

This section defines the CI/CD Architecture, build pipeline, deployment pipeline, release workflow, rollback strategy, validation framework, and governance controls.

The CI/CD layer provides automated, auditable, traceable, reproducible, and secure software delivery across all TODP environments.

TDD Progress

🔗 Section 1 — Document Overview & Technical Scope

🔗 Section 2 — Repository Technical Structure

🔗 Section 3 — Backend Package Design

🔗 Section 4 — Frontend Package Design

🔗 Section 5 — Database Physical Schema Design

🔗 Section 6 — SQLAlchemy Models Design

🔗 Section 7 — Alembic Migration Design

🔗 Section 8 — API Contract Standards

🔗 Section 9 — Authentication APIs

- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design
- 🔗 Section 23 — Logging Architecture
- 🔗 Section 24 — Observability & Monitoring Architecture
- 🔗 Section 25 — Security Architecture
- 🔗 Section 26 — Testing Architecture
- 🔗 Section 27 — CI/CD Architecture

- 🔗 Section 28 — Deployment Architecture

TDD V1 — Section 28

Deployment Architecture

Document: Technical Design Document (TDD V1)

Section: 28

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)
- IAD V1 (Frozen)

28.1 Purpose

This section defines the Deployment Architecture for TODP.

Deployment Architecture is responsible for:

- Runtime hosting
- Service deployment
- Infrastructure topology
- Environment management
- Scalability support
- Operational reliability

The deployment architecture transforms the application into a production-operational platform.

28.2 Deployment Principles

DEP-01

Infrastructure is reproducible.

DEP-02

Deployments are automated.

DEP-03

Services are isolated.

DEP-04

Scaling is supported.

DEP-05

Recovery is supported.

DEP-06

Deployments are auditable.

28.3 High-Level Deployment Topology

Users



Load Balancer



Frontend Application



Backend API



Redis



PostgreSQL



Observability Stack

28.4 Deployment Environments

Development

Testing

Staging

Production

Each environment remains isolated.

28.5 Environment Purpose

Development

Feature Development

Testing

Automated Validation

Staging

Production Simulation

Production

Live Operations

28.6 Containerization Strategy

All deployable components use:

Docker

Benefits:

Consistency

Portability

Isolation

Reproducibility

28.7 Deployable Services

Frontend Service

Backend API Service

Redis Service

PostgreSQL Service

Background Worker Service

Observability Service

28.8 Frontend Deployment

Technology:

React

Served Through:

Nginx

Deployment Unit:

Docker Container

28.9 Backend Deployment

Technology:

FastAPI

Server:

Uvicorn

Deployment Unit:

Docker Container

28.10 Database Deployment

Technology:

PostgreSQL

Deployment Model:

Managed Database Preferred

Persistent Storage Required

28.11 Redis Deployment

Technology:

Redis

Purpose:

Caching

Rate Limiting

Session Storage

28.12 Worker Deployment

Purpose:

Background Processing

Examples:

Index Updates

Maintenance Jobs

Notification Jobs

Future Async Tasks

28.13 Observability Deployment

Components:

Metrics Collection

Log Aggregation

Tracing

Dashboards

28.14 Network Topology

Internet



Load Balancer



Frontend Network



Backend Network



Database Network

28.15 Network Isolation Rules

Public Access:

Frontend

Load Balancer

Private Access:

Database

Redis

Internal Services

28.16 Load Balancer Responsibilities

Traffic Distribution

TLS Termination

Health Checks

Routing

Availability

28.17 TLS Configuration

Minimum Version:

TLS 1.3

HTTPS Mandatory

28.18 Backend Scaling Strategy

Scale Horizontally

Multiple API Instances Supported

Stateless Service Design

28.19 Frontend Scaling Strategy

Scale Horizontally

Multiple Frontend Instances Supported

28.20 Database Scaling Strategy

Primary Database

↓

Read Replicas

(Optional)

Future Scaling Support

28.21 Redis Scaling Strategy

Single Node Initially

Cluster Support Planned

28.22 Storage Requirements

Persistent Storage:

Database

Audit Data

Snapshots

Backups

28.23 Backup Architecture

Database Backup



Secure Storage



Recovery Validation

28.24 Backup Frequency

Database

Daily

Audit Data

Daily

Configuration

Daily

28.25 Disaster Recovery Objectives

RPO:

< 24 Hours

RTO:

< 4 Hours

28.26 Infrastructure as Code

All Infrastructure Defined As:

Code

Version Controlled

Auditable

28.27 Deployment Workflow

Build Artifact

↓

Container Build

↓

Security Validation

↓

Registry Push



Deployment



Health Validation

28.28 Service Discovery

Services Communicate Through:

Configured Endpoints

Future Service Discovery Supported

28.29 Health Monitoring

Every Service Exposes:

/health

/live

/ready

28.30 Health Check Requirements

Validate:

Service Availability

Database Connectivity

Redis Connectivity

Critical Dependencies

28.31 Resource Allocation

Every Service Defines:

CPU Limits

Memory Limits

Storage Requirements

28.32 Secret Management

Store Separately From:

Source Code

Container Images

Logs

Examples:

Database Credentials

JWT Secrets

API Keys

28.33 Deployment Metadata

Store:

Deployment ID

Version

Environment

Timestamp

Operator

28.34 Deployment Auditability

Every Deployment Generates:

Deployment Event

↓

Audit Record



Trace Record



Correlation ID

28.35 Deployment Traceability

Store:

Version

Environment

Build ID

Deployment ID

Operator

Commit Hash

28.36 Deployment Reproducibility

Support:

Environment Recreation

Deployment Replay

Version Recreation

Rollback Replay

28.37 Rollback Strategy

Rollback Supported For:

Application Versions

Configuration Versions

Infrastructure Versions

28.38 Deployment Security

Controls:

TLS

Authentication

Authorization

Network Isolation

Secret Management

Audit Logging

28.39 Operational Monitoring

Monitor:

API Availability

Database Health

Redis Health

Resource Utilization

Error Rates

Latency

28.40 Capacity Planning

Track:

Traffic Growth

Database Growth

Storage Growth

Cache Growth

User Growth

28.41 Availability Target

Platform Availability

99.9%

28.42 Reliability Requirements

Support:

Failure Recovery

Restart Recovery

Rollback Recovery

Backup Recovery

28.43 Governance Rules

DEP-GOV-01

Deployments must be automated.

DEP-GOV-02

Infrastructure must be reproducible.

DEP-GOV-03

Deployment history must be preserved.

DEP-GOV-04

Rollback support is mandatory.

DEP-GOV-05

Health validation is mandatory.

DEP-GOV-06

Backups are mandatory.

28.44 Architectural Constraints

AC-DEP-01

Direct production modification is prohibited.

AC-DEP-02

Infrastructure changes must be version controlled.

AC-DEP-03

Secrets cannot exist in source code.

AC-DEP-04

Database must remain private.

AC-DEP-05

Deployment validation cannot be bypassed.

AC-DEP-06

Governance controls cannot be bypassed.

28.45 Section Summary

This section defines the Deployment Architecture, runtime topology, infrastructure layout, scaling strategies, backup architecture, disaster recovery objectives, deployment workflows, and governance controls.

The deployment layer provides the operational foundation that enables TODP to run reliably, securely, reproducibly, and at scale across all environments.

TDD Progress

🔗 [Section 1 — Document Overview & Technical Scope](#)

🔗 [Section 2 — Repository Technical Structure](#)

🔗 [Section 3 — Backend Package Design](#)

🔗 [Section 4 — Frontend Package Design](#)

🔗 [Section 5 — Database Physical Schema Design](#)

🔗 [Section 6 — SQLAlchemy Models Design](#)

🔗 [Section 7 — Alembic Migration Design](#)

🔗 [Section 8 — API Contract Standards](#)

🔗 [Section 9 — Authentication APIs](#)

🔗 [Section 10 — Opportunity APIs](#)

🔗 [Section 11 — Authority APIs](#)

🔗 [Section 12 — Evidence APIs](#)

🔗 [Section 13 — Retrieval APIs](#)

🔗 [Section 14 — Ranking APIs](#)

🔗 [Section 15 — Recommendation APIs](#)

🔗 [Section 16 — Governance APIs](#)

🔗 [Section 17 — Audit APIs](#)

🔗 [Section 18 — DTO & Schema Design](#)

🔗 [Section 19 — Domain Service Interfaces](#)

🔗 [Section 20 — Repository Interfaces](#)

🔗 Section 21 — Event Contracts

🔗 Section 22 — Redis Key Design

🔗 Section 23 — Logging Architecture

🔗 Section 24 — Observability & Monitoring Architecture

🔗 Section 25 — Security Architecture

🔗 Section 26 — Testing Architecture

🔗 Section 27 — CI/CD Architecture

🔗 Section 28 — Deployment Architecture

🔗 Section 29 — Operational Runbooks & Incident Management

TDD V1 — Section 29

Operational Runbooks & Incident Management

Document: Technical Design Document (TDD V1)

Section: 29

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

29.1 Purpose

This section defines the Operational Runbooks and Incident Management Architecture for TODP.

Operational Runbooks are responsible for:

- Operational procedures
- Incident response
- Service recovery
- Escalation workflows
- Operational consistency
- Governance compliance

Runbooks provide repeatable operational guidance.

29.2 Operational Principles

OPS-01

Operations must be documented.

OPS-02

Incidents must be traceable.

OPS-03

Recovery procedures must be reproducible.

OPS-04

Critical incidents require escalation.

OPS-05

Incident history must be preserved.

OPS-06

Operational actions must be auditable.

29.3 Operational Scope

Application Operations

Infrastructure Operations

Database Operations

Security Operations

Governance Operations

Recovery Operations

29.4 Runbook Categories

Deployment Runbooks

Database Runbooks

Security Runbooks

Recovery Runbooks

Incident Runbooks

Maintenance Runbooks

29.5 Runbook Repository Structure

docs/runbooks/

└─ deployment/

└─ database/

└─ security/

└─ recovery/

└─ incident/

└─ maintenance/

29.6 Incident Severity Levels

SEV-1

Critical

SEV-2

High

SEV-3

Medium

SEV-4

Low

29.7 SEV-1 Definition

Examples:

Complete Platform Outage

Database Failure

Security Breach

Data Corruption

Authentication Failure

29.8 SEV-2 Definition

Examples:

Major Feature Failure

Search Failure

Recommendation Failure

Deployment Failure

29.9 SEV-3 Definition

Examples:

Performance Degradation

Partial Feature Failure

Cache Failure

Observability Failure

29.10 SEV-4 Definition

Examples:

Minor Defects

Documentation Issues

Non-Critical Alerts

29.11 Incident Lifecycle

Detection



Classification



Investigation



Containment



Resolution



Validation



Postmortem

29.12 Detection Sources

Monitoring Alerts

Logs

Metrics

Traces

User Reports

Security Alerts

29.13 Incident Record Structure

Fields:

incident_id

severity

status

summary

detected_at

resolved_at

correlation_id

29.14 Incident Status Values

OPEN

INVESTIGATING

MITIGATING

RESOLVED

CLOSED

29.15 Escalation Matrix

SEV-1

Immediate Escalation

SEV-2

Operations Escalation

SEV-3

Team Escalation

SEV-4

Normal Queue

29.16 Communication Requirements

Communicate:

Incident Status

Impact

Recovery Progress

Resolution

Postmortem Findings

29.17 Platform Outage Runbook

Detect Outage



Validate Scope



Check Infrastructure



Check Database



Check Application



Restore Service



Validate Recovery

29.18 Database Failure Runbook

Detect Failure



Validate Connectivity



Validate Storage



Restore Backup If Needed



Validate Data Integrity



Restore Service

29.19 Redis Failure Runbook

Detect Failure



Switch To Database Reads



Restart Redis



Restore Cache



Validate Operations

29.20 Authentication Failure Runbook

Validate Authentication Service



Validate Token Service



Validate Sessions



Restore Authentication



Validate Login Flow

29.21 Search Failure Runbook

Validate Search Service



Validate Embeddings



Validate Database



Validate Retrieval APIs



Restore Search

29.22 Ranking Failure Runbook

Validate Ranking Engine



Validate Inputs



Validate Formula Version



Restore Ranking Service

29.23 Recommendation Failure Runbook

Validate Ranking Dependency

↓

Validate Authority Dependency

↓

Validate Evidence Dependency

↓

Restore Recommendation Engine

29.24 Security Incident Runbook

Contain Threat

↓

Preserve Evidence



Restrict Access



Investigate



Remediate



Validate Security



Postmortem

29.25 Data Corruption Runbook

Identify Scope



Preserve Current State



Restore Backup



Validate Integrity



Restore Operations

29.26 Deployment Failure Runbook

Stop Deployment



Rollback Release



Validate Health



Restore Service



Create Incident Record

29.27 Backup Recovery Runbook

Identify Recovery Point



Restore Backup



Validate Data



Validate Application



Resume Operations

29.28 Maintenance Runbook

Notify Stakeholders



Prepare Environment



Execute Maintenance



Validate Systems



Close Maintenance Window

29.29 Operational Checklists

Pre-Deployment Checklist

Post-Deployment Checklist

Recovery Checklist

Security Checklist

Maintenance Checklist

29.30 Postmortem Requirements

Required For:

SEV-1

SEV-2

Major Governance Violations

Security Incidents

29.31 Postmortem Structure

Summary

Timeline

Impact

Root Cause

Resolution

Lessons Learned

Actions

29.32 Root Cause Analysis

Required Inputs:

Logs

Metrics

Traces

Audit Records

Incident Timeline

29.33 Recovery Validation

Validate:

Health Endpoints

Authentication

Search

Ranking

Recommendations

Audit Systems

29.34 Incident Metrics

Incident Count

Resolution Time

Recovery Time

Incident Recurrence

Severity Distribution

29.35 Recovery Objectives

SEV-1

Immediate Response

SEV-2

Rapid Response

SEV-3

Scheduled Resolution

SEV-4

Normal Resolution

29.36 Auditability Requirements

Every incident generates:

Incident Record

↓

Audit Record



Trace Record



Correlation ID

29.37 Traceability Requirements

Store:

Timeline

Actions Taken

Operators

Affected Components

Recovery Metadata

29.38 Reproducibility Requirements

Support:

Incident Replay

Recovery Replay

Deployment Replay

Root Cause Reconstruction

29.39 Operational Monitoring Integration

Runbooks Use:

Logs

Metrics

Traces

Alerts

Dashboards

29.40 Security Requirements

Operational Actions Require:

Authentication

Authorization

Audit Logging

Role Validation

29.41 Governance Rules

OPS-GOV-01

Critical incidents require postmortems.

OPS-GOV-02

Incident history must be preserved.

OPS-GOV-03

Recovery procedures must be documented.

OPS-GOV-04

Operational actions must be auditable.

OPS-GOV-05

Security incidents require escalation.

OPS-GOV-06

Recovery validation is mandatory.

29.42 Architectural Constraints

AC-OPS-01

Critical incidents cannot bypass documentation.

AC-OPS-02

Incident records cannot be deleted.

AC-OPS-03

Recovery validation cannot be skipped.

AC-OPS-04

Security incidents require investigation.

AC-OPS-05

Operational actions require traceability.

AC-OPS-06

Governance controls cannot be bypassed.

29.43 Section Summary

This section defines the Operational Runbook Architecture, incident lifecycle, escalation framework, recovery procedures, postmortem requirements, and governance controls.

The operational layer provides structured and repeatable procedures for maintaining reliability, recovering from failures, managing incidents, preserving accountability, and ensuring operational resilience across TODP.

TDD Progress

- 🔗 [Section 1 — Document Overview & Technical Scope](#)
- 🔗 [Section 2 — Repository Technical Structure](#)
- 🔗 [Section 3 — Backend Package Design](#)
- 🔗 [Section 4 — Frontend Package Design](#)
- 🔗 [Section 5 — Database Physical Schema Design](#)
- 🔗 [Section 6 — SQLAlchemy Models Design](#)
- 🔗 [Section 7 — Alembic Migration Design](#)
- 🔗 [Section 8 — API Contract Standards](#)
- 🔗 [Section 9 — Authentication APIs](#)
- 🔗 [Section 10 — Opportunity APIs](#)
- 🔗 [Section 11 — Authority APIs](#)
- 🔗 [Section 12 — Evidence APIs](#)
- 🔗 [Section 13 — Retrieval APIs](#)
- 🔗 [Section 14 — Ranking APIs](#)
- 🔗 [Section 15 — Recommendation APIs](#)
- 🔗 [Section 16 — Governance APIs](#)
- 🔗 [Section 17 — Audit APIs](#)
- 🔗 [Section 18 — DTO & Schema Design](#)
- 🔗 [Section 19 — Domain Service Interfaces](#)
- 🔗 [Section 20 — Repository Interfaces](#)
- 🔗 [Section 21 — Event Contracts](#)
- 🔗 [Section 22 — Redis Key Design](#)
- 🔗 [Section 23 — Logging Architecture](#)
- 🔗 [Section 24 — Observability & Monitoring Architecture](#)
- 🔗 [Section 25 — Security Architecture](#)

🔗 Section 26 — Testing Architecture

🔗 Section 27 — CI/CD Architecture

🔗 Section 28 — Deployment Architecture

🔗 Section 29 — Operational Runbooks & Incident Management

🔗 Section 30 — Technical Risks, Assumptions & Future Roadmap

TDD V1 — Section 30

Technical Risks, Assumptions & Future Roadmap

Document: Technical Design Document (TDD V1)

Section: 30

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

- IAD V1 (Frozen)

30.1 Purpose

This section defines the technical risks, assumptions, constraints, future evolution strategy, and roadmap for TODP.

The purpose is to:

- Identify architectural risks
- Document assumptions
- Capture known constraints
- Guide future evolution
- Preserve decision rationale
- Support long-term maintainability

30.2 Strategic Design Position

TODP V1 is designed as:

A deterministic opportunity decision platform.

Core priorities:

Traceability

Auditability

Governance

Reproducibility

Transparency

Operational Reliability

30.3 Major Architectural Assumptions

ASSUMP-01

Users require transparent recommendations.

ASSUMP-02

Ranking must remain deterministic.

ASSUMP-03

Evidence validation remains mandatory.

ASSUMP-04

Authority verification remains mandatory.

ASSUMP-05

Auditability remains a platform requirement.

ASSUMP-06

Governance remains a first-class concern.

30.4 Infrastructure Assumptions

PostgreSQL remains primary datastore.

Redis remains cache layer.

Docker remains deployment standard.

FastAPI remains backend framework.

React remains frontend framework.

30.5 Scalability Assumptions

V1 assumes:

Moderate traffic volumes.

Single-region deployment.

Single primary database.

Redis single-node deployment.

30.6 Security Assumptions

TLS is always available.

Secrets are externally managed.

Authentication remains mandatory.

Authorization remains mandatory.

30.7 Data Assumptions

Data quality is not guaranteed.

Evidence quality varies.

Authority quality varies.

Validation processes mitigate risk.

30.8 Governance Assumptions

Governance policies evolve over time.

Policy versions must be preserved.

Historical governance records remain available.

30.9 Technical Constraints

Constraint 1

Deterministic ranking required.

Constraint 2

Audit history preserved indefinitely.

Constraint 3

Recommendation transparency mandatory.

Constraint 4

Reproducibility mandatory.

Constraint 5

Traceability mandatory.

30.10 Operational Constraints

Limited operational team.

Automated operations preferred.

Automated deployment preferred.

Automated recovery support preferred.

30.11 Risk Classification Model

LOW

MEDIUM

HIGH

CRITICAL

30.12 Risk Area: Data Quality

Risk:

Poor evidence quality.

Impact:

Incorrect recommendations.

Classification:

HIGH

Mitigation:

Evidence validation.

Versioning.

Governance review.

30.13 Risk Area: Authority Quality

Risk:

Incorrect authority verification.

Impact:

Recommendation degradation.

Classification:

HIGH

Mitigation:

Verification workflow.

Confidence scoring.

Auditability.

30.14 Risk Area: Search Relevance

Risk:

Poor retrieval quality.

Impact:

Missed opportunities.

Classification:

MEDIUM

Mitigation:

Hybrid retrieval.

Evaluation framework.

Continuous tuning.

30.15 Risk Area: Ranking Formula Evolution

Risk:

Formula becomes insufficient.

Impact:

Recommendation quality decline.

Classification:

MEDIUM

Mitigation:

Versioned formulas.

Replay testing.

Governance approval.

30.16 Risk Area: Governance Complexity

Risk:

Governance overhead increases.

Impact:

Operational slowdown.

Classification:

MEDIUM

Mitigation:

Workflow automation.

Policy automation.

Tooling support.

30.17 Risk Area: Security Events

Risk:

Credential compromise.

Impact:

Unauthorized access.

Classification:

CRITICAL

Mitigation:

RBAC.

Audit logging.

Monitoring.

Secret management.

30.18 Risk Area: Infrastructure Failure

Risk:

Database outage.

Impact:

Platform unavailable.

Classification:

CRITICAL

Mitigation:

Backups.

Recovery procedures.

Monitoring.

Failover planning.

30.19 Risk Area: Dependency Failure

Risk:

External component failure.

Impact:

Feature degradation.

Classification:

MEDIUM

Mitigation:

Isolation.

Fallback procedures.

Operational monitoring.

30.20 Risk Area: Human Error

Risk:

Operational mistakes.

Impact:

Deployment failures.

Configuration failures.

Classification:

MEDIUM

Mitigation:

CI/CD controls.

Automation.

Approvals.

Runbooks.

30.21 Future Evolution Principles

EVOLVE-01

Backward compatibility preferred.

EVOLVE-02

Version everything.

EVOLVE-03

Preserve auditability.

EVOLVE-04

Preserve traceability.

EVOLVE-05

Preserve reproducibility.

EVOLVE-06

Governance remains mandatory.

30.22 Roadmap Phase 1

Current Scope

TODP V1

Capabilities:

CRUD

Authority Validation

Evidence Validation

Hybrid Retrieval

Ranking

Recommendations

Governance

Auditability

30.23 Roadmap Phase 2

Enhanced Retrieval

Potential Features:

Advanced Search

Query Expansion

Improved Semantic Retrieval

Advanced Fusion Algorithms

30.24 Roadmap Phase 3

Advanced Recommendation Layer

Potential Features:

Personalization

Adaptive Ranking

Scenario Analysis

Comparative Recommendations

30.25 Roadmap Phase 4

Workflow Automation

Potential Features:

Automated Governance

Automated Validation

Workflow Rules Engine

Task Automation

30.26 Roadmap Phase 5

Advanced Analytics

Potential Features:

Decision Analytics

Governance Analytics

Trend Analysis

Historical Intelligence

30.27 Roadmap Phase 6

Multi-Region Architecture

Potential Features:

Regional Deployments

Geo Redundancy

Advanced Failover

Global Scaling

30.28 Future Search Evolution

Current:

Hybrid Retrieval

Future:

Knowledge Graph Retrieval

Multi-Stage Retrieval

Advanced Re-Ranking

30.29 Future Ranking Evolution

Current:

Total Cost Formula

Future:

Versioned Ranking Models

Scenario-Based Ranking

Policy-Aware Ranking

30.30 Future Recommendation Evolution

Current:

Deterministic Recommendations

Future:

Explanation Enhancements

Decision Simulations

Multi-Scenario Recommendations

30.31 Future Governance Evolution

Current:

Policy Validation

Future:

Policy Automation

Governance Dashboards

Compliance Automation

30.32 Future Audit Evolution

Current:

Replay

Snapshots

Traceability

Future:

Advanced Forensics

Automated Root Cause Analysis

30.33 Future Security Evolution

Current:

RBAC

Audit Logging

Monitoring

Future:

Risk-Based Access

Adaptive Security Controls

Advanced Threat Detection

30.34 Future Observability Evolution

Current:

Metrics

Logs

Traces

Future:

Predictive Monitoring

Anomaly Detection

Automated Remediation

30.35 Technical Debt Strategy

Identify



Document



Prioritize



Remediate



Validate

30.36 Architecture Review Cadence

Quarterly Review

Major Release Review

Governance Review

30.37 Deprecation Strategy

Announce

↓

Support Window

↓

Migration Guidance



Removal

30.38 Long-Term Maintainability Goals

Clear Boundaries

Versioned Contracts

Strong Governance

Automated Validation

Operational Visibility

30.39 Auditability Requirements

Future Changes Must Preserve:

Audit Records

Replay Capability

Historical Reconstruction

30.40 Traceability Requirements

Future Changes Must Preserve:

Correlation Tracking

Version History

Workflow History

Entity Relationships

30.41 Reproducibility Requirements

Future Changes Must Preserve:

Replay Support

Version Replay

Historical Replay

Decision Replay

30.42 Governance Rules

ROADMAP-GOV-01

Architectural evolution must be governed.

ROADMAP-GOV-02

Backward compatibility preferred.

ROADMAP-GOV-03

Critical capabilities require versioning.

ROADMAP-GOV-04

Auditability must be preserved.

ROADMAP-GOV-05

Traceability must be preserved.

ROADMAP-GOV-06

Reproducibility must be preserved.

30.43 Architectural Constraints

AC-FUTURE-01

Future changes cannot bypass governance.

AC-FUTURE-02

Future changes cannot remove auditability.

AC-FUTURE-03

Future changes cannot remove traceability.

AC-FUTURE-04

Future changes cannot remove reproducibility.

AC-FUTURE-05

Critical workflows remain deterministic unless formally approved.

AC-FUTURE-06

Platform principles remain enforceable.

30.44 Final Technical Summary

TODP V1 establishes:

A deterministic, auditable, traceable, reproducible, and governance-driven opportunity decision platform.

The architecture prioritizes:

Evidence

↓

Authority

↓

Retrieval

↓

Ranking



Recommendation



Governance



Auditability

All major platform capabilities are versioned, monitored, secured, testable, observable, and operationally supportable.

30.45 Document Completion Status

TDD V1 Status:

COMPLETE

Total Sections:

30

Document State:

APPROVED

Ready For:

Implementation

Testing

Deployment

Governance Review

Production Planning

TDD Progress

- 🔗 Section 1 — Document Overview & Technical Scope
- 🔗 Section 2 — Repository Technical Structure
- 🔗 Section 3 — Backend Package Design
- 🔗 Section 4 — Frontend Package Design
- 🔗 Section 5 — Database Physical Schema Design
- 🔗 Section 6 — SQLAlchemy Models Design
- 🔗 Section 7 — Alembic Migration Design
- 🔗 Section 8 — API Contract Standards
- 🔗 Section 9 — Authentication APIs
- 🔗 Section 10 — Opportunity APIs
- 🔗 Section 11 — Authority APIs
- 🔗 Section 12 — Evidence APIs
- 🔗 Section 13 — Retrieval APIs
- 🔗 Section 14 — Ranking APIs
- 🔗 Section 15 — Recommendation APIs
- 🔗 Section 16 — Governance APIs
- 🔗 Section 17 — Audit APIs
- 🔗 Section 18 — DTO & Schema Design
- 🔗 Section 19 — Domain Service Interfaces
- 🔗 Section 20 — Repository Interfaces
- 🔗 Section 21 — Event Contracts
- 🔗 Section 22 — Redis Key Design
- 🔗 Section 23 — Logging Architecture
- 🔗 Section 24 — Observability & Monitoring Architecture

📄 Section 25 — Security Architecture

📄 Section 26 — Testing Architecture

📄 Section 27 — CI/CD Architecture

📄 Section 28 — Deployment Architecture

📄 Section 29 — Operational Runbooks & Incident Management

📄 Section 30 — Technical Risks, Assumptions & Future Roadmap

📄 TDD V1 COMPLETE